T.R.

GEBZE TECHNICAL UNIVERSITY

ELM451-FINAL PROJECT

Design and Layout of an 8-Bit ALU Unit

210102002028 - Berkutay Şen

210102002032 - Betül Özçelik

210102002004 - Bilal Savaş

210102002070 – Enes Akkuş

220102002173 – Melih Kaya

DEPARTMENT OF ELECTRONICS ENGINEERING

Table of Contents

1. INTRODUCTION 4
2. RTL-BASED DESIGN OF ALU SUBMODULES 5
   2.1. Arithmetic Processing Units 5
   2.1.1. Adder Design 5
   2.1.2. Subtractor Design 6
   2.1.3. Multiplier Design 6
   2.2. Logical Processing Units 7
   2.2.1. AND 7
   2.2.2. OR 7
   2.2.3. XOR 8
   2.2.4. NOT(Inverter) 8
   2.3. Shifter Units 9
   2.4. Integrated ALU Architecture 10
3. DESIGN OF ALU SUBMODULES 11
   3.1. Arithmetic Unit Design 11
   3.1.1. One-Bit and Eight-Bit Adder Design 11
   3.1.2. Eight-Bit Subtractor 13
   3.1.3. Eight-Bit Multiplier 14
   3.2. Logical Unit Design 16
   3.2.1.AND 16
   3.2.2. OR 17
   3.2.3. XOR 18
   3.2.4. NOT(Inverter) 19
   3.2.5.MUX 20
   3.3. Shifter Unit Design 21
   3.3.1. Right Shifter 25
   3.3.2.Left Shifter 27
   3.4. Control Logic 30
   3.5.Output Unit 34
4. LAYOUT 39
   4.1.Arithmetic Design 39
   4.1.1.Adder 39
   4.1.2.Subtractor 43

4.1.3.Multiplier 45
4.2.Logical Unit 47
4.2.1 AND 47
4.2.2.OR 48
4.2.3.XOR 49
4.2.4.NOT (Inverter) 50
4.3.Shifter Unit 51
4.3.1.Right Shift 51
4.3.2.Left Shift 53
4.4.Opcode 55
4.5.Zero Flag 56
5. SIMULATION RESULTS 57
5.1. Simulation of Arithmetic Sub-Blocks 57
5.1.1. Adder Simulation 57
5.1.2. Subtractor Simulation 59
5.1.3. Multiplier Simulation 61
5.2. Simulation of Logic Sub-Blocks 63
5.2.1 AND Gate Simulation 63
5.2.2 OR Gate Simulation 65
5.2.3 XOR Gate Simulation 67
5.2.4 NOT (Inverter) Simulation 69
5.3. Shifter Unit Simulation 71
6.CONCLUSION 73
CODE 74
Arithmetic Unit 74
Logical Unit 77
Shifter Unit 78
EIGHT-BIT ALU 79
Verification Checklist 80
REFERENCES 81

1. INTRODUCTION

Arithmetic Logic Units (ALUs) form the computational backbone of modern digital systems and microprocessors. They are responsible for executing essential arithmetic and logical operations required for data processing and control tasks. By integrating arithmetic computation with logical decision-making, the ALU plays a fundamental role in determining the overall performance and efficiency of a processor.

Beyond basic arithmetic functions such as addition and subtraction, an ALU is capable of performing logical operations including AND, OR, XOR, and NOT. These operations are indispensable for bit-level manipulation, comparison, and control-flow decisions in digital architectures. In more advanced designs, ALUs may also support multiplication, shifting, and other data transformation operations that enhance computational flexibility.

A conventional ALU architecture is composed of several functional blocks, namely arithmetic units, logic units, and control circuitry. The arithmetic unit handles numerical computations, while the logic unit performs Boolean operations on input data. The control logic interprets the opcode signals and selects the appropriate operation to be executed. This modular structure allows the ALU to operate efficiently while maintaining design scalability.

In this project, an 8-bit ALU is designed and implemented to execute a set of arithmetic and logical operations on two 8-bit input operands. The operation to be performed is determined by control signals, enabling dynamic selection among different functional units. Each submodule of the ALU has been carefully designed, simulated, and verified to ensure correct functionality and reliable performance.

The proposed 8-bit ALU design demonstrates the fundamental principles of digital arithmetic and logic circuit implementation. It also serves as a foundational building block for higher-bit-width systems, highlighting how complex processing units can be constructed from simpler components. Through this design, the importance of modularity, correctness, and efficiency in VLSI system design is clearly illustrated.

2. RTL-BASED DESIGN OF ALU SUBMODULES

2.1. Arithmetic Processing Units

Arithmetic units are responsible for performing fundamental numerical operations such as addition, subtraction, and multiplication. These units form the core of the ALU's computational capability.

2.1.1. Adder Design

The addition operation is implemented using a hierarchical approach, starting from a basic 1-bit full adder and extending to an 8-bit ripple carry adder structure. This methodology ensures design clarity and allows scalable arithmetic construction.

The 1-bit full adder serves as the fundamental building block of multi-bit arithmetic operations. It takes two single-bit inputs and a carry-in signal, producing a sum and a carry-out.

This basic structure enables cascading multiple full adders to construct wider adders. The RTL schematic of the 1-bit full adder is shown in Figure 1.

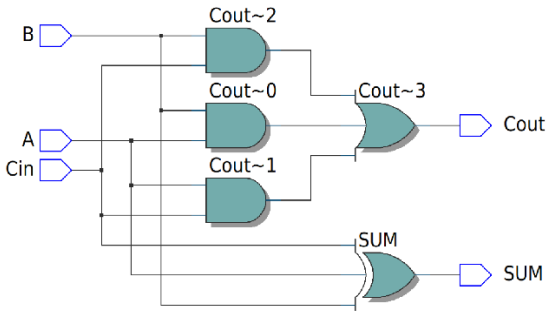


Figure 1. RTL schematic of the 1-bit full adder

The 8-bit adder is constructed by cascading eight 1-bit full adder cells in a ripple carry configuration. In this structure, the carry-out of each stage is connected to the carry-in of the next stage, allowing multi-bit addition.

Although this architecture introduces carry propagation delay, it is simple to implement and well-suited for RTL-level design and educational purposes. The RTL schematic of the 8-bit ripple carry adder is shown in Figure 2.

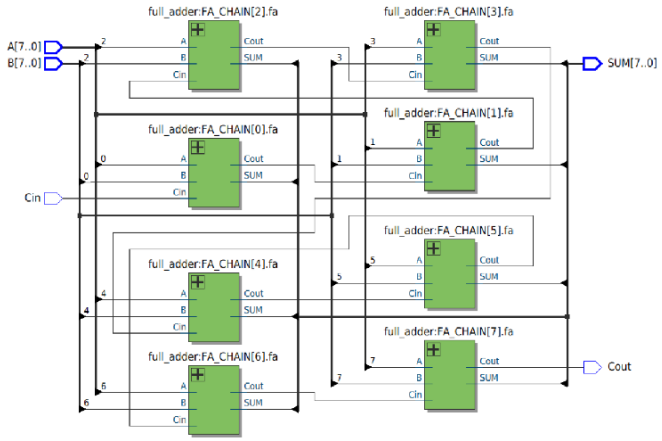


Figure 2. RTL schematic of the 8-bit ripple carry adder

2.1.2. Subtractor Design

Subtraction is implemented using two's complement arithmetic, which allows subtraction to be performed using the same adder hardware. In this approach, the second operand is bitwise inverted and an initial carry-in of logic '1' is applied to the least significant bit.This method reduces hardware redundancy and improves design efficiency. The RTL schematic of the 8-bit subtractor is shown in Figure 3**.**

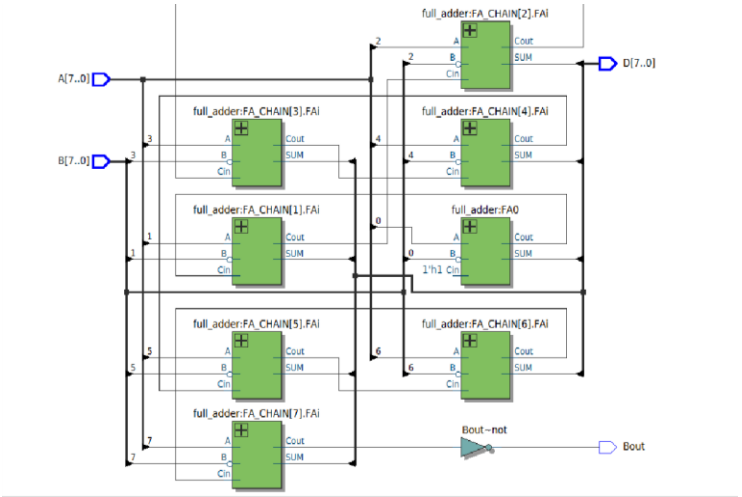


Figure 3. RTL schematic of the 8-bit array subtractor

2.1.3. Multiplier Design

The multiplication operation is realized using an 8-bit array multiplier architecture. Partial products are generated through bitwise AND operations and summed using cascaded adder stages.This design provides a regular and modular structure that is suitable for RTL modeling and further physical implementation. The RTL schematic of the multiplier is shown in Figure 4**.**

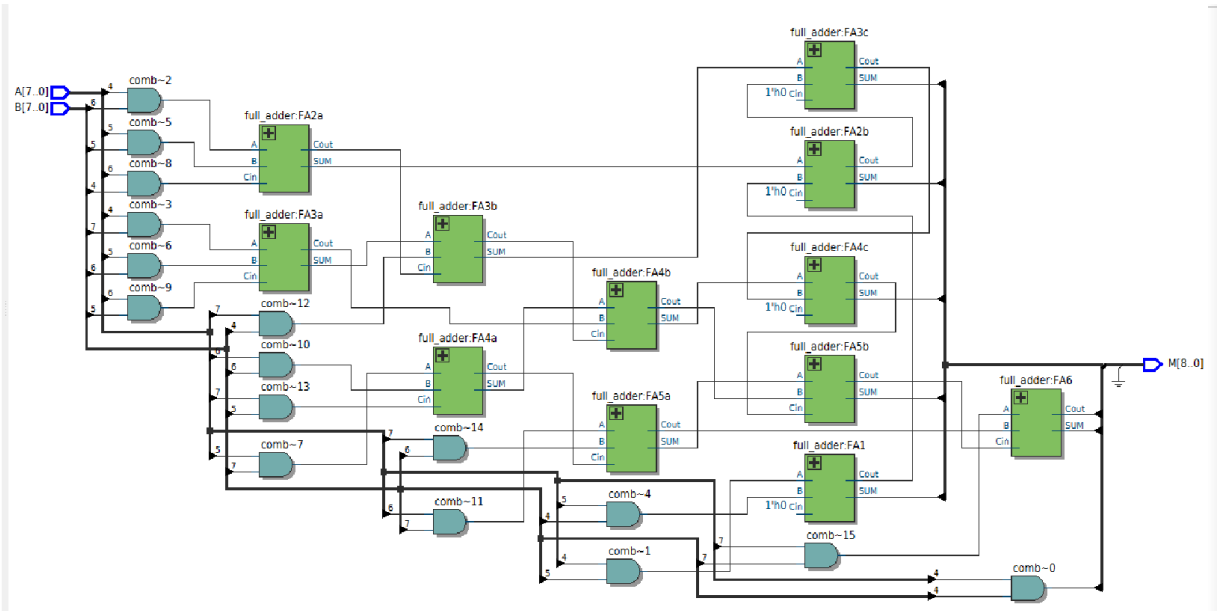


Figure 4. RTL schematic of the 8-bit array multiplier

2.2. Logical Processing Units

Logical units perform bitwise operations that are essential for data manipulation and control logic within the ALU. These operations are implemented in parallel to ensure efficient processing.

2.2.1. AND

The 8-bit AND operation is implemented by applying logical AND operations to corresponding bits of the input operands. Each bit is processed independently, enabling parallel execution.

The RTL schematic of the AND unit is shown in Figure 5.

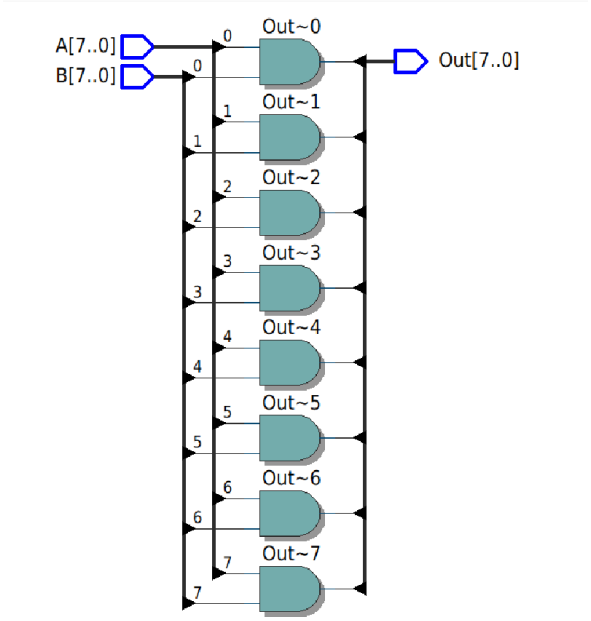


Figure 5. RTL schematic of the 8-bit AND unit

2.2.2. OR

The OR unit generates a logic high output when at least one of the input bits is high. The design follows a parallel structure for all eight bits.

The RTL schematic of the OR unit is shown in Figure 6.

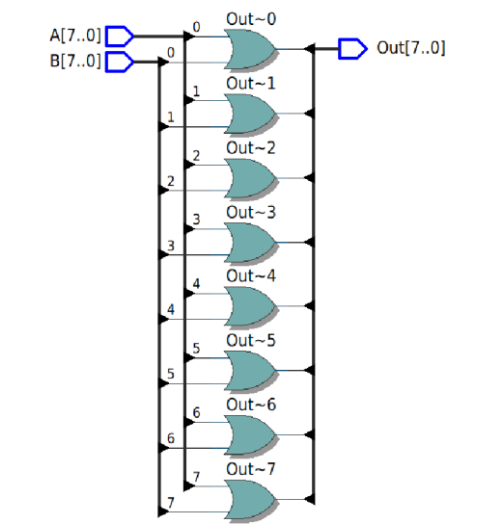


Figure 6. RTL schematic of the 8-bit OR unit

2.2.3. XOR

The XOR unit produces a logic high output when the input bits differ. This operation is commonly used in arithmetic and comparison logic.

The RTL schematic of the 8-bit XOR unit is shown in Figure 7.

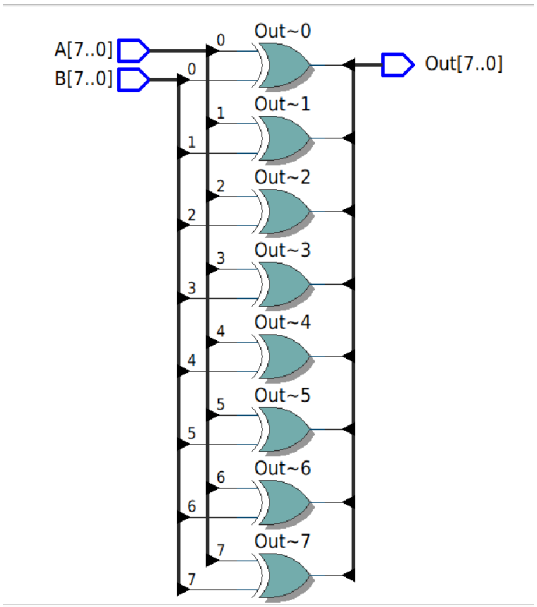


Figure 7. RTL schematic of the 8-bit XOR unit

2.2.4. NOT(Inverter)

The inverter unit performs a bitwise NOT operation on the input vector, generating the logical complement of each bit.The RTL schematic of the inverter unit is shown in Figure 8.

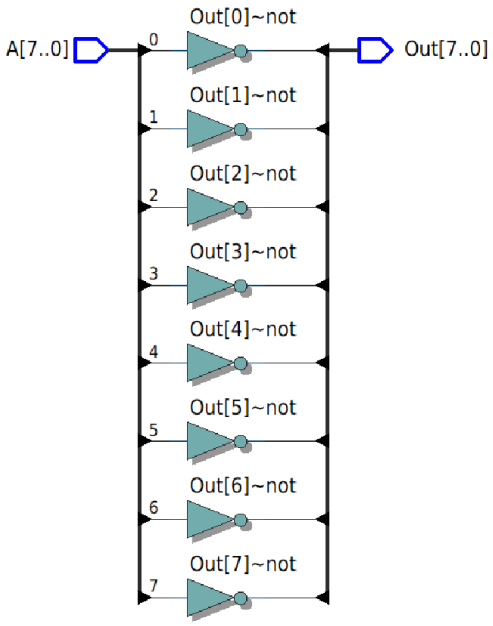


Figure 8. RTL schematic of the 8-bit inverter uni

2.3. Shifter Units

The Shifter Unit enables bit-level shifting of the input data either to the left or to the right. The direction of the shift is controlled by a single control signal. When the control signal is set to logic zero, a left shift operation is performed, whereas a logic one selects the right shift operation. This unified design allows both shifting directions to be handled within a single RTL block, simplifying the overall architecture.

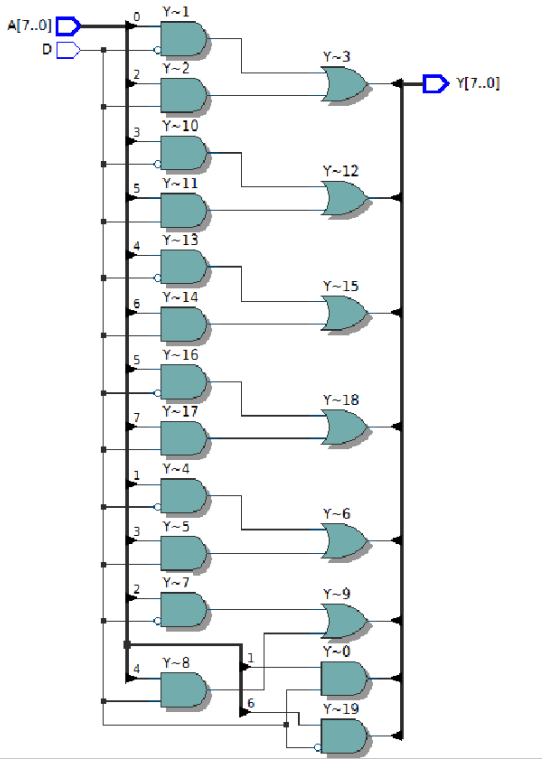


Figure 9. RTL schematic of the 8-bit shifter

2.4. Integrated ALU Architecture

The complete 8-bit ALU is formed by integrating the arithmetic unit, logical unit, shifter unit, control logic, and output selection circuitry. Based on the opcode provided by the control logic, the ALU selects the appropriate functional block and routes its output to the final result bus. This modular integration ensures flexibility, scalability, and clear functional separation within the design. The RTL schematic of the complete ALU illustrates how individual sub-blocks interact to produce the final output.

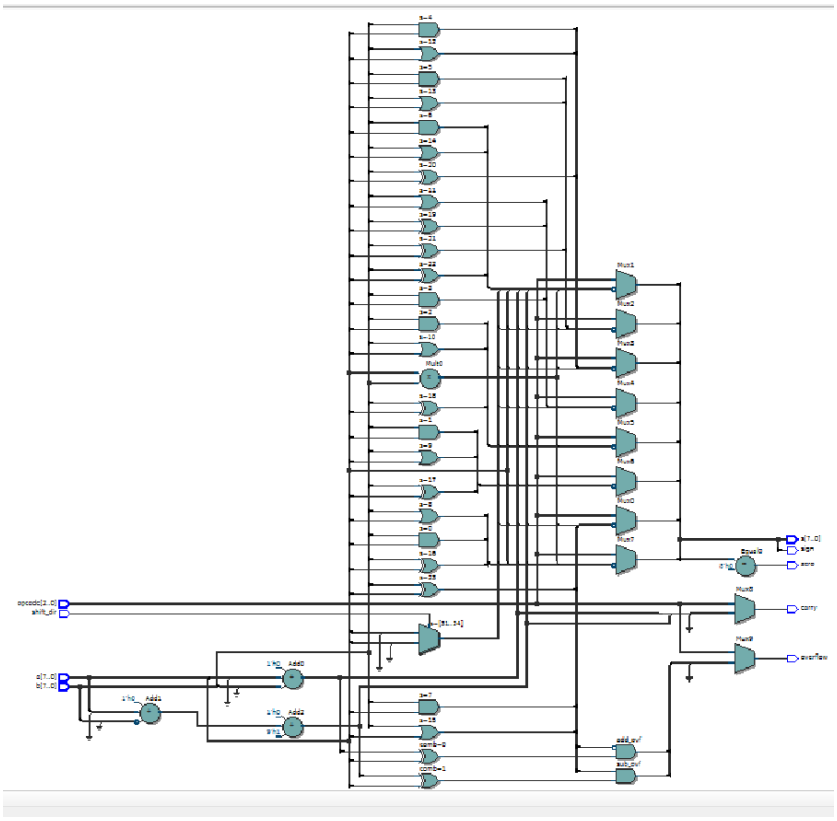


Figure 10. RTL schematic of the ALU

3. DESIGN OF ALU SUBMODULES

3.1. Arithmetic Unit Design

The Arithmetic Unit is responsible for performing fundamental mathematical operations within the ALU. In this design, the Arithmetic Unit supports addition, subtraction, and multiplication. Each operation is implemented using a dedicated transistor-level schematic, ensuring precise control over signal propagation and internal logic behavior.

The arithmetic circuits are constructed hierarchically, beginning with basic building blocks and extending to multi-bit structures. This approach improves scalability and simplifies debugging during the design process.

3.1.1. One-Bit and Eight-Bit Adder Design

The addition operation is implemented using a ripple-carry structure based on a 1-bit full adder. The 1-bit full adder is designed at the transistor level using basic logic gates to compute the sum and carry-out signals from two input bits and an incoming carry. The schematic of the 1-bit full adder is shown in Figure 11.

After validating the correctness of the 1-bit full adder, an 8-bit adder is constructed by cascading eight identical full adder blocks. In this ripple-carry configuration, the carry-out of each stage is directly connected to the carry-in of the subsequent stage, allowing multi-bit addition to be performed across the entire input word. Although this structure introduces a carry propagation delay, it provides a simple, reliable, and area-efficient solution suitable for an 8-bit ALU.

The schematic of the complete 8-bit adder is presented in Figure 12, where the repeated structure of the full adders and the carry chain can be clearly observed.

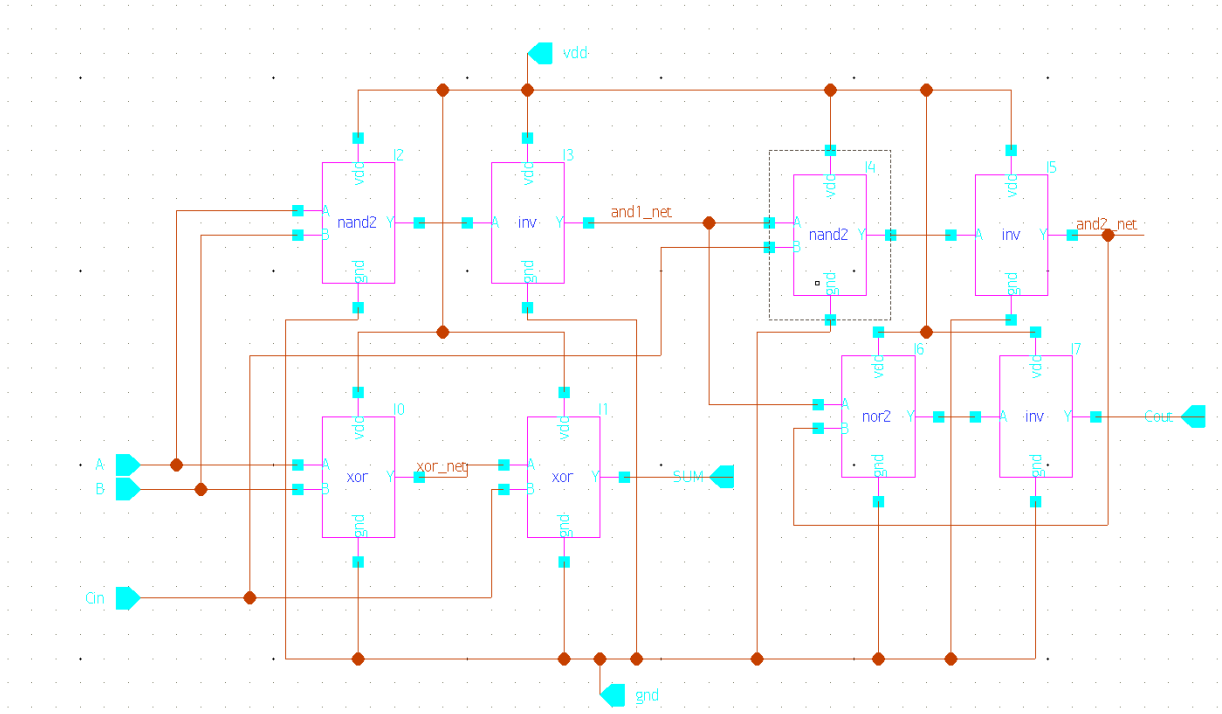


Figure 11. One-Bit Adder Schematic

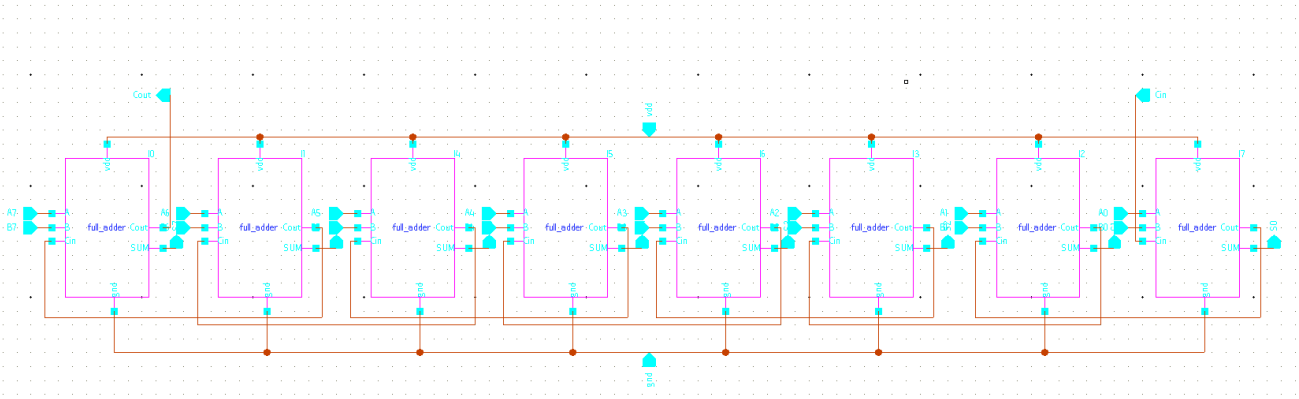


Figure 12. Eight- Bit Adder Schematic

3.1.2. Eight-Bit Subtractor

The subtraction operation is implemented by modifying the adder structure using two's complement arithmetic. In this approach, the second operand is first inverted, and an initial carry-in is applied to perform subtraction using the same ripple-carry adder topology.

The 8-bit subtractor is constructed by reusing the full adder blocks, ensuring consistency with the adder design and reducing overall circuit complexity. The borrow information is derived from the final carry-out signal, allowing correct subtraction results across all bit positions.

The transistor-level schematic of the 8-bit subtractor is shown in Figure 13, where the inversion of the second operand and the carry propagation mechanism are explicitly visible.

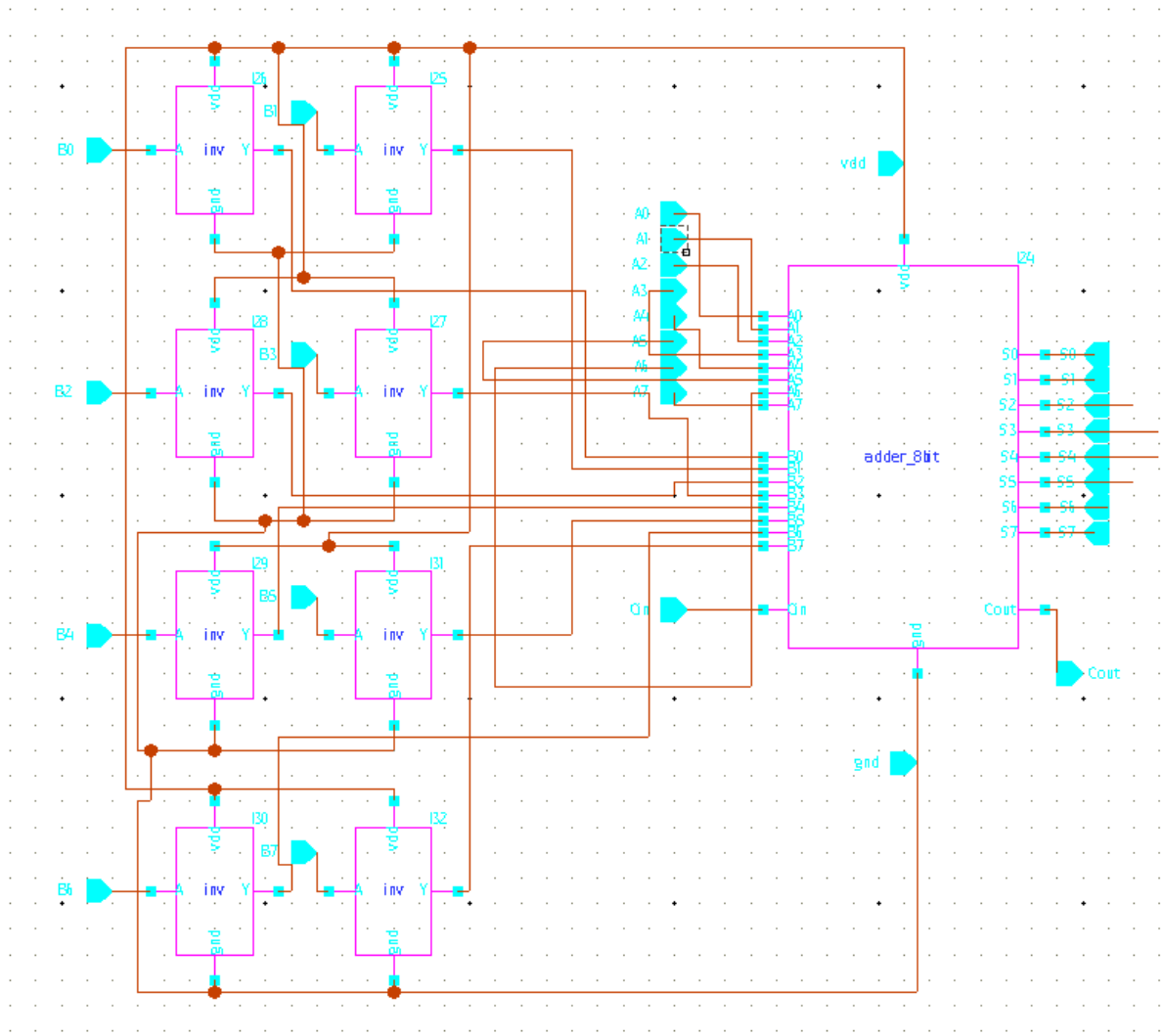


Figure13. Eight- Bit Subtractor Schematic

3.1.3. Eight-Bit Multiplier

The multiplication operation is implemented using an array-based combinational multiplier structure. Partial products are generated by performing bitwise AND operations between selected bits of the input operands. These partial products are then summed using multiple full adder stages arranged in a structured manner.

In this design, the multiplier operates on selected portions of the input operands to generate an 8-bit output. The use of parallel partial product generation enables efficient computation while maintaining a clear schematic layout. The summation network ensures correct accumulation of intermediate results to produce the final multiplication output.

The schematic of the 8-bit multiplier is illustrated in Figure 14, which demonstrates the hierarchical organization of partial products and adder stages.

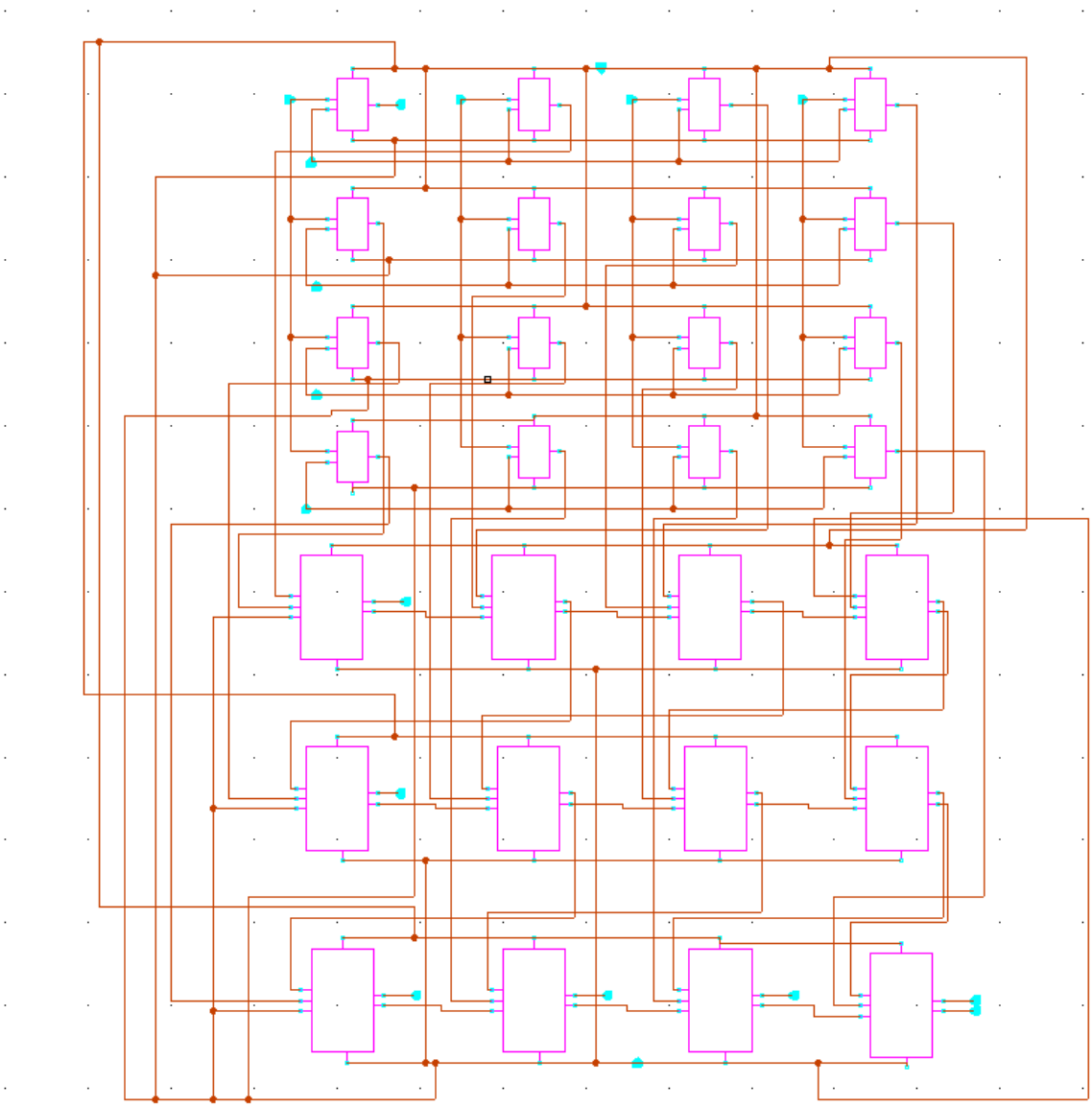


Figure 14. Eight- Bit Multiplier Schematic

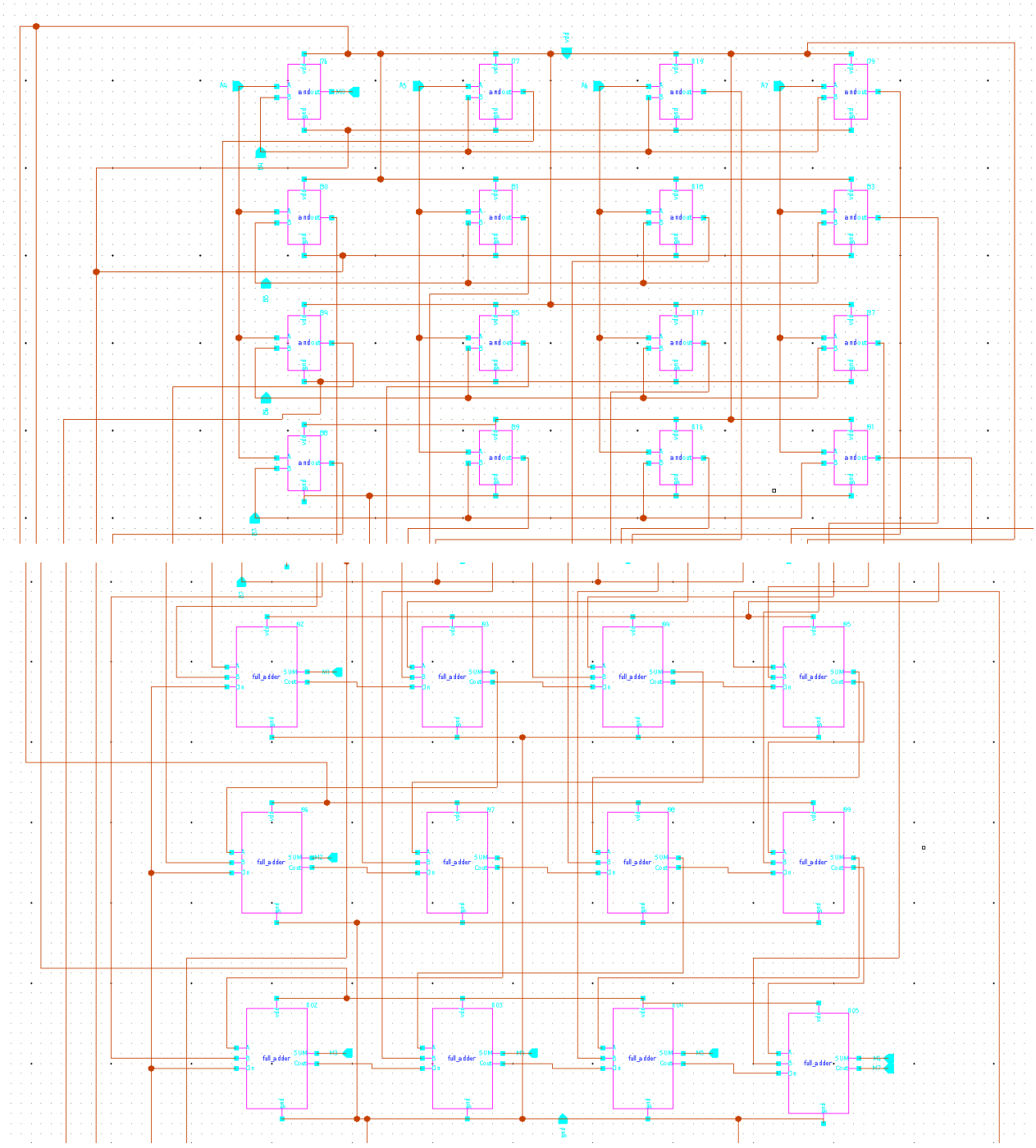


Figure 15.Zoomed Eight- Bit Multiplier Schematic

3.2. Logical Unit Design

The Logical Unit performs bitwise logical operations on the input operands. All logical functions are implemented using transistor-level schematic designs and operate independently on each bit of the input data. This ensures uniform behavior across the entire word length.

Instead of designing a single monolithic logic block, each logical operation is constructed from a basic 1-bit circuit and then expanded to an 8-bit structure by parallel replication. This design philosophy improves modularity and simplifies verification.

3.2.1.AND

The AND operation is implemented using a basic 1-bit AND gate designed at the transistor level. This gate produces a logic-high output only when both input signals are high. The 1-bit AND schematic serves as the fundamental building block for the 8-bit AND unit.

To perform an 8-bit AND operation, eight identical 1-bit AND gates are connected in parallel, each operating on the corresponding bit positions of the input operands. The schematic of the 1-bit AND gate is shown in Figure 15, while the complete 8-bit AND schematic is presented in Figure 16.

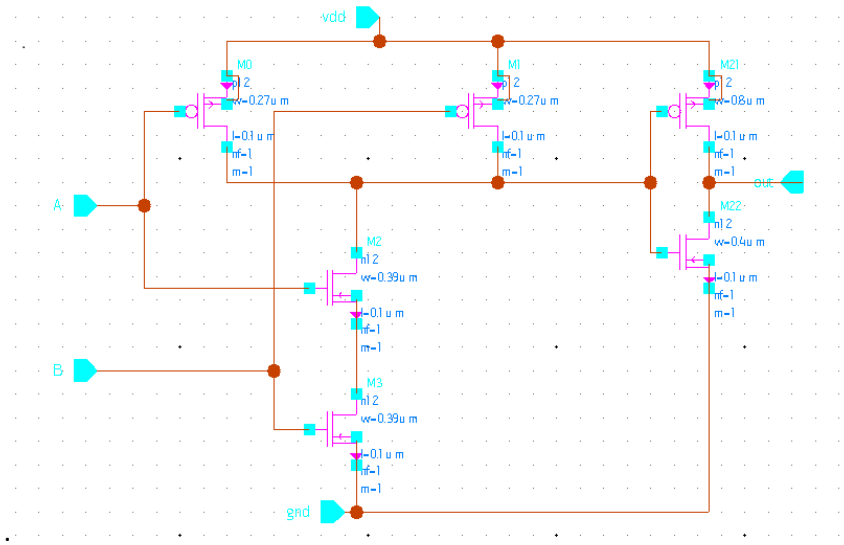


Figure 15.One-Bit AND Schematic

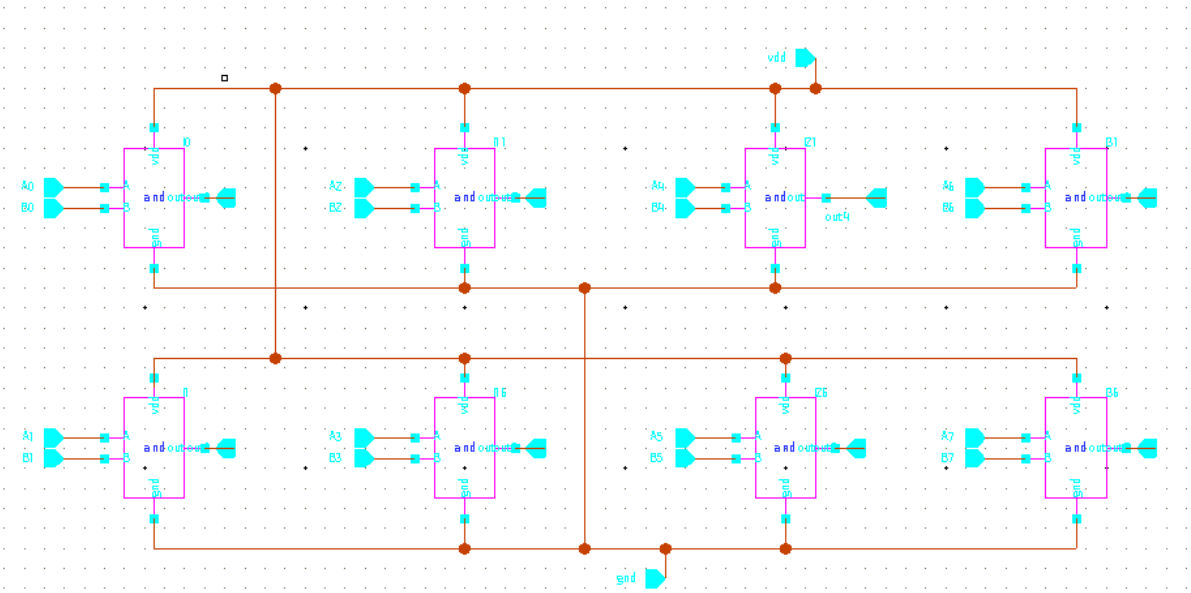


Figure 16. Eight- Bit AND Schematic

3.2.2. OR

The OR operation is designed using a 1-bit OR gate implemented at the transistor level. This gate produces a logic-high output when at least one of the input signals is high. Similar to the AND unit, the OR operation is extended to 8 bits by replicating the 1-bit OR gate structure across all bit positions.

The transistor-level schematic of the 1-bit OR gate is shown in Figure 17, and the corresponding 8-bit OR unit is illustrated in Figure 18.

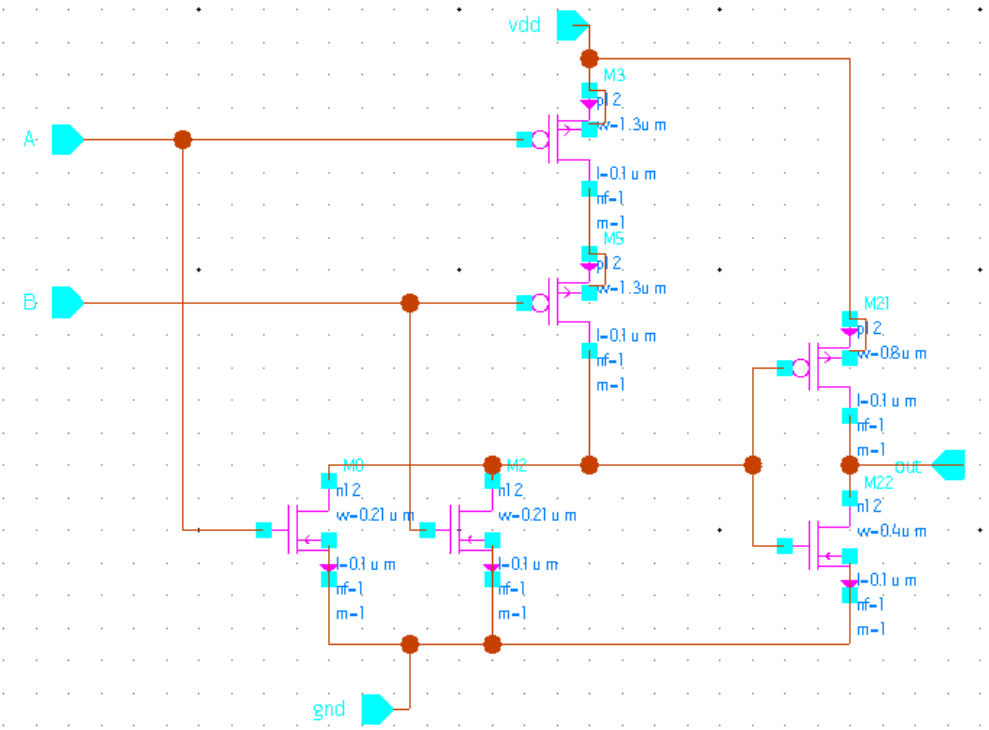


Figure 17.One-Bit OR Schematic

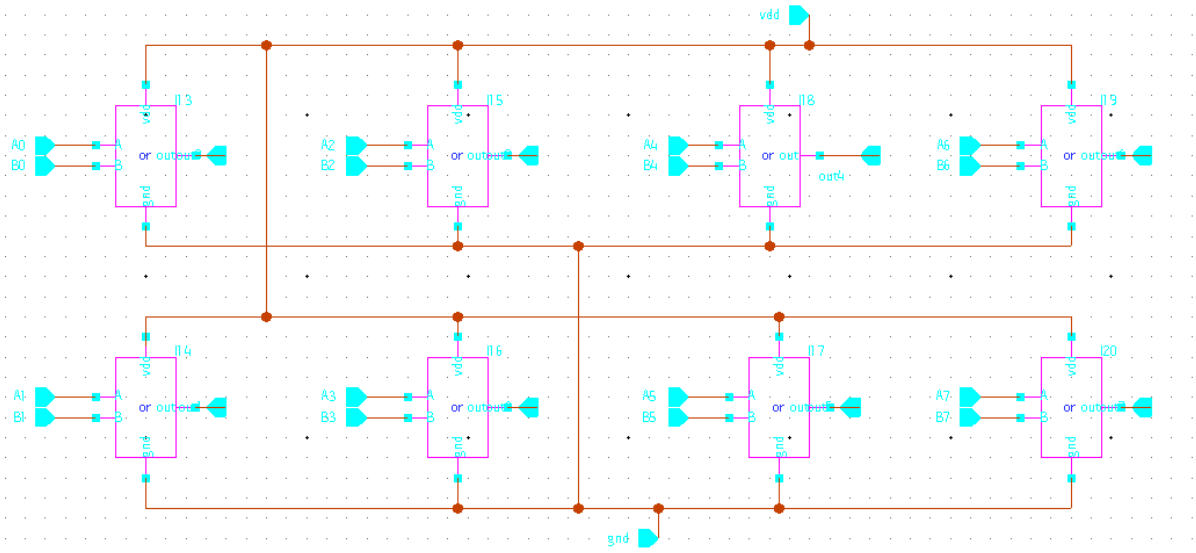


Figure 18. Eight- Bit OR Schematic

3.2.3. XOR

The XOR operation detects bit-level differences between the input operands. A 1-bit XOR gate is implemented at the transistor level to produce a logic-high output when the two input bits differ.

The 8-bit XOR unit is formed by arranging eight identical 1-bit XOR gates in parallel. This structure allows simultaneous comparison of all input bits. The schematic representations of the 1-bit XOR gate and the 8-bit XOR unit are shown in Figure 19 and Figure 20, respectively.

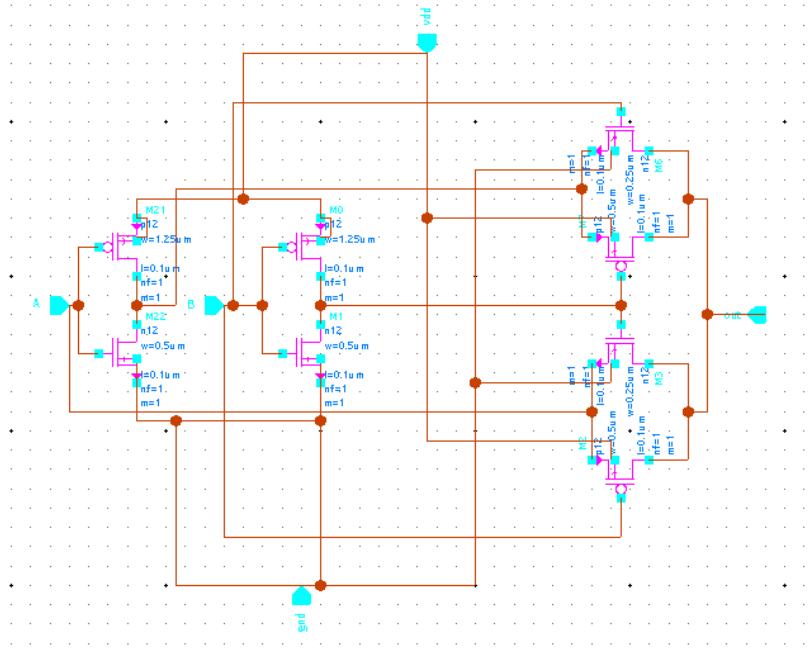


Figure 19.One-Bit XOR Schematic

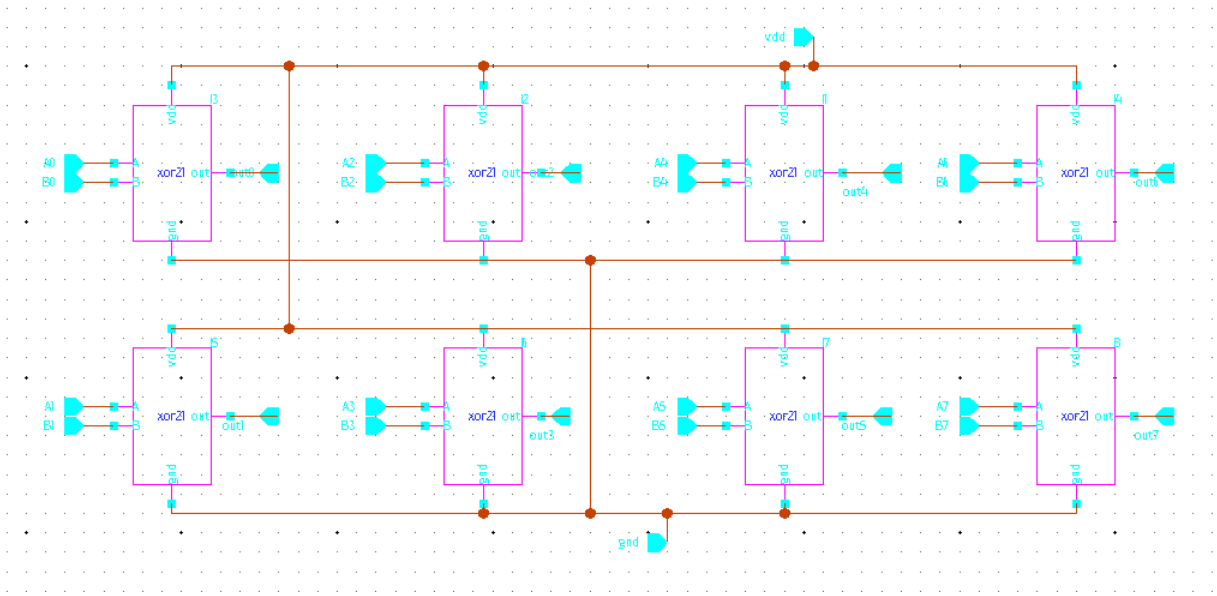


Figure 20. Eight- Bit XOR Schematic

3.2.4. NOT(Inverter)

The NOT operation is implemented using a simple inverter circuit at the transistor level. Each inverter flips the logic level of the corresponding input bit, producing the bitwise complement of the input data.

An 8-bit NOT unit is constructed by connecting eight identical inverter circuits in parallel. This design ensures independent inversion of each bit. The transistor-level schematic of the 1-bit inverter is shown in Figure 21, and the complete 8-bit NOT unit is presented in Figure 22.

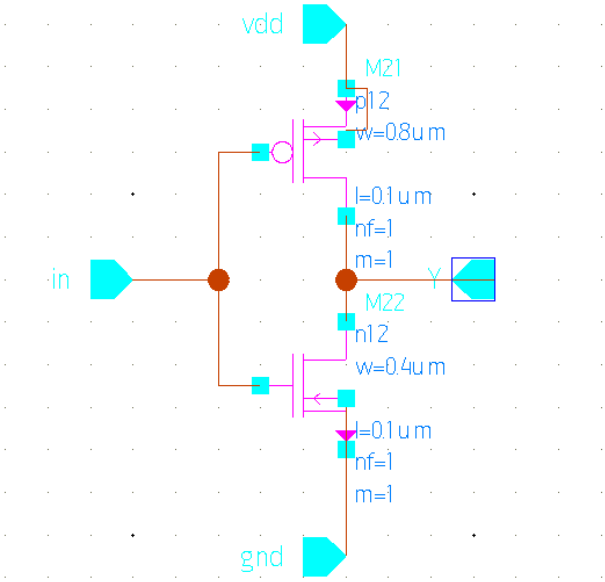


Figure 21.One-Bit NOT Schematic

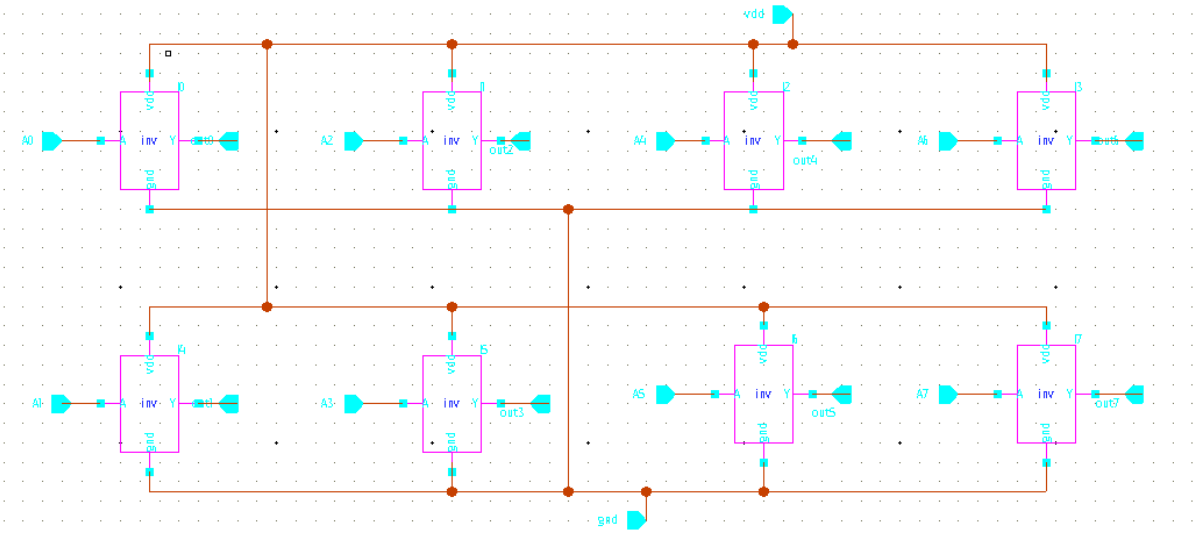


Figure 22. Eight- Bit NOT Schematic

3.2.5.MUX

The multiplexer (MUX) unit is used to select one of multiple input signals and route it to a single output based on the value of a control signal. In this design, a 2-to-1 multiplexer is implemented at the transistor level using CMOS logic.The MUX operation is controlled by the select signal S, which determines whether input A or input B is propagated to the output. When S = 0, the multiplexer selects input A, whereas when S = 1, input B is transferred to the output. This behavior allows conditional data routing and is a fundamental building block in arithmetic and logic units.At the transistor level, the multiplexer is realized using complementary pull-up (PMOS) and pull-down (NMOS) networks. The select signal and its complement are used to control the conduction paths, ensuring that only the desired input is actively connected to the output node at any given time. This structure guarantees correct logic functionality while maintaining low static power consumption.The transistor-level schematic of the 2-to-1 multiplexer is shown in Figure 23, illustrating how the select signal controls the switching behavior between inputs A and B. The CMOS-based implementation provides reliable operation, good noise margins, and compatibility with the overall VLSI design methodology adopted in this work.

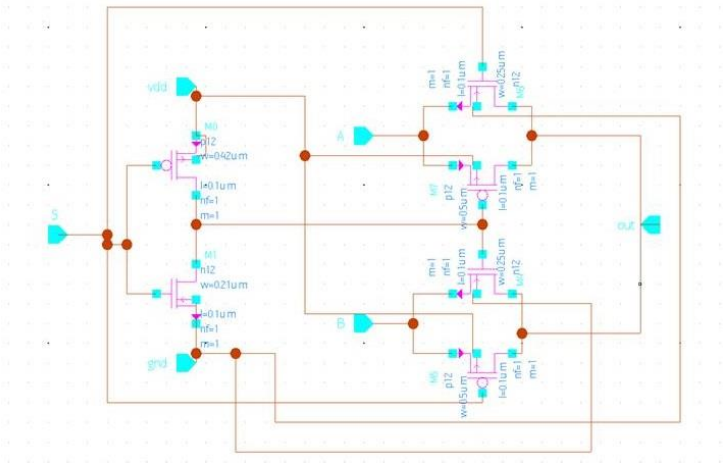


Figure 23.MUX Schematic

3.3. Shifter Unit Design

The shifter unit is responsible for performing bitwise shift operations on multi-bit input data, which are commonly required in arithmetic, logic, and data manipulation tasks. In this work, the shifter is implemented as a purely combinational circuit constructed using cascaded 2-to-1 multiplexers, allowing efficient and flexible shift operations without the need for clocked elements.

The basic building block of the shifter is a 2-to-1 multiplexer, where the select signal determines which input is propagated to the output. By organizing these multiplexers in multiple stages, different shift amounts can be realized in a structured and scalable manner. Each stage corresponds to a specific shift distance and is controlled by an individual select signal.

Figure 24 illustrates the architecture of the 8-bit right shifter, where the input vector A_7–A_0 is routed through a network of multiplexers. The select signals S_2, S_1, S_0control the amount of shift by enabling or bypassing the appropriate paths within each stage. This hierarchical structure allows the shifter to perform multiple shift operations within a single combinational path.

The multiplexer-based implementation ensures modularity and ease of extension to wider word lengths. Additionally, the fully combinational design provides low latency and makes the shifter well-suited for integration into the overall arithmetic logic unit (ALU) architecture.

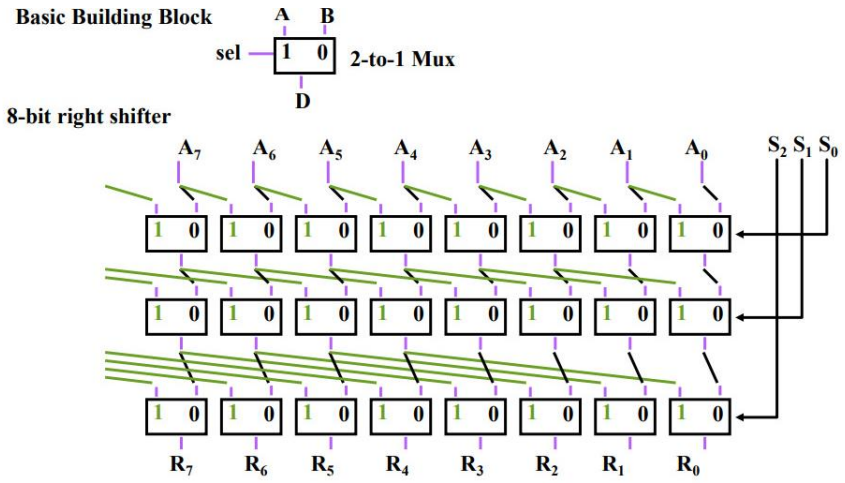


Figure 24. MUX-Based 8-Bit Right Shifter Architecture

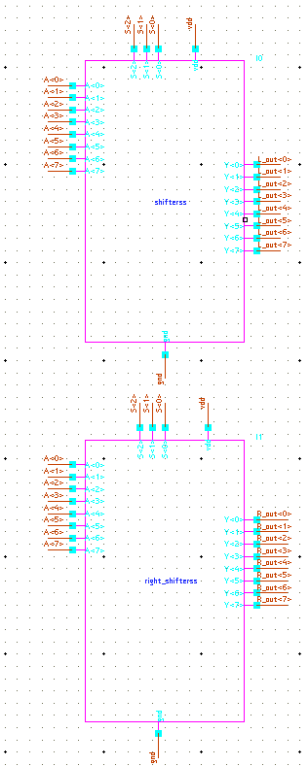


Figure 25. Symbol of Shifter

Figure 25 shows the symbol-level representation of the shifter unit supporting both left and right shift operations. The symbol abstracts the internal left and right shifter implementations and highlights the external interface of the shifter. The input vector A_7–A_0 represents the data to be shifted, while the output vector Y_7–Y_0 provides the shifted result. The control inputs S_2, S_1, S_0determine the shift magnitude, and the LR_Sel signal selects the shift direction. Power supply connections (VDD and GND) are included for proper CMOS operation.

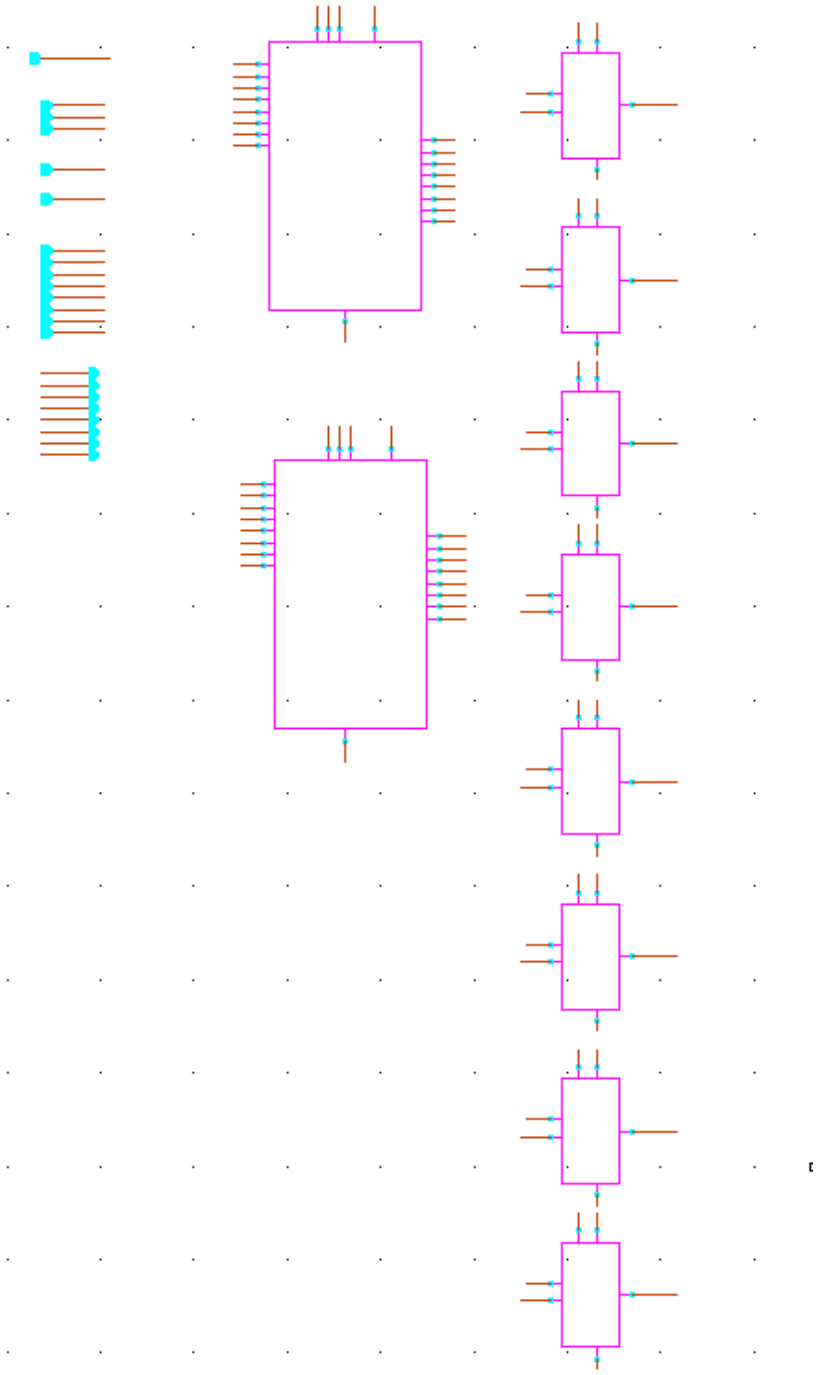


Figure 26. Schematic of Shifter

The schematic of the shifter is presented in Figure 26. The circuit consists of a left shifter and a right shifter implemented in parallel, each constructed using 24 identical 2-to-1 multiplexers. An additional selection stage controlled by the LR_Sel signal routes the output of the selected shift path to the final output.

This structure enables both shift directions to be available simultaneously, while maintaining a fully combinational data path. The use of identical multiplexer cells improves layout regularity and enhances scalability for wider word lengths.

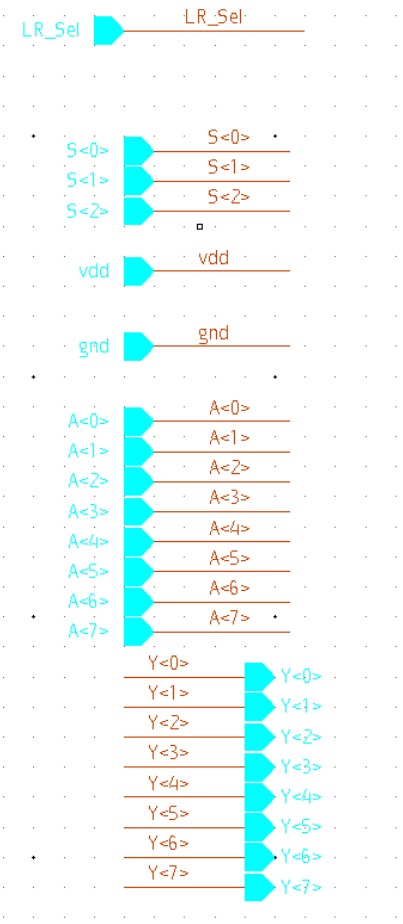


Figure 27. Pins of Shifter

Figure 27 illustrates the pin configuration of the shifter unit. The data input pins A_7–A_0 receive the input word, while the output pins Y_7–Y_0 provide the shifted result. The select pins S_2, S_1, S_0control the shift magnitude, and the LR_Sel pin determines the shift direction. The VDD and GND pins supply the required power for CMOS operation, enabling seamless integration of the shifter into the overall ALU architecture.

3.3.1. Right Shifter

The right shifter unit performs a bitwise right shift operation on an 8-bit input data word. In a right shift operation, the input bits are shifted toward the least significant bit (LSB) direction by a specified number of positions, while the vacated most significant bit (MSB) positions are filled with logic zero. Such operations are widely used in arithmetic scaling, data alignment, and logical manipulation within arithmetic logic units (ALUs).

In this design, the right shifter is implemented as a fully combinational circuit using cascaded 2-to-1 multiplexers. The amount of shift is controlled by the select signals S_2, S_1, S_0, which enable different routing paths inside the multiplexer network. This structure allows multiple shift distances to be realized efficiently without the need for sequential elements.

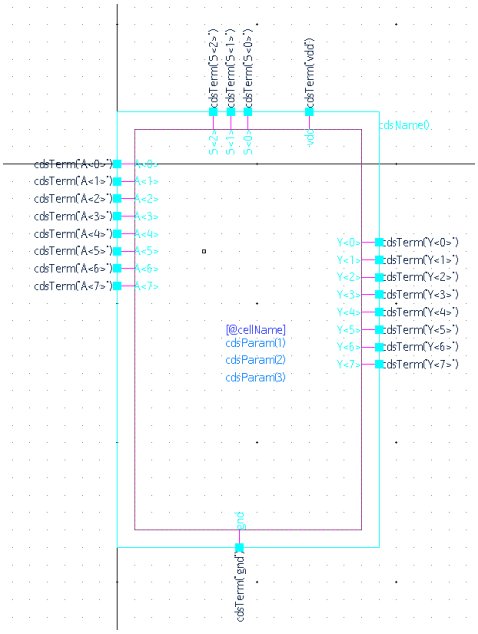


Figure 28. Symbol of Right Shifter

Figure 28 shows the symbol-level representation of the 8-bit right shifter. The block symbol abstracts the internal multiplexer network and highlights the external interface of the unit. The input vector A_7–A_0 represents the data to be shifted, while the output vector Y_7–Y_0 provides the shifted result. The control inputs S_2, S_1, S_0determine the number of positions by which the input data is shifted to the right. Power supply connections (VDD and GND) are also indicated to ensure proper CMOS operation.

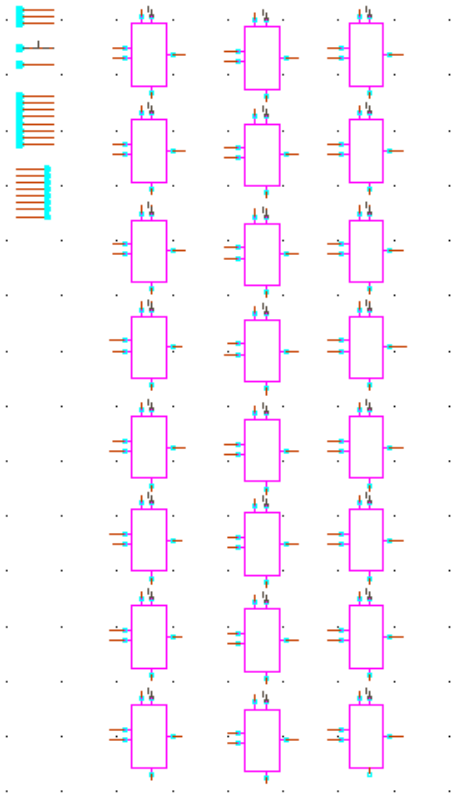


Figure 29. Schematic of Right Shifter

The transistor-level schematic of the right shifter is presented in Figure 30. The circuit is implemented using a total of 24 identical 2-to-1 multiplexers, organized into multiple cascaded stages. Each stage corresponds to a specific shift distance and is controlled by one of the select signals. By enabling or bypassing certain signal paths, the input bits are conditionally propagated toward the output nodes.

This structured arrangement of multiplexers enables the realization of different shift amounts within a single combinational path. The use of identical multiplexer cells improves layout regularity and design scalability, allowing the shifter to be efficiently extended to wider word lengths if required.

3.3.2.Left Shifter

The left shifter unit performs a bitwise left shift operation on an 8-bit input data word. In a left shift operation, the input bits are shifted toward the most significant bit (MSB) direction by a specified number of positions, while the vacated least significant bit (LSB) positions are filled with logic zero. Left shift operations are commonly used for arithmetic scaling, bit manipulation, and data alignment in digital systems.

Similar to the right shifter, the left shifter is implemented as a fully combinational circuit using cascaded 2-to-1 multiplexers. The shift amount is controlled by the select signals S_2, S_1, S_0, which configure the internal routing paths of the multiplexer network. This design approach enables efficient realization of multiple shift distances without the need for clocked elements.

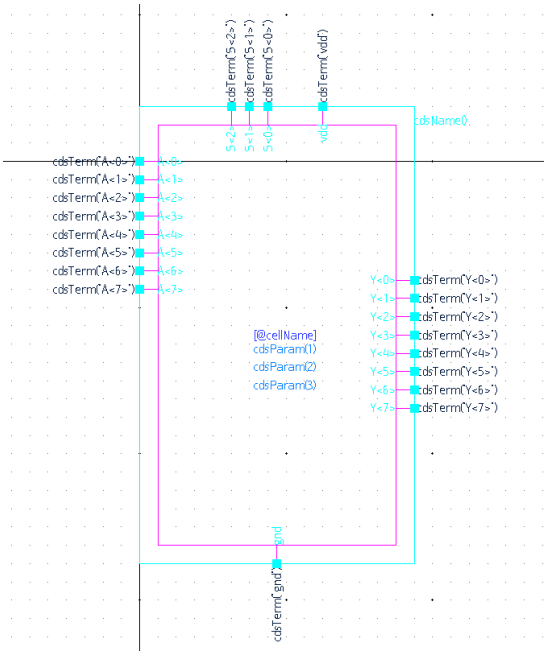


Figure 30. Symbol of Left Shifter

Figure 30 shows the symbol-level representation of the 8-bit left shifter. The block symbol abstracts the internal multiplexer-based implementation and highlights the external interface of the unit. The input data vector A_7–A_0 is applied to the left shifter, and the shifted result is provided at the output vector Y_7–Y_0. The control signals S_2, S_1, S_0determine the number of left shift positions applied to the input data. Power supply connections (VDD and GND) are included for proper CMOS operation.

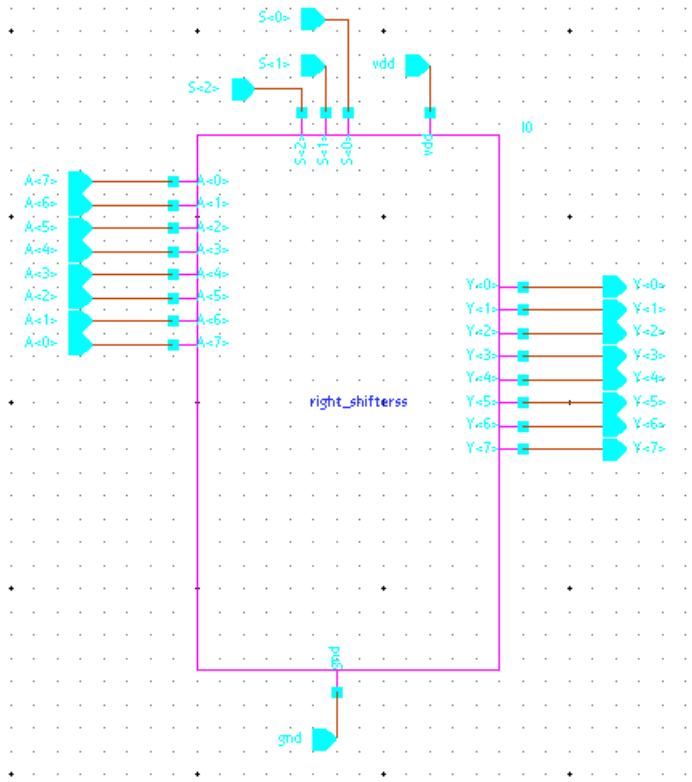


Figure 31. Schematic of Left Shifter

The transistor-level schematic of the left shifter is illustrated in Figure 33. The circuit is constructed using a total of 24 identical 2-to-1 multiplexers, arranged in multiple cascaded stages. Each stage corresponds to a specific left shift distance and is controlled by one of the select signals. Depending on the selected configuration, the input bits are routed toward higher significance positions at the output.

The use of identical multiplexer cells results in a regular and modular schematic structure, which simplifies the physical layout and improves design scalability. This architecture allows the left shifter to be efficiently extended to larger word sizes when required.

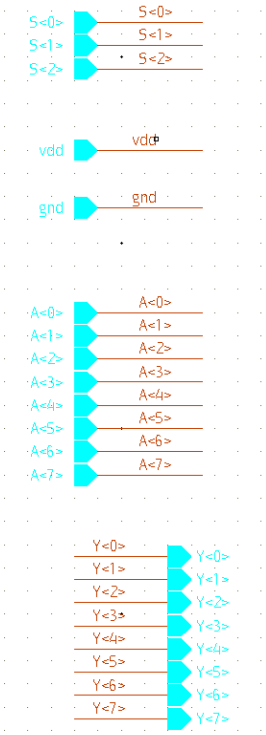


Figure 32. Pins of Left Shifter

Figure 32 presents the pin configuration of the left shifter unit. The data input pins A_7–A_0 accept the input word, while the output pins Y_7–Y_0 provide the left-shifted result. The select pins S_2, S_1, S_0control the shift magnitude, and the VDD and GND pins supply the necessary power for CMOS operation. This pin arrangement enables straightforward integration of the left shifter into the overall ALU architecture.

3.4. Control Logic

In the proposed ALU design, a dedicated control logic (opcode decoder) is used to determine which operation will be executed. The primary function of the control logic is to decode the opcode values generated by the control unit and to activate the corresponding functional block within the ALU. This mechanism allows multiple arithmetic, logical, and shift operations to be implemented using a single shared hardware structure.

The control logic receives a 4-bit opcode input (Op0–Op3**)** and generates individual control signals for each ALU operation, such as addition, subtraction, logical AND, logical OR, exclusive OR, logical NOT, left shift, and right shift. At any given time, only one control signal is asserted, while all other signals remain inactive. This one-hot operation selection prevents unintended concurrent activation of multiple functional blocks and ensures correct ALU behavior.

The opcode decoding process is realized using inverter (NOT) gates and multi-input AND gates. Both the original and inverted forms of the opcode bits are generated and combined through AND logic to detect specific opcode patterns. Each unique opcode combination enables exactly one control signal, which selects the corresponding ALU operation.

This control logic structure provides a modular, readable, and hardware-efficient solution for operation selection. Furthermore, the close correspondence between the RTL-level description and the gate-level implementation ensures consistent functionality during synthesis and simulation, which is confirmed through transient analysis results.

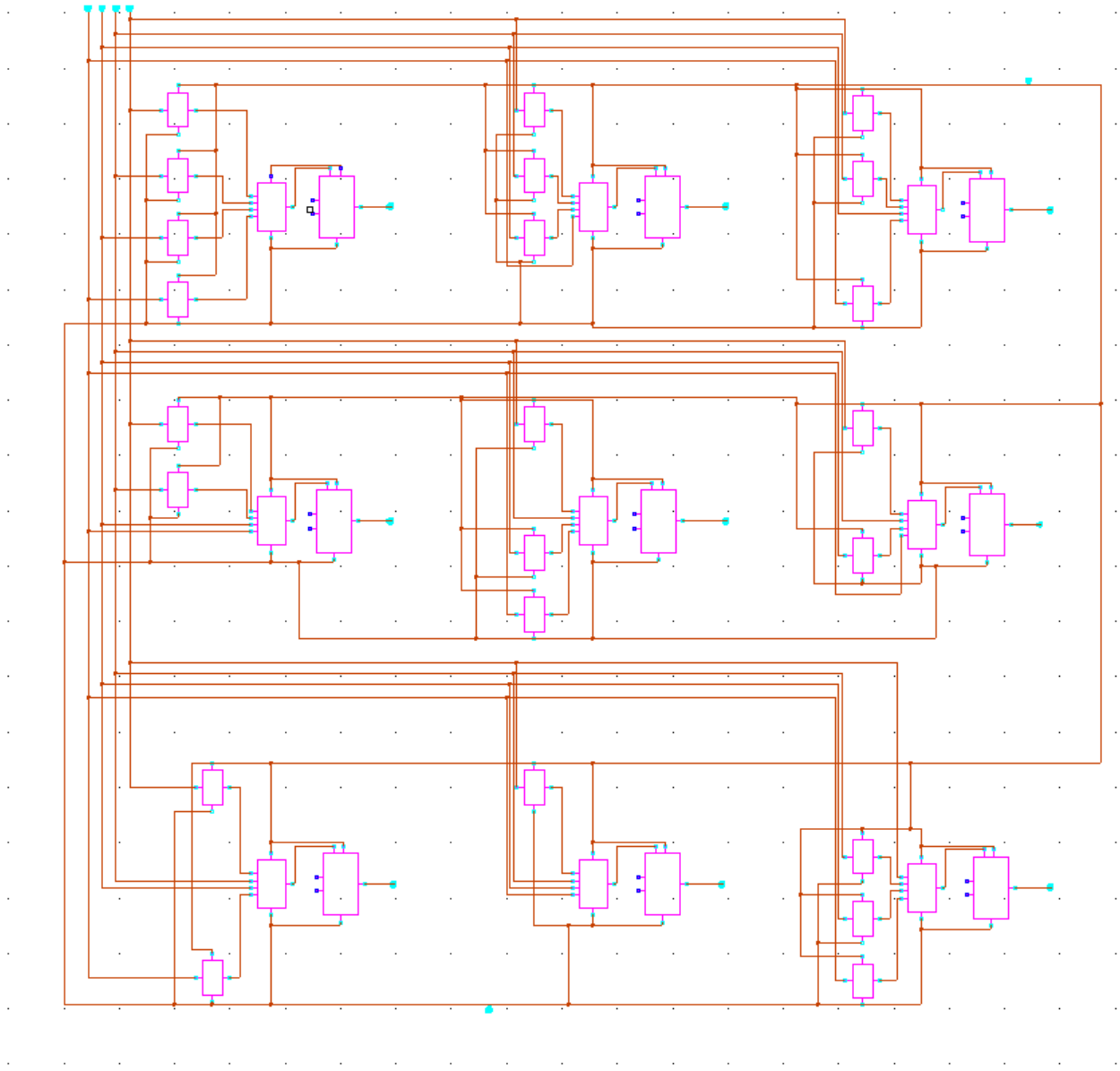
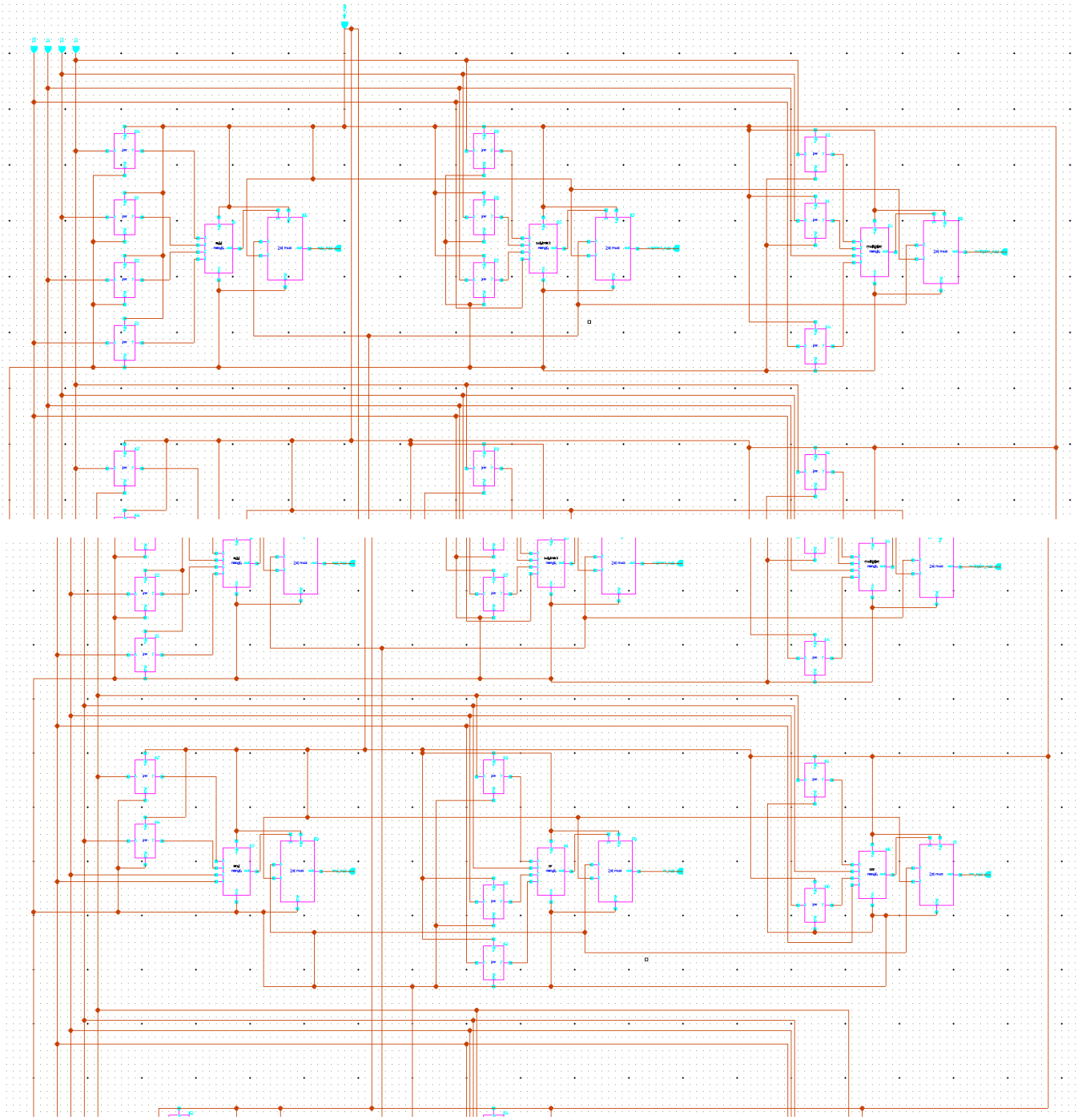


Figure 33. Schematic Implementation of the Opcode Decoder

Figure 33 shows the schematic-level implementation of the opcode decoder control logic, where the decoding function is realized using four-input AND (AND4) gates. The 4-bit opcode inputs (Op0–Op3) and their complemented forms are combined through AND4 gates to detect specific opcode patterns. Each AND4 gate corresponds to a unique opcode combination and generates a dedicated control signal for a single ALU operation. This approach ensures one-hot operation selection, such that only one control signal is asserted for any given opcode value. The schematic clearly demonstrates how the opcode decoder logic is constructed from basic logic gates and serves as a concrete realization of the control logic defined at the RTL level.



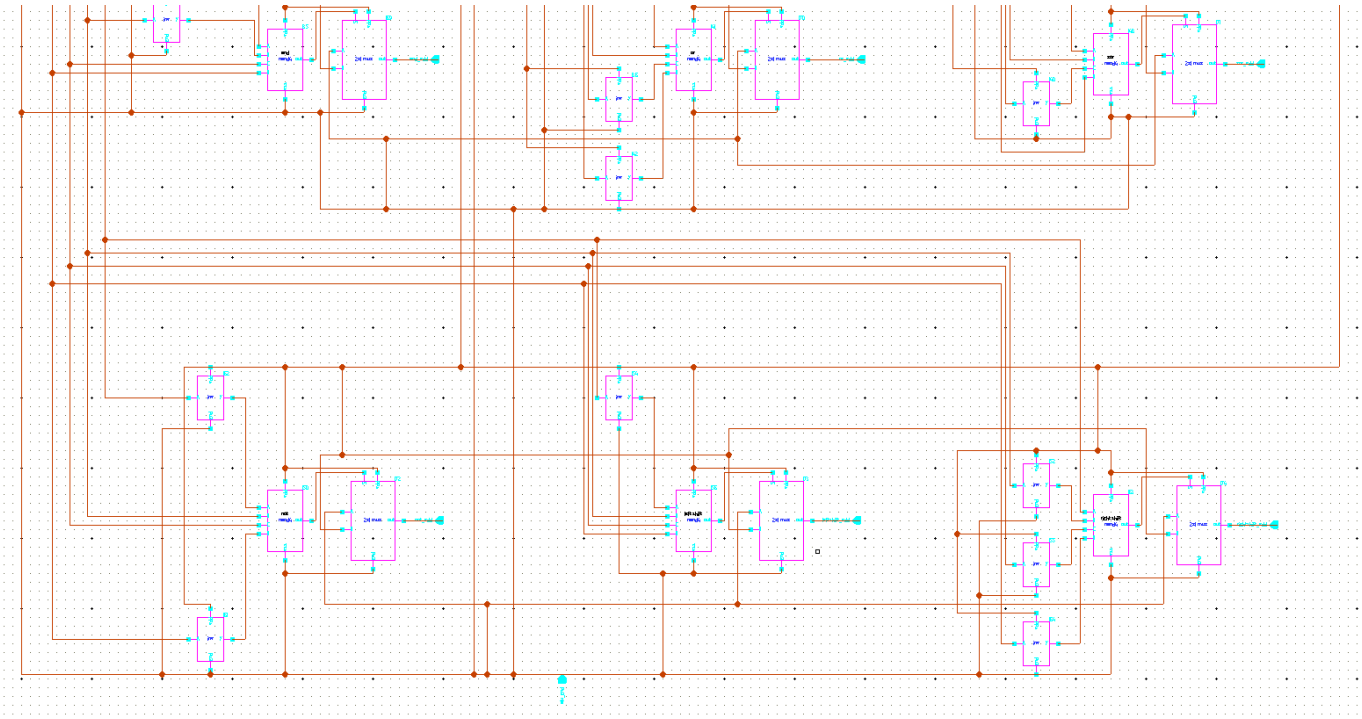


Figure 34. Zoomed-In Schematic View of the Opcode

3.5.Output Unit

The Output Unit is responsible for producing the final ALU result based on the operation selected by the control logic. After the arithmetic, logical, or shift operation is executed within the corresponding functional block, the generated result is forwarded to the Output Unit. This unit serves as the final stage of the ALU datapath and ensures that the correct result is presented at the ALU output.

The Output Unit receives multiple candidate results from different functional blocks but allows only the result of the active operation to propagate to the output. Operation selection is governed by the control signals generated by the opcode decoder, ensuring that only one result is valid at any given time. This structure prevents bus contention and guarantees deterministic ALU behavior.

At this stage of the design, the Output Unit focuses solely on result selection and propagation. Additional status information such as condition flags (e.g., zero, carry, overflow, sign) will be incorporated in a later step. By separating result generation from status flag computation, the design maintains modularity and allows the Output Unit to be easily extended without modifying the core datapath.

Flags

In addition to producing the computation result, the ALU generates a set of status flags that provide information about the outcome of the executed operation. These flags are commonly used by the control unit to support conditional operations, branching, and decision-making in higher-level system behavior.

The following flags are considered in this design:

- #### Zero (Z) Flag:
  The zero flag is asserted when the result produced by the ALU is equal to zero. This flag is typically used to detect equality conditions and to support zero-based comparisons.

- #### Carry (C) Flag:
  The carry flag indicates the presence of a carry-out from the most significant bit during arithmetic operations such as addition or subtraction. It is mainly used in unsigned arithmetic and multi-word calculations.

- #### Overflow (V) Flag:
  The overflow flag is set when a signed arithmetic operation produces a result that exceeds the representable range of the output word length. This flag helps distinguish between valid and invalid results in signed number operations.

- #### Sign (S) Flag:
  The sign flag reflects the most significant bit of the result and indicates whether the output value is positive or negative when interpreted as a signed number.

At this stage of the design, the flags are generated based on the final ALU result and the nature of the selected operation. The flag generation logic is designed as a separate functional block, allowing the ALU datapath and control logic to remain modular. This separation enables future extensions and simplifies debugging and verification.

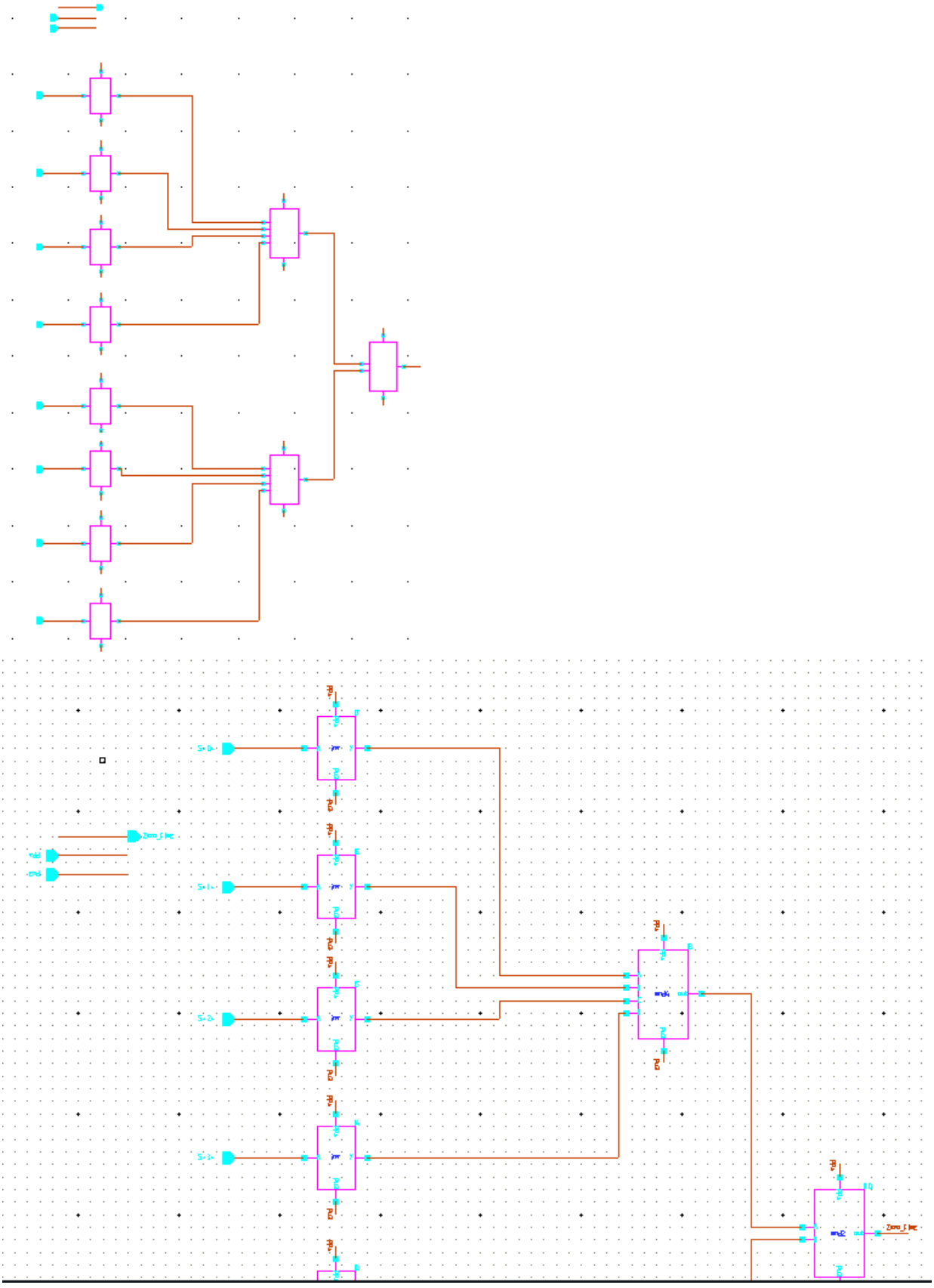


Figure 35.Schematic of Zero Flag

This figure illustrates the schematic-level implementation of the Zero Flag generation logic. The logic evaluates all bits of the ALU output to determine whether the result is zero. Internally, the output bits are combined using logical operations to check if any bit is set. If all bits are zero, the Zero Flag output is asserted. This schematic provides a detailed view of how the Zero Flag is derived from the ALU result and serves as the gate-level realization of the zero detection mechanism.

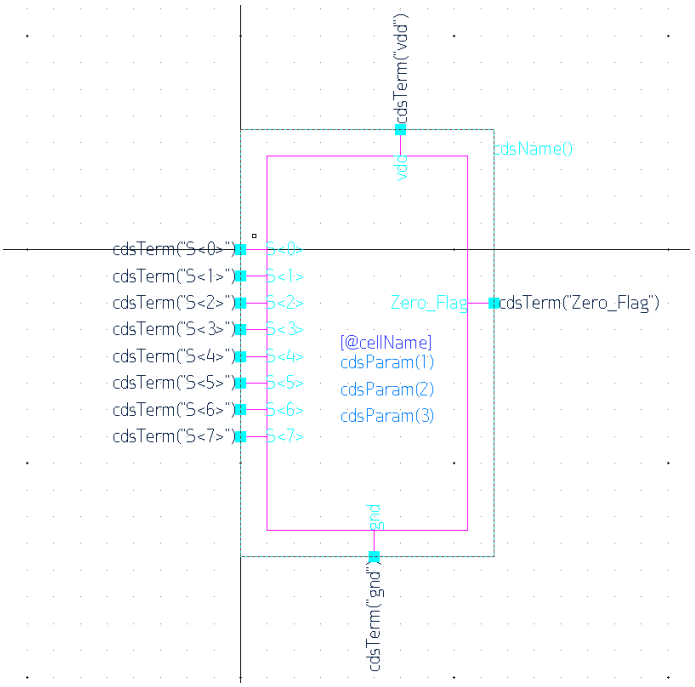


Figure 36.Symbol of Zero Flag

This figure shows the symbol representation of the Zero Flag block used in the ALU design. The block receives the final ALU result as input and generates a single-bit zero flag output. The Zero Flag is asserted when all bits of the ALU result are equal to zero, indicating that the operation result is zero. This symbol represents the abstract interface of the zero flag logic, allowing easy integration with the ALU datapath and control logic.

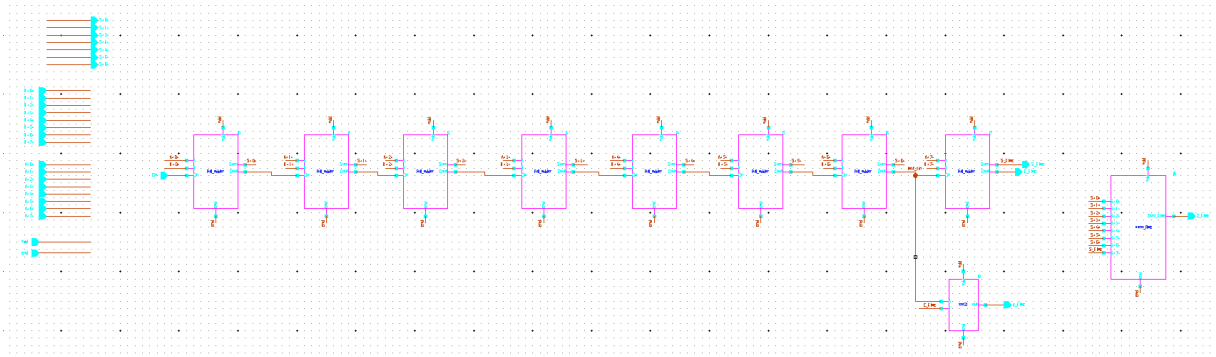


Figure 37. Schematic Implementation of the ALU Flag Generation Logic

This figure shows the schematic-level implementation of the ALU flag generation logic, which produces multiple status flags based on the final ALU result and arithmetic operation outputs. The logic generates the Sign, Carry, and Overflow flags by processing the result bits and carry signals produced

during arithmetic operations. The Sign flag is derived from the most significant bit of the ALU result, indicating the sign of the output in signed number representation. The Carry flag is generated from the carry-out of the most significant bit and reflects unsigned arithmetic overflow. The Overflow flag is produced by evaluating the carry signals associated with signed arithmetic to detect results that exceed the representable range. This schematic provides a detailed gate-level view of the internal flag logic and complements the symbolic representations used for system-level integration.

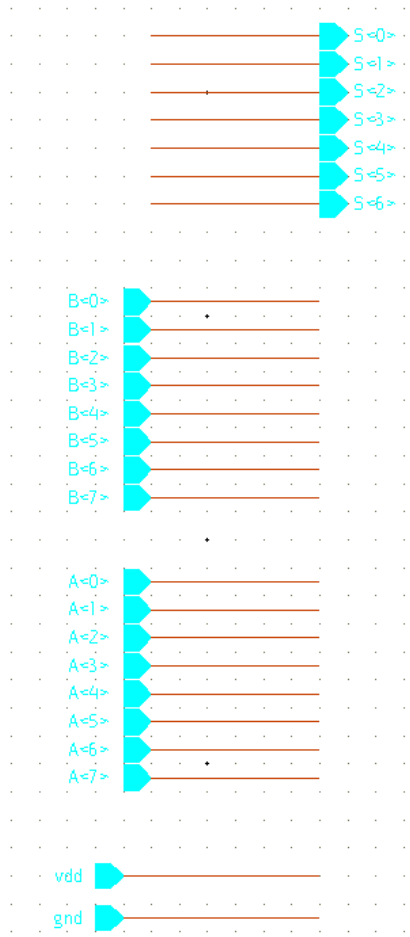


Figure 38. Pin Configuration of the ALU Flag Generation Unit

This figure presents the pin configuration of the ALU flag generation unit. The input pins include the ALU result bits and carry-related signals required for flag computation. The output pins correspond to the generated status flags, namely Sign, Carry, and Overflow. The pin-level view clearly defines the direction and function of each signal, ensuring correct connectivity between the flag generation unit and the ALU datapath. This representation is especially useful during schematic integration and verification.

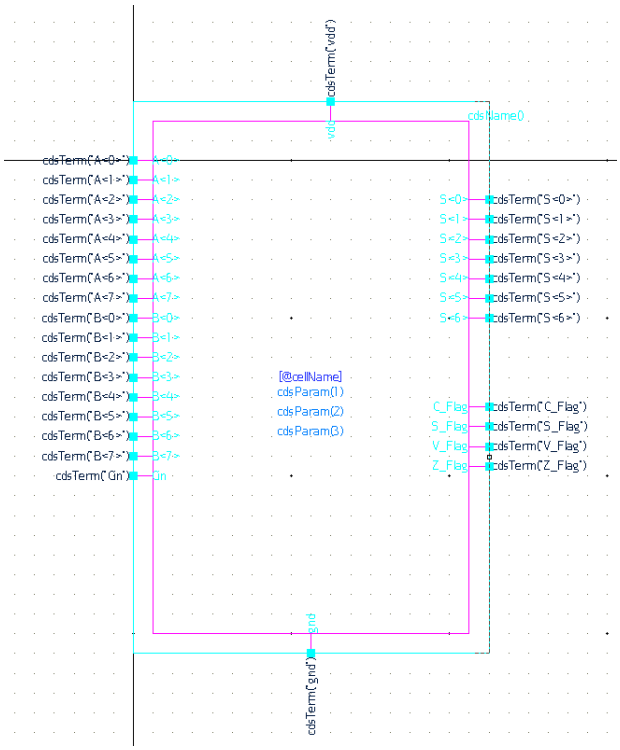


Figure 39. Symbol of the ALU Flag Generation Unit

This figure shows the symbol representation of the ALU flag generation unit. The symbol provides an abstract view of the block, defining only the input and output interfaces while hiding the internal logic implementation. The unit receives the final ALU result and arithmetic-related signals and produces the corresponding status flags, including Sign, Carry, and Overflow. This symbolic representation simplifies system-level integration and improves the readability of the overall ALU design.

4. LAYOUT

In this section, the physical layouts of the designed sub-blocks and the complete system arepresented. For each functional unit, both the layout and the layout with parasitic extraction are provided. The layout figures illustrate the physical placement of transistors, interconnections, and power distribution, while the parasitic-aware layouts include the effects of interconnect resistance and capacitance. This approach enables a realistic evaluation of post-layout performance and ensures that the designed circuits operate correctly under physical implementation constraints.

4.1.Arithmetic Design

4.1.1.Adder

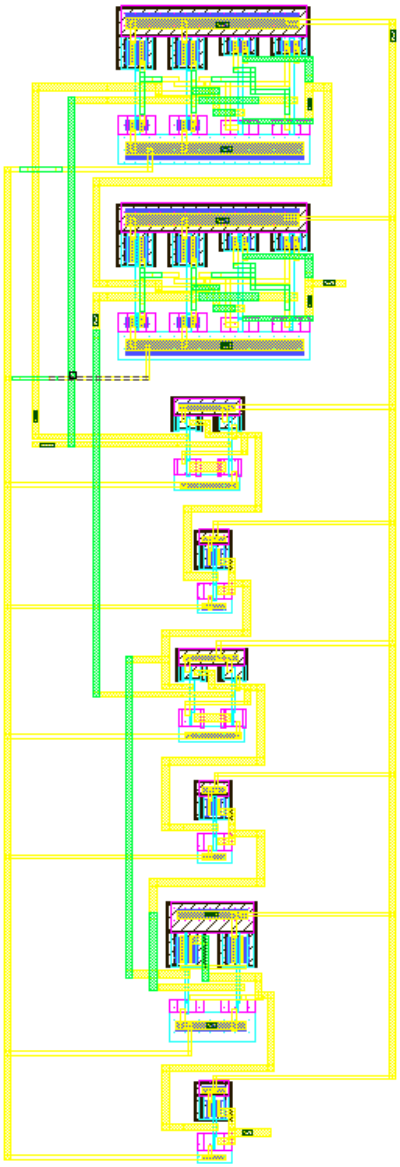


Figure 40.Full Adder Layout

This figure shows the physical layout of the full adder circuit. The layout is designed using a regular and compact structure to minimize area and interconnect length. Power and ground rails are shared across the cell, while signal routing is optimized to reduce propagation delay and improve overall performance.

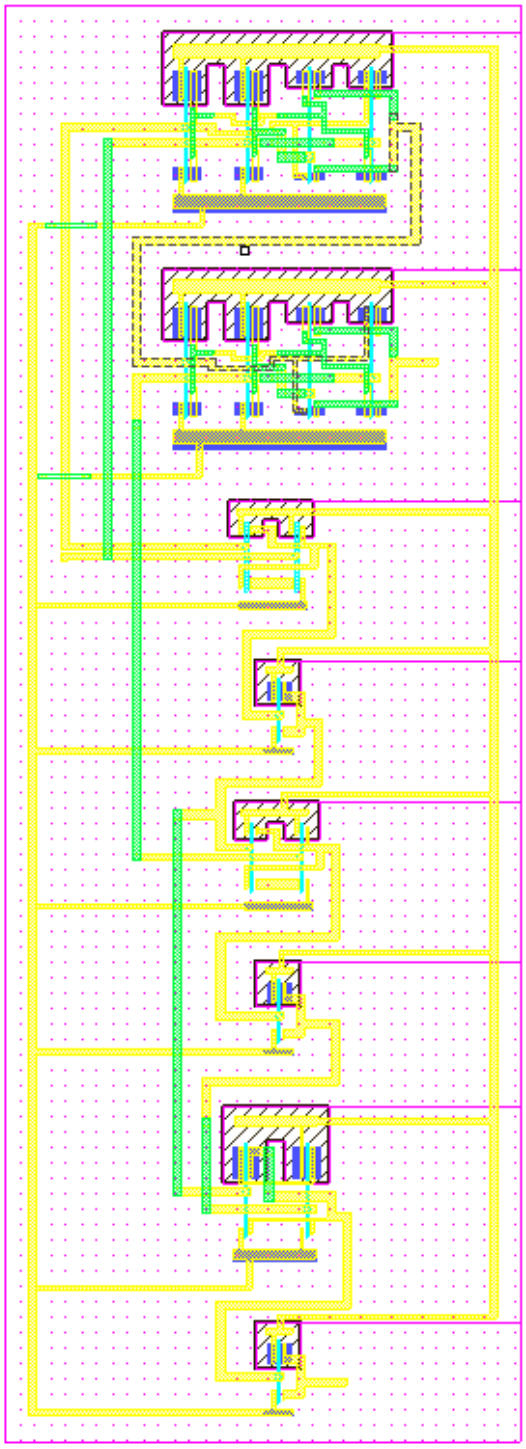


Figure 41.Full Adder Layout with Parasitics

This figure illustrates the full adder layout including extracted parasitic effects. Interconnect resistance and capacitance introduced by routing and device placement are considered to obtain a more realistic timing and power behavior. Parasitic-aware analysis ensures accurate post-layout performance evaluation.

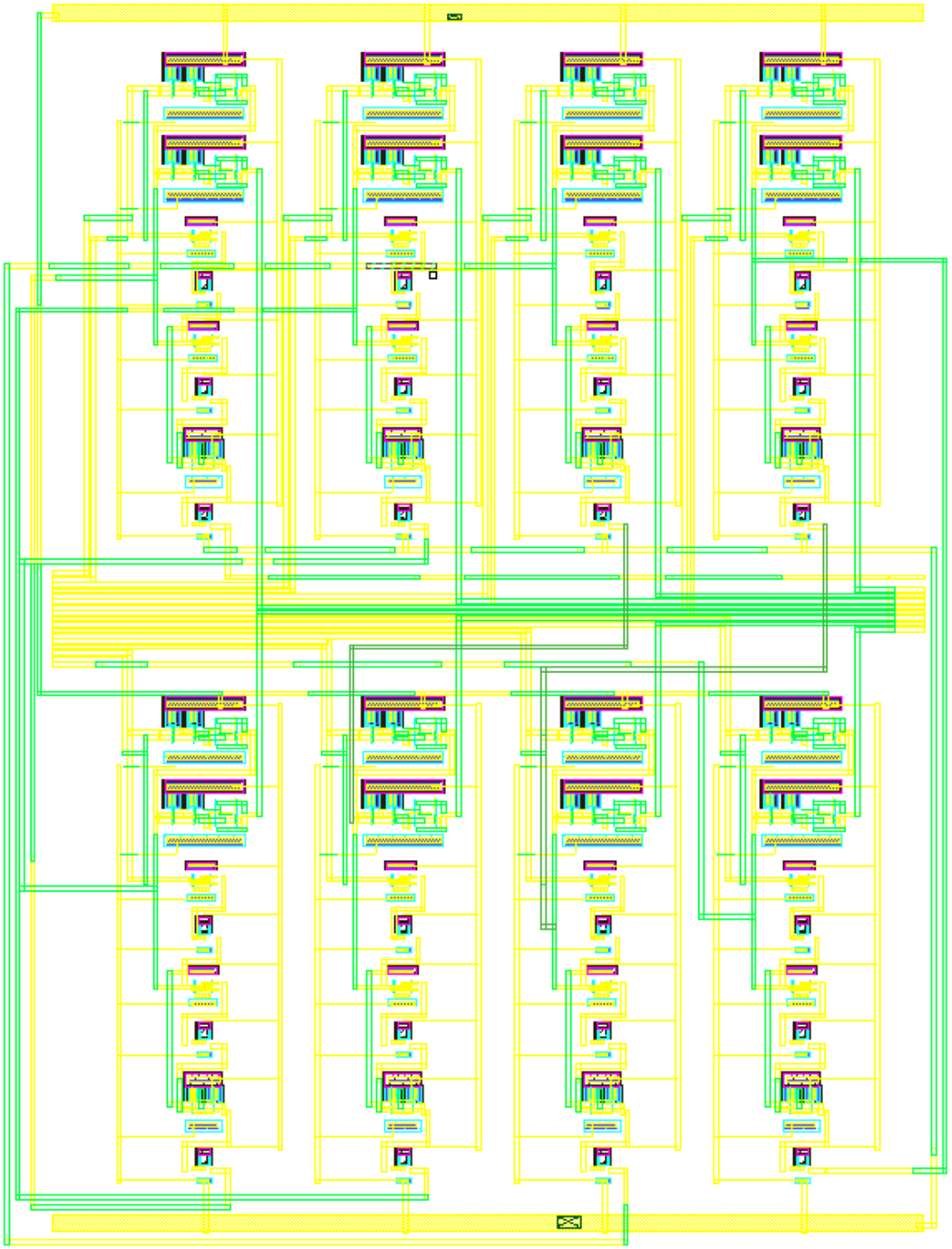


Figure 42. Eight-Bit Adder Layout

This figure presents the layout of the 8-bit adder constructed by cascading single-bit full adder cells in a ripple-carry architecture. A control signal is used to select between addition and subtraction operations. The modular layout approach simplifies routing and maintains signal consistency across all bit slices.

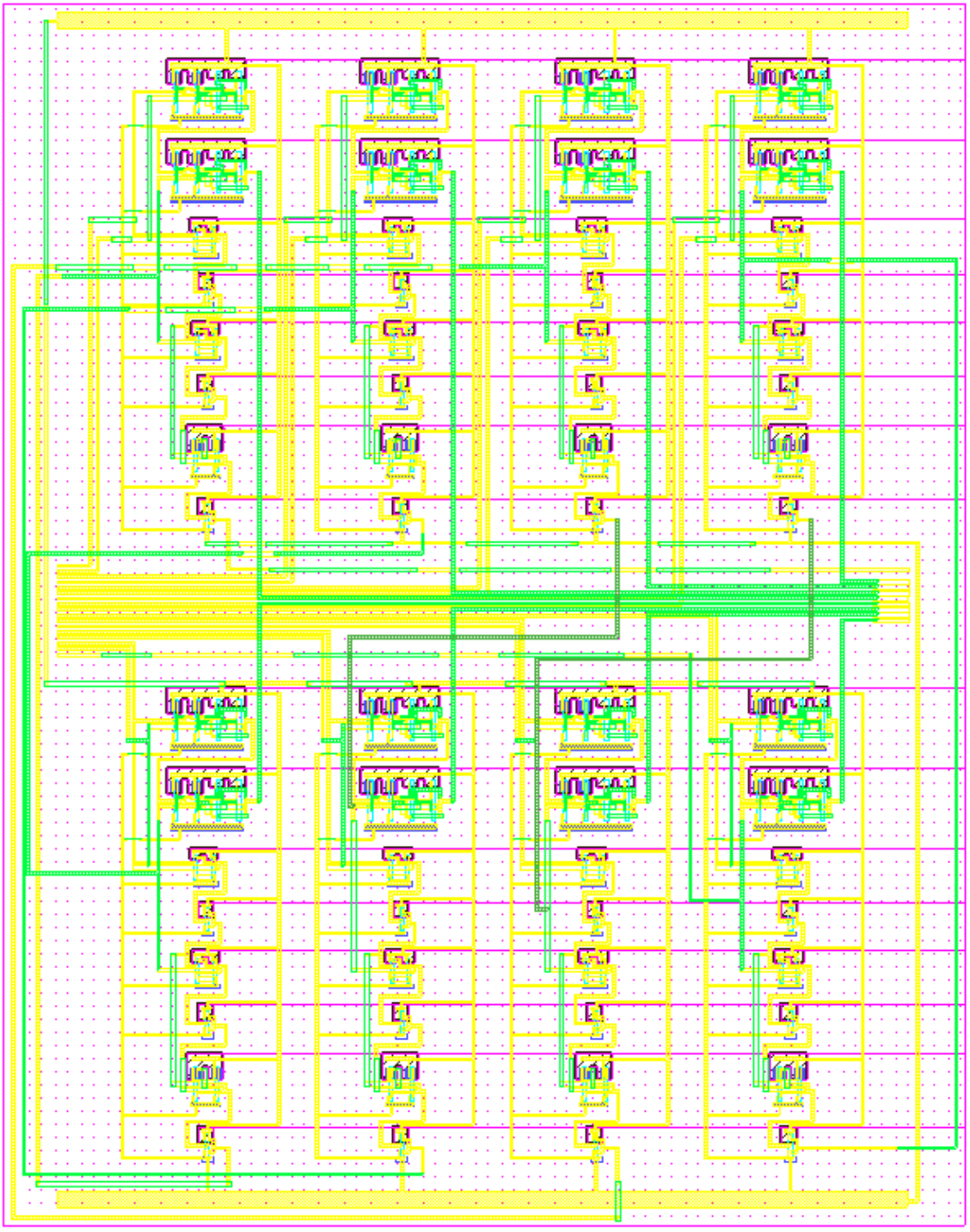


Figure 43. Eight-Bit Adder Layout with Parasitics

This layout includes parasitic extraction results for the 8-bit adder. The impact of carry-chain routing and long interconnects is analyzed to evaluate worst-case delay and ensure reliable arithmetic operation under realistic conditions.

4.1.2.Subtractor

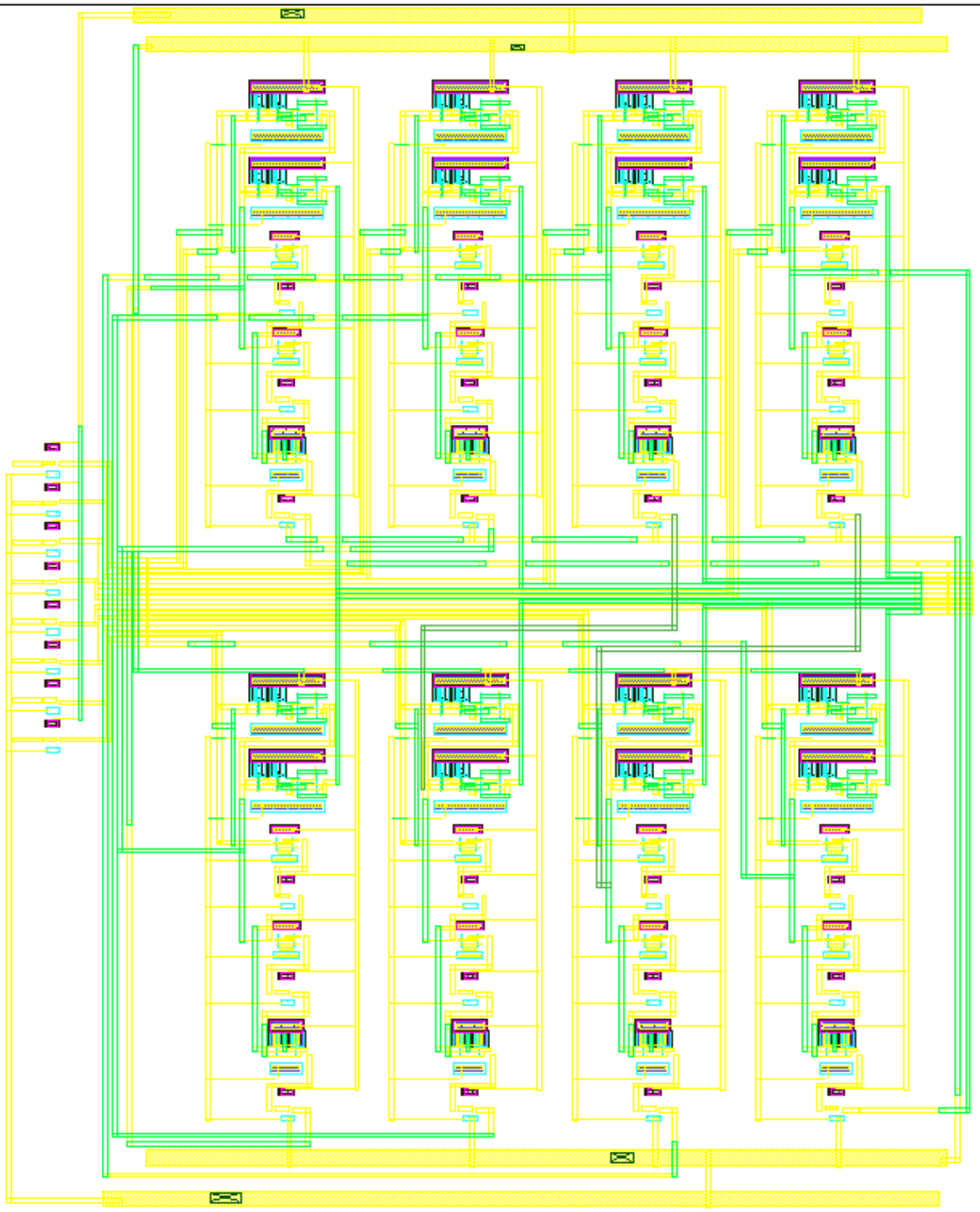


Figure 44. Eight-Bit Subtractor Layout

This figure presents the layout of the 8-bit subtractor constructed by cascading single-bit full adder cells in a ripple-carry architecture. A control signal is used to select between addition and subtraction operations. The modular layout approach simplifies routing and maintains signal consistency across all bit slices.

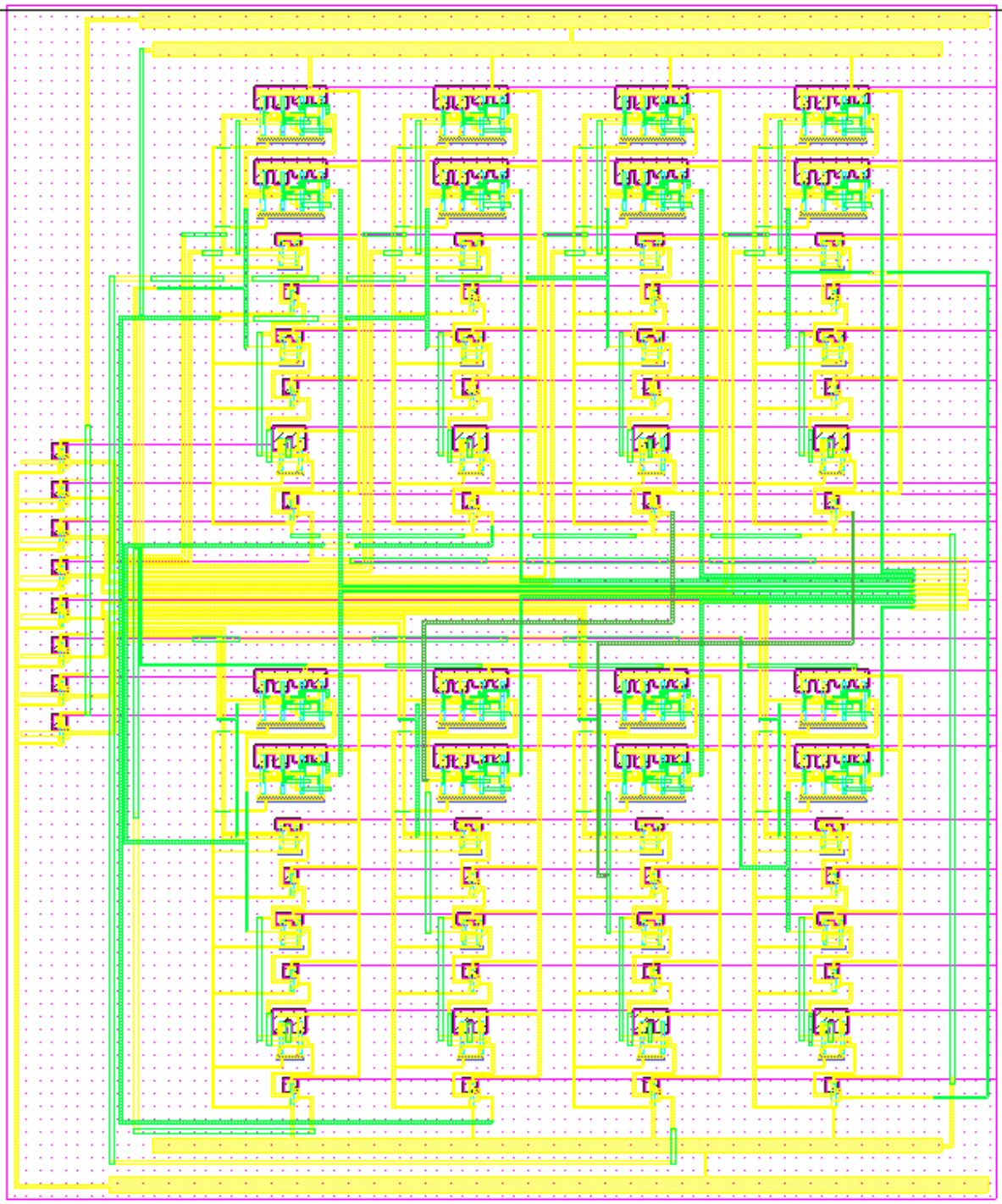


Figure 45. Eight-Bit Subtractor Layout with Parasitics

This layout includes parasitic extraction results for the 8-bit subtractor. The impact of carry-chain routing and long interconnects is analyzed to evaluate worst-case delay and ensure reliable arithmetic operation under realistic conditions.

4.1.3.Multiplier

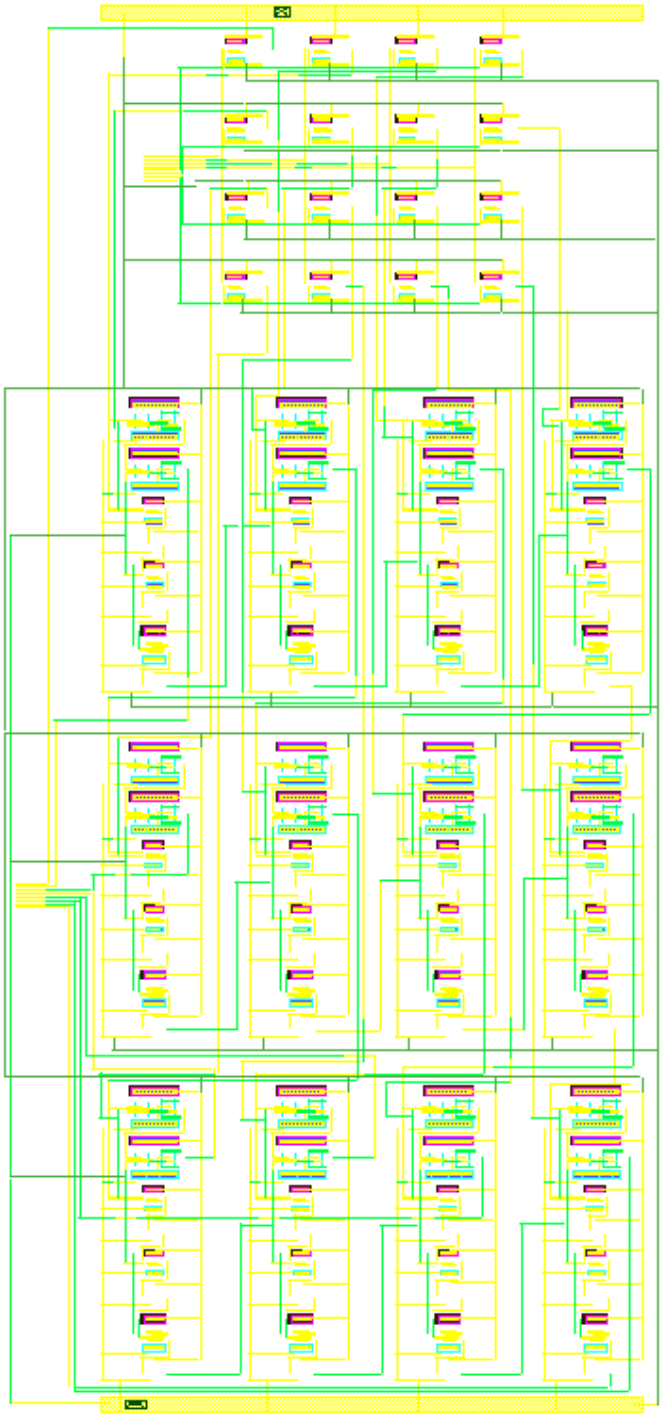


Figure 46. Eight-Bit Multiplier Layout

This figure shows the layout of the multiplier circuit, implemented using repeated partial product generation and accumulation blocks. The layout emphasizes regularity and symmetry to simplify routing and balance signal paths, which helps reduce timing mismatches between parallel computation stages.

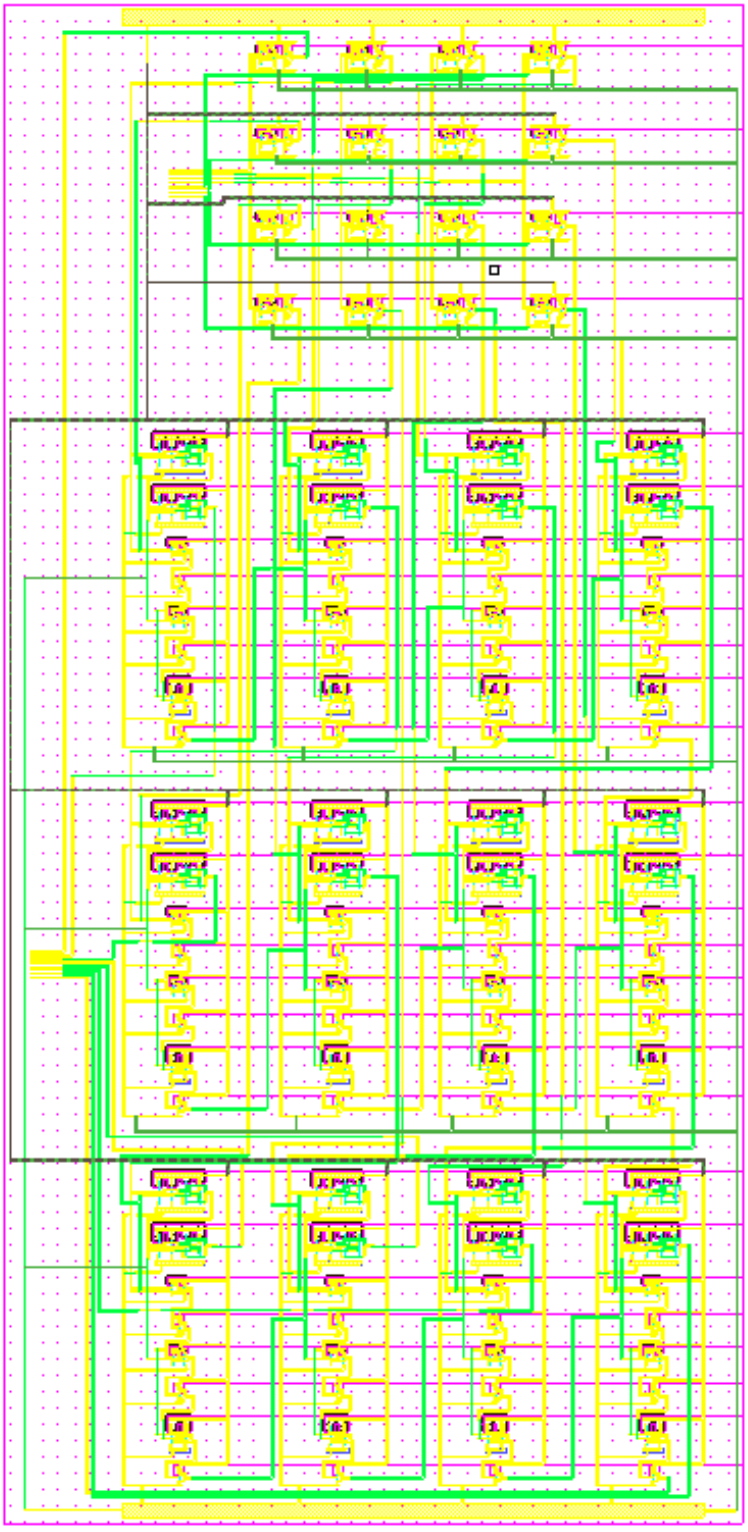


Figure 47. Eight-Bit Multiplier Layout with Parasitics

The parasitic-aware multiplier layout accounts for increased interconnect complexity due to multiple parallel signal paths. Extracted parasitics are used to evaluate delay and power consumption under realistic operating conditions.

4.2.Logical Unit

4.2.1 AND

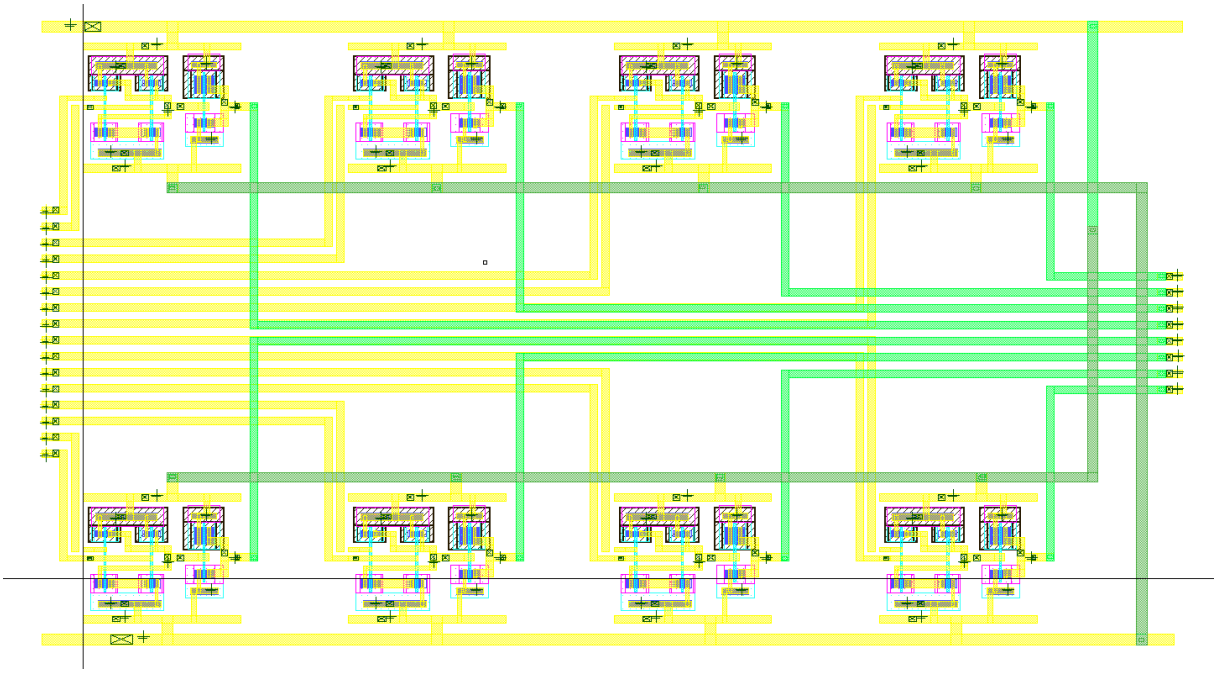


Figure 48. Eight-Bit AND Layout

This figure illustrates the layout of the AND logic gate. Transistors are placed compactly to minimize area and interconnect length. The simple structure results in low parasitic effects and fast switching behavior.

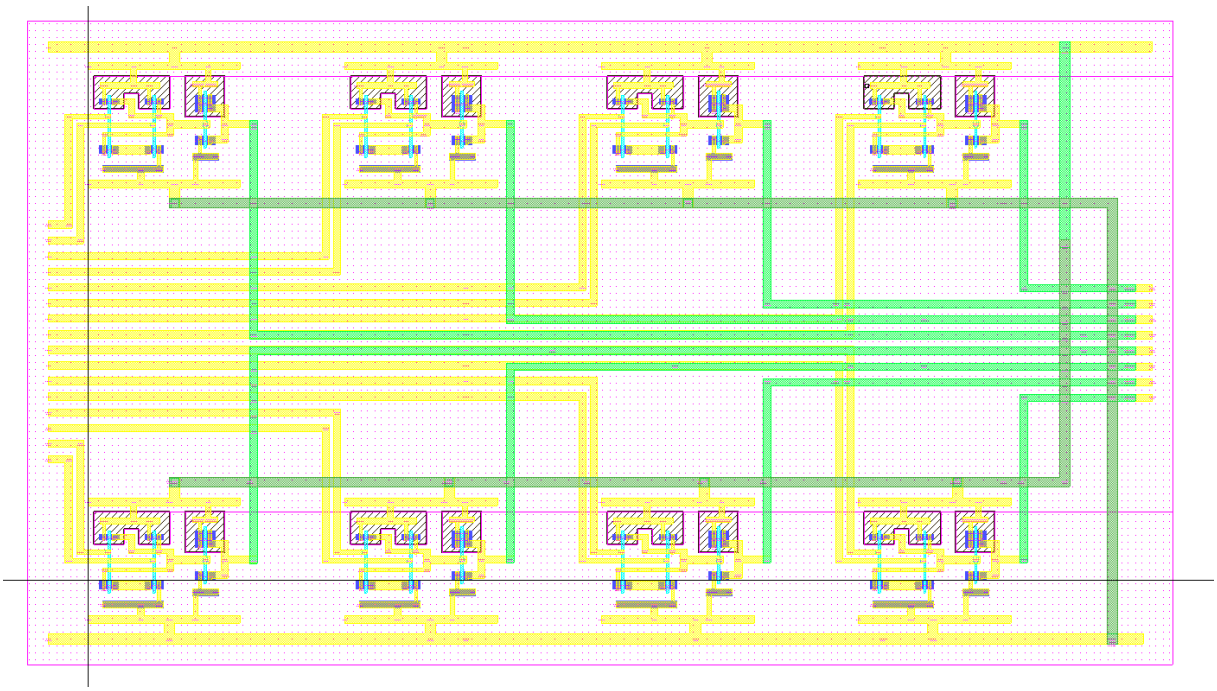


Figure 49. Eight-Bit AND Layout with Parasitics

This layout includes parasitic extraction, highlighting the minor impact of routing capacitances and resistances due to the gate's compact design.

4.2.2.OR

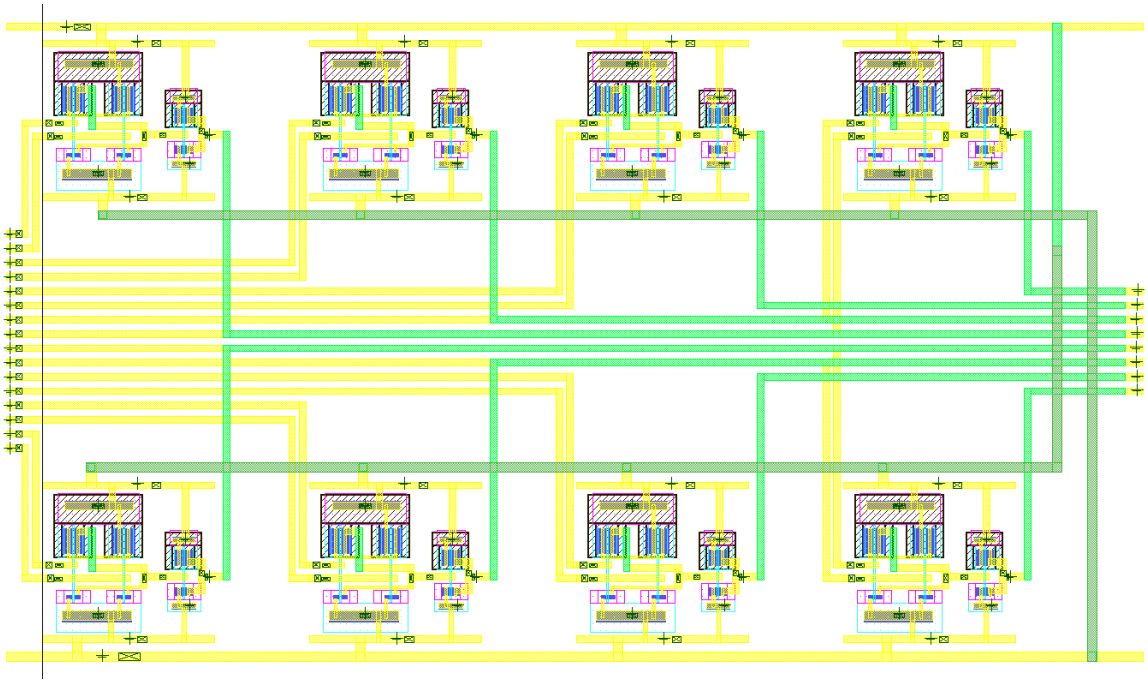


Figure 50. Eight-Bit OR Layout

This figure presents the physical layout of the OR gate. The layout is optimized for symmetry and short routing paths to ensure balanced signal propagation and stable logic operation.

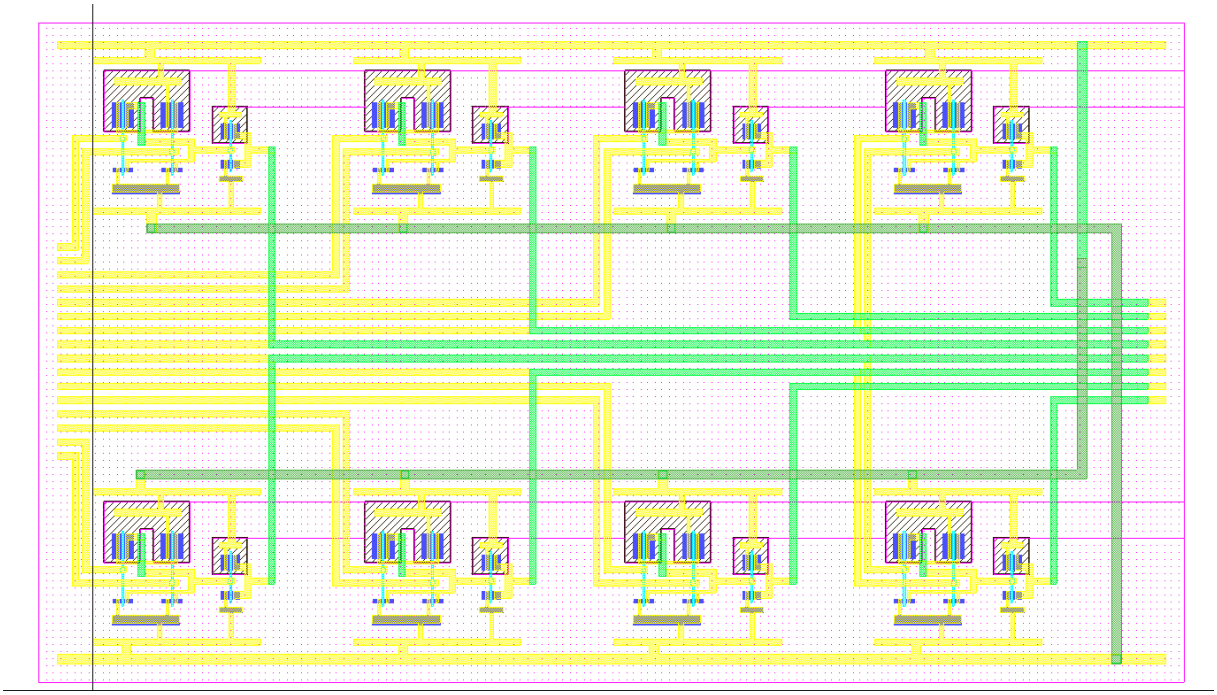


Figure 51. Eight-Bit OR Layout with Parasitics

Parasitic components extracted from the OR gate layout are used to assess realistic timing behavior, confirming correct operation after layout-level effects are included.

4.2.3.XOR

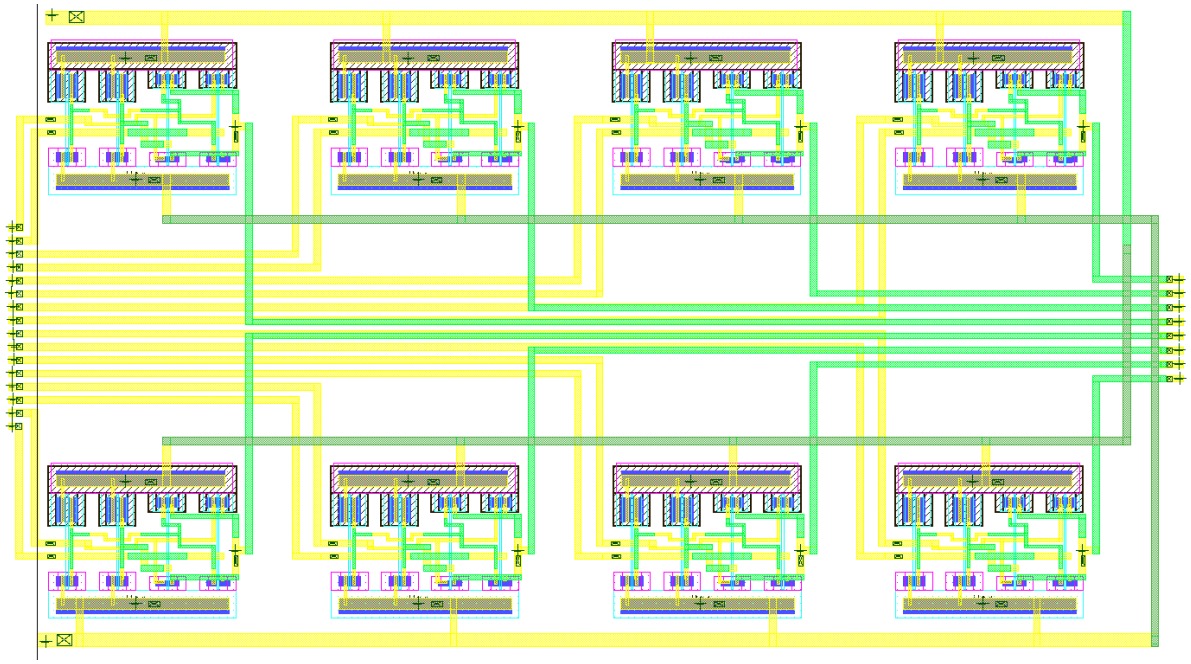


Figure 52. Eight-Bit XOR Layout

This figure shows the layout of the XOR gate, which has a more complex transistor structure compared to basic logic gates. The layout is carefully arranged to minimize internal routing complexity and reduce delay imbalance between input paths.

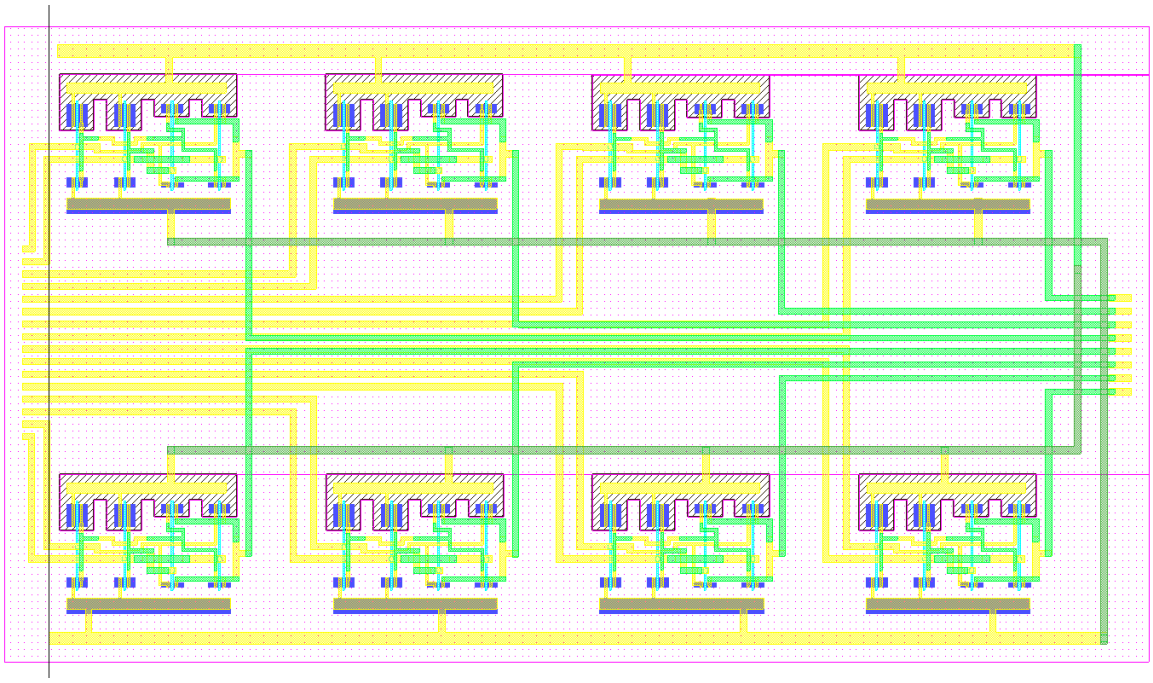


Figure 53. Eight-Bit XOR Layout with Parasitics

This figure includes parasitic extraction for the XOR gate. Due to increased routing complexity, parasitic effects play a more significant role in timing performance and are evaluated accordingly

4.2.4.NOT (Inverter)

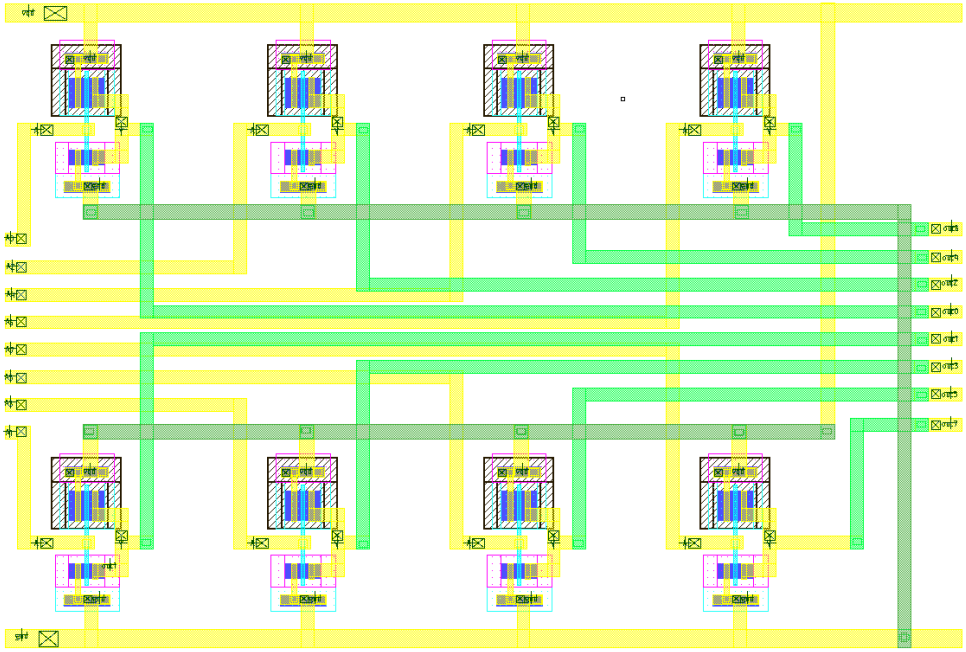


Figure 54. Eight-Bit NOT Layout

This figure illustrates the layout of the inverter. A simple and highly compact structure is used, providing minimal delay and low parasitic capacitance. The inverter serves as a fundamental building block throughout the design.

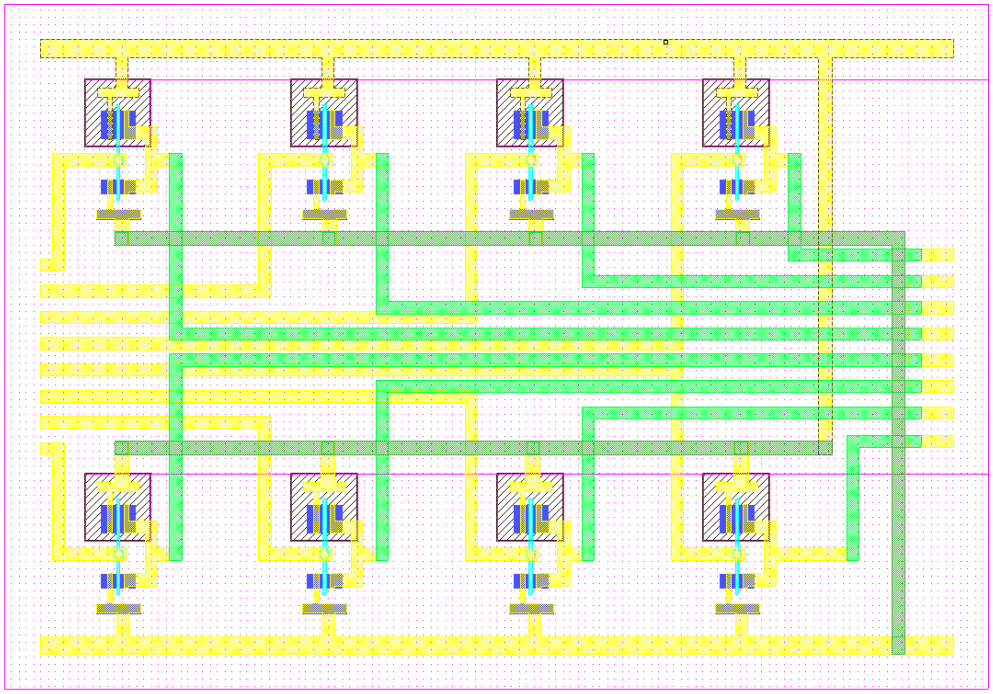


Figure 55. Eight-Bit NOT Layout with Parasitics

The parasitic-aware inverter layout confirms minimal resistance and capacitance effects, resulting in predictable and fast switching behavior.

4.3.Shifter Unit

4.3.1.Right Shift

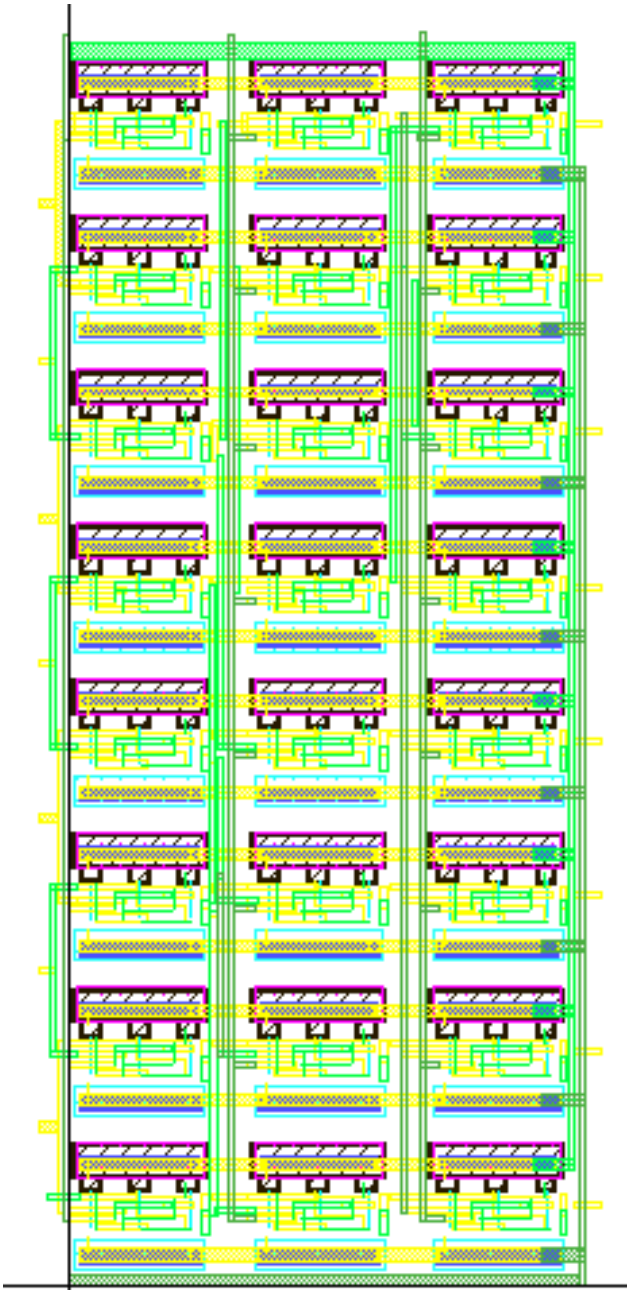


Figure 56.Right Shifter Layout

This figure shows the physical layout of the right shifter circuit implemented using multiplexers. The layout is organized in a modular and regular structure to ensure proper signal alignment between stages. Input and output routing is arranged to minimize wire crossings and reduce routing complexity, contributing to reliable signal propagation and compact area utilization.

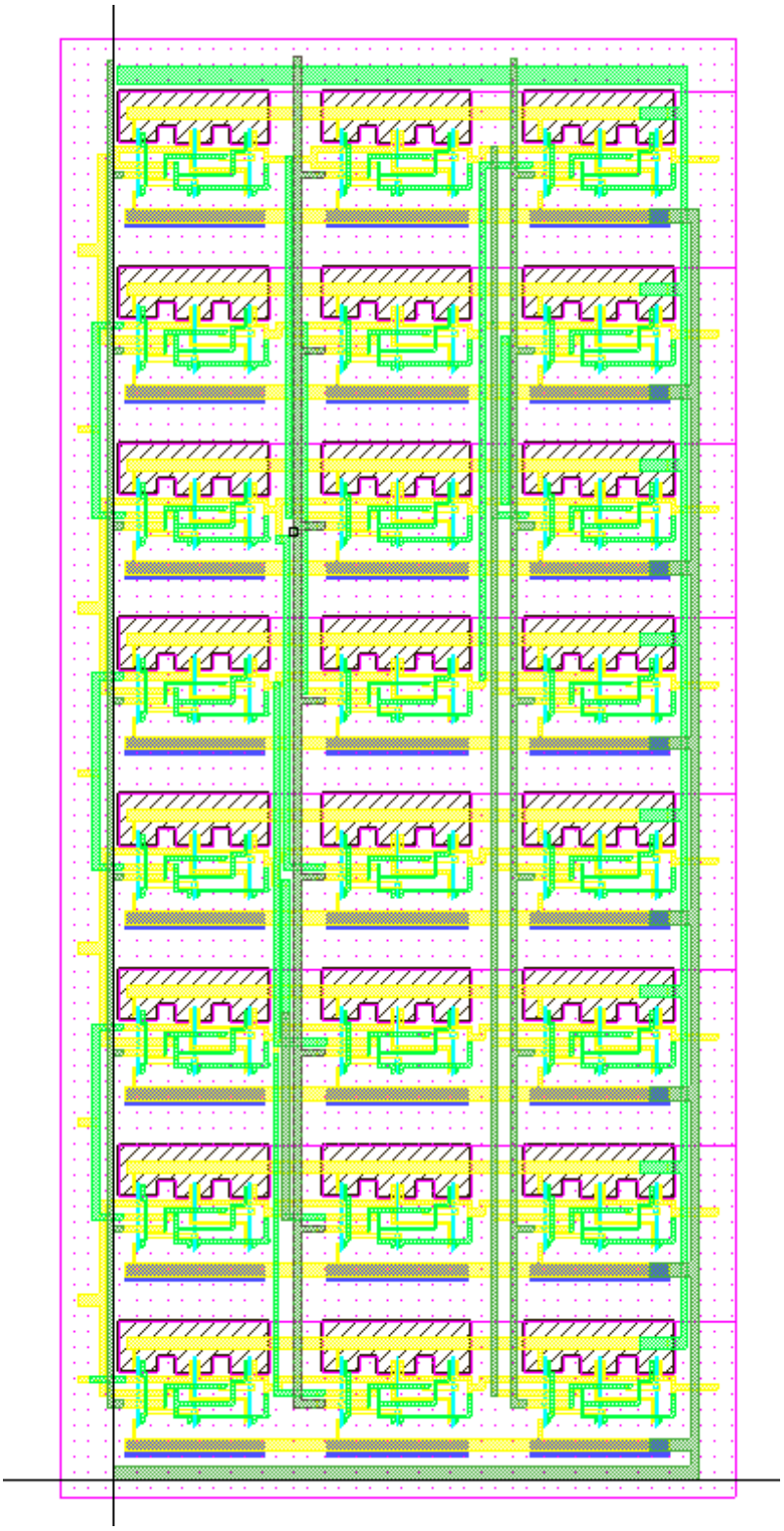


Figure 57.Right Shifter Layout with Parasitics

4.3.2.Left Shift

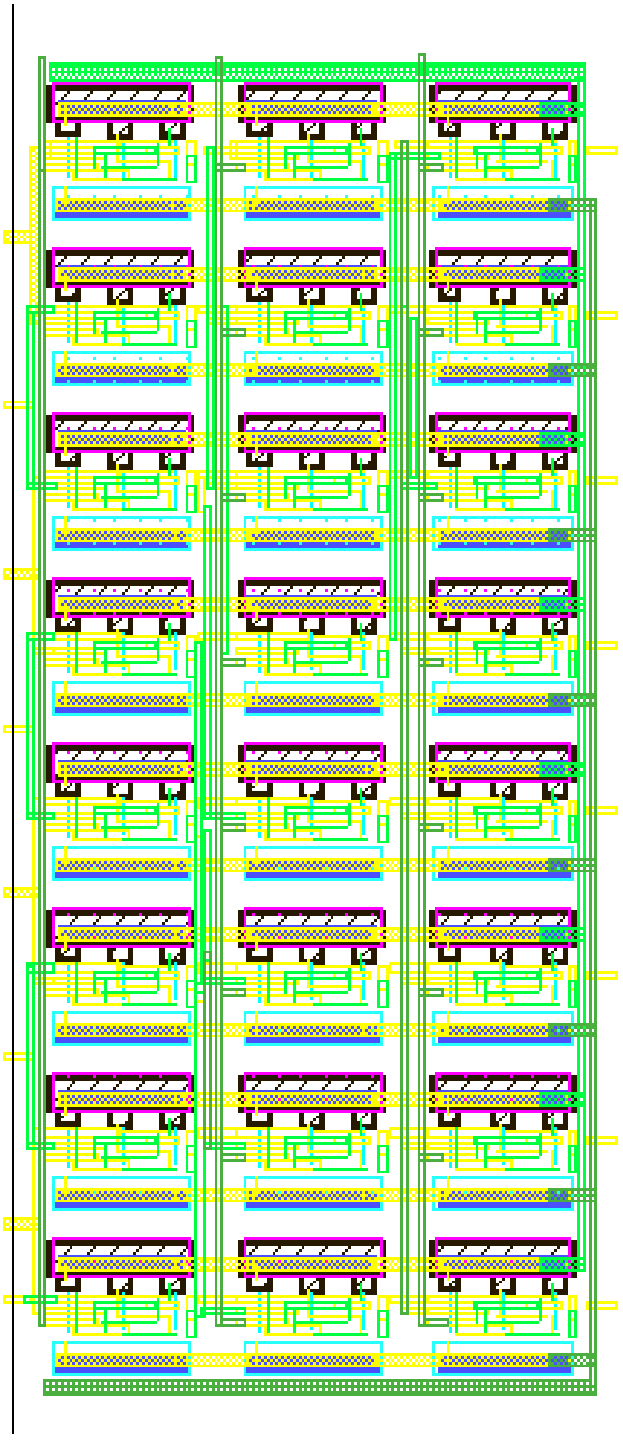


Figure 58.Left Shifter Layout

This figure illustrates the layout of the left shifter circuit. Similar to the right shifter, the design is implemented in a modular fashion using multiplexers to perform left-shift operations. The layout maintains consistent signal flow and balanced routing across all bit positions, ensuring correct bit alignment and efficient physical realization.

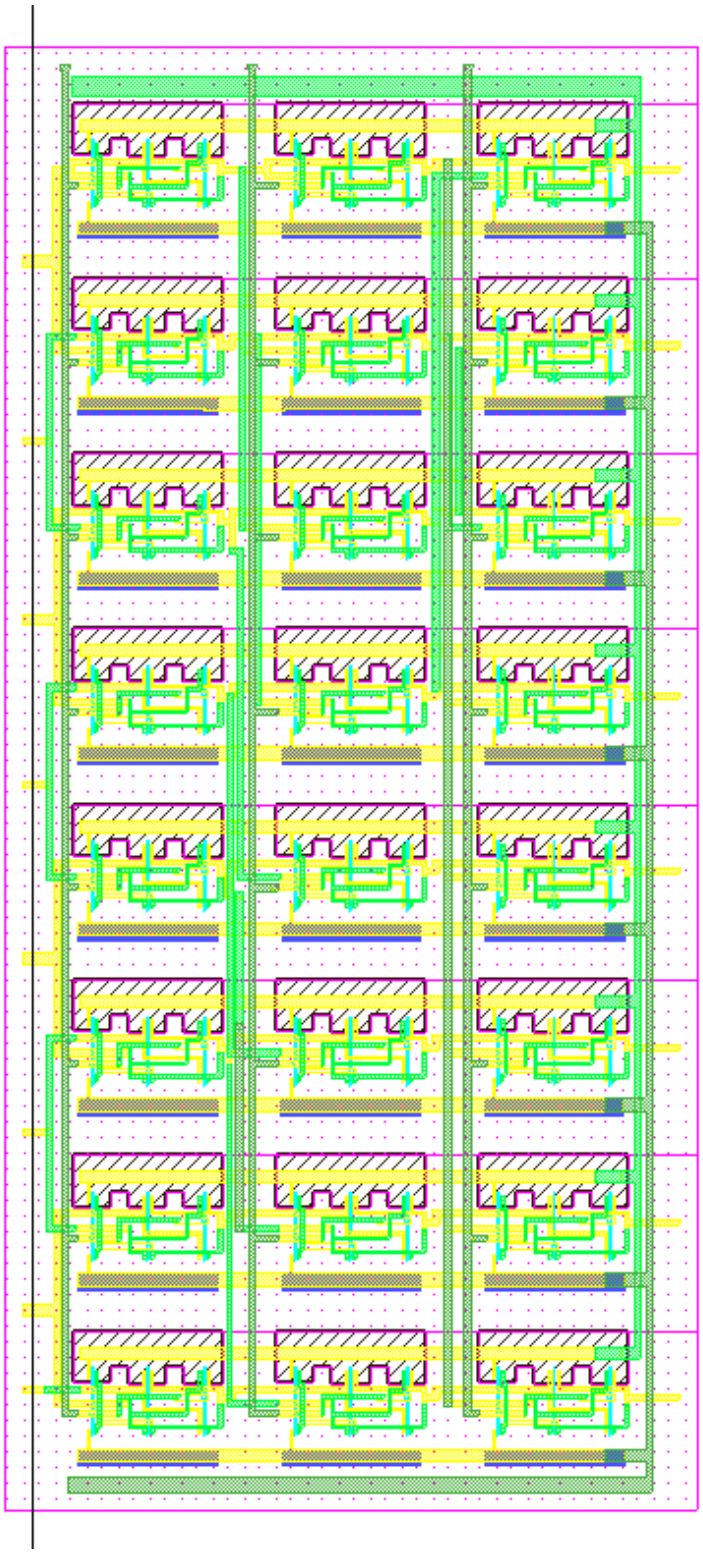


Figure 59.Left Shifter Layout with Parasitics

This figure shows the left shifter layout with extracted parasitic components. The impact of routing-related parasitic resistance and capacitance is included to accurately model post-layout behavior. This analysis allows verification that the left shifter maintains correct functionality and acceptable performance after physical implementation.

4.4.Opcode



Figure 60.Opcode Layout

This figure illustrates the physical layout of the opcode decoder logic used in the 8-bit ALU. The opcode block is responsible for decoding the control bits that select the desired ALU operation, such as arithmetic, logical, or shift functions.

The layout implements combinational decoding logic that distributes the opcode signals to the corresponding functional sub-blocks within the ALU. Careful placement and routing are applied to ensure balanced signal paths and to minimize routing congestion, since the opcode signals fan out to multiple destinations.

The structured layout organization helps reduce parasitic effects and ensures reliable control signal propagation. The correctness of the opcode decoder is functionally verified through the successful selection of ALU operations observed in simulation results, confirming proper control of the ALU data path.

4.5.Zero Flag

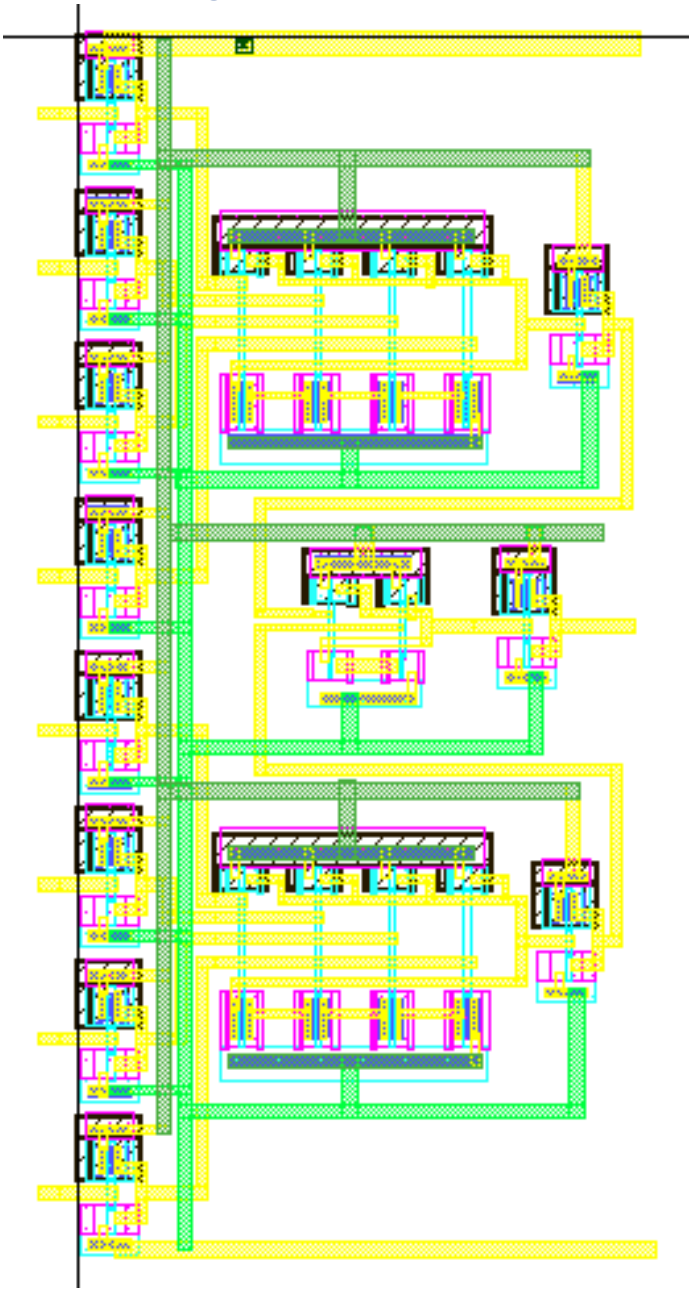


Figure 61.Zero Flag Layout

This figure illustrates the physical layout implementation of the Zero flag logic within the 8-bit ALU. The Zero flag is generated by monitoring the ALU output bus and determining whether all output bits are equal to logic ‘0’. In the layout, the Zero flag circuitry consists of combinational logic that aggregates the individual output bits and produces an active signal only when the entire result equals zero.

The layout is designed in a compact and structured manner to minimize routing complexity and parasitic effects. Signal paths from the ALU output bits are routed efficiently toward the Zero flag logic to ensure consistent timing behavior. Although the Zero flag logic is clearly observable at the layout level, a dedicated testbench and post-layout simulation were not performed for this flag. Instead, its correctness is inferred from the verified ALU output behavior and the direct dependency of the Zero flag on the output bus signals.

5. SIMULATION RESULTS

In this section, the simulation results of the designed sub-blocks and the complete 8-bit ALU are presented. Each functional unit is verified individually before being integrated into the full system. The simulations are performed to validate functional correctness and to analyze signal behavior under different operating conditions.

5.1. Simulation of Arithmetic Sub-Blocks

5.1.1. Adder Simulation

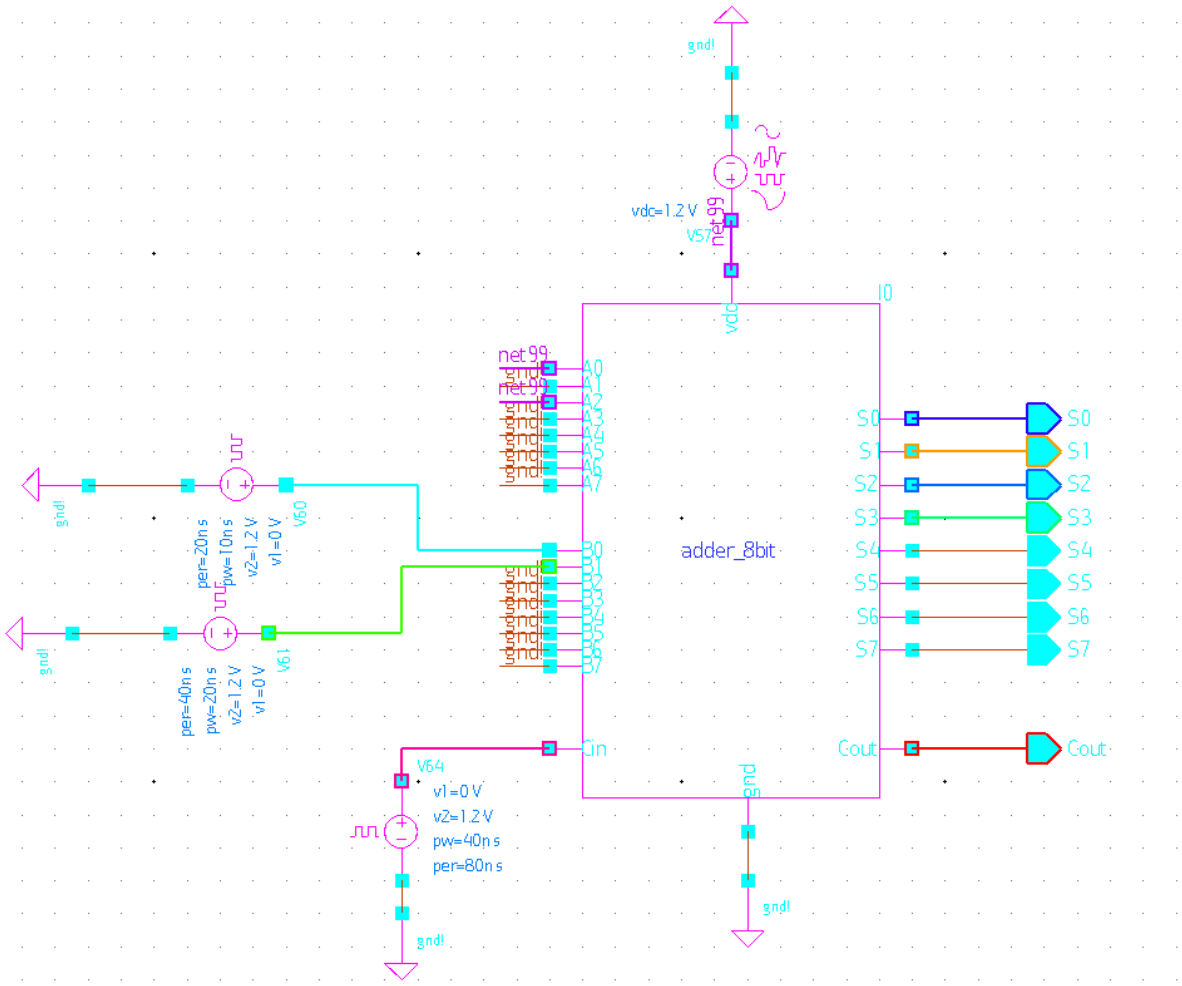


Figure 62.Adder Testbench

This figure shows the testbench configuration used for the adder simulation. The input A is set to a constant value of 00000101 (5) by connecting A0 and A2 to VDD, while the remaining A input lines are connected to GND. To generate a continuously increasing input pattern, B0 and B1 are driven by VPULSE sources, whereas the remaining B inputs (B2–B7) are tied to GND. The carry-in (Cin) input is also driven by a VPULSE source, configured such that it remains logic '0' during the first half of the period and logic '1' during the second half. This testbench setup enables the verification of correct adder operation under varying input and carry-in conditions.



Figure 63.Adder Waveforms

This figure shows the simulated output waveforms of the adder circuit under the applied testbench conditions. The input A is held constant, while the input B varies over time, resulting in a continuously changing sum output. The Cin signal is toggled periodically to verify correct carry-in handling. The resulting sum outputs (S[0]–S[7]) and carry-out (Cout) signals demonstrate correct arithmetic behavior, confirming proper addition and carry propagation throughout the simulation interval.

5.1.2. Subtractor Simulation

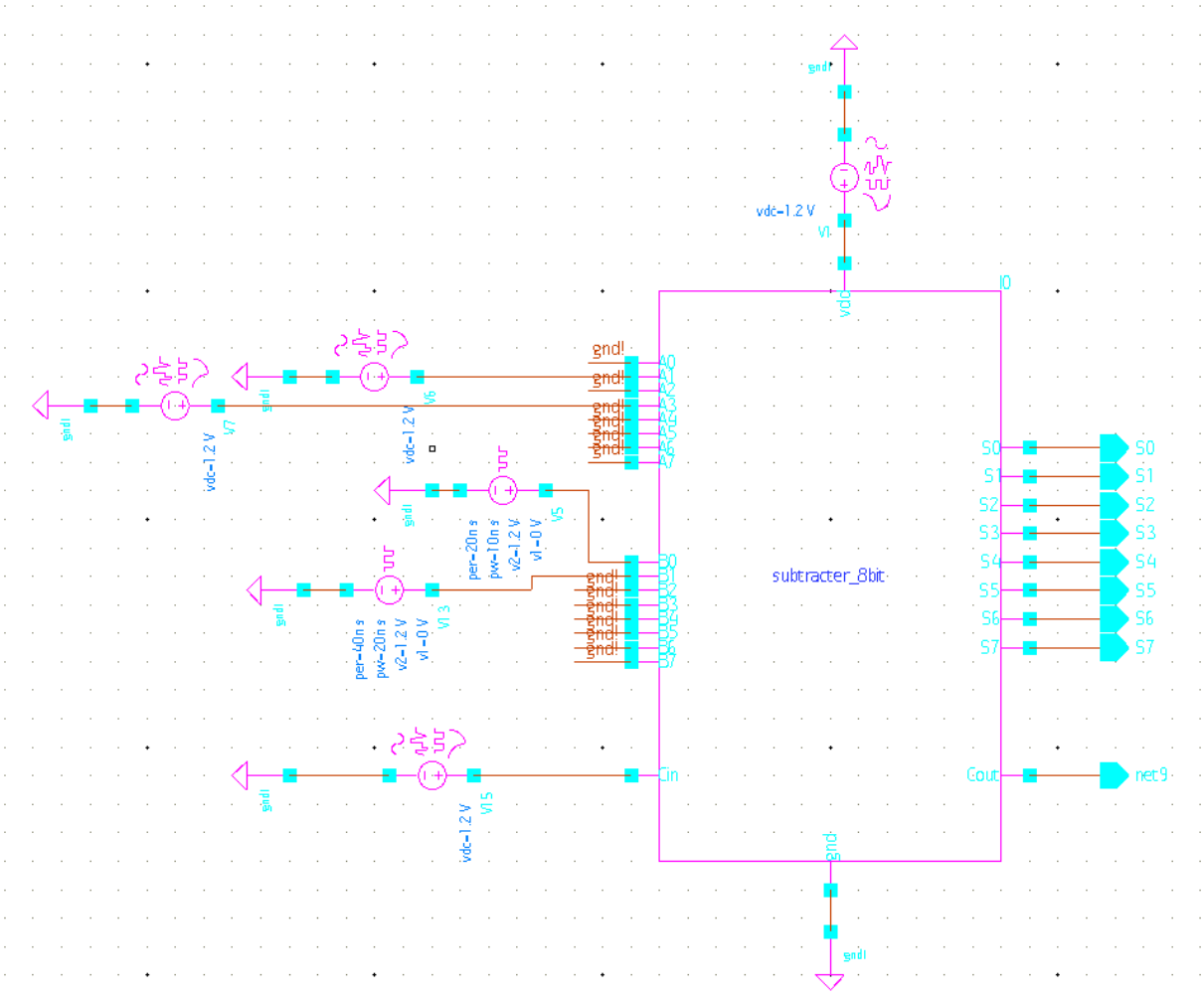


Figure 64. Subtractor Testbench

This figure illustrates the testbench configuration used for the simulation of the subtractor circuit. The A input bus is driven by predefined logic values to represent the minuend, while the B input bus is driven by VPULSE voltage sources to generate time-varying logic levels, enabling observation of different subtraction scenarios.

The (Cin) is controlled using a VPULSE source to allow evaluation of subtraction behavior both with and without an initial borrow. This setup ensures that the subtractor is tested under multiple operating conditions. The VDD pin is connected to a constant DC voltage source to provide the required supply voltage, and the GND pin is connected to ground to establish a stable reference level. Output pins are monitored directly to observe the resulting difference and carry/borrow signals.



Figure 65.Subtractor Waveforms

This figure presents the transient simulation waveforms of the subtractor circuit under the applied testbench conditions. The output difference bits respond correctly to changes in the input operands and the borrow input. When the input values remain constant, the output remains stable, and when either the B input or the borrow input changes, the output difference updates accordingly.

Correct subtractor operation is verified by comparing the output waveforms with the expected binary subtraction results. The observed difference bits match the theoretical subtraction outcomes for each input combination, while the carry/borrow output reflects correct borrow propagation behavior. The absence of incorrect transitions or unexpected glitches confirms stable and reliable subtractor functionality throughout the simulation interval.

5.1.3. Multiplier Simulation

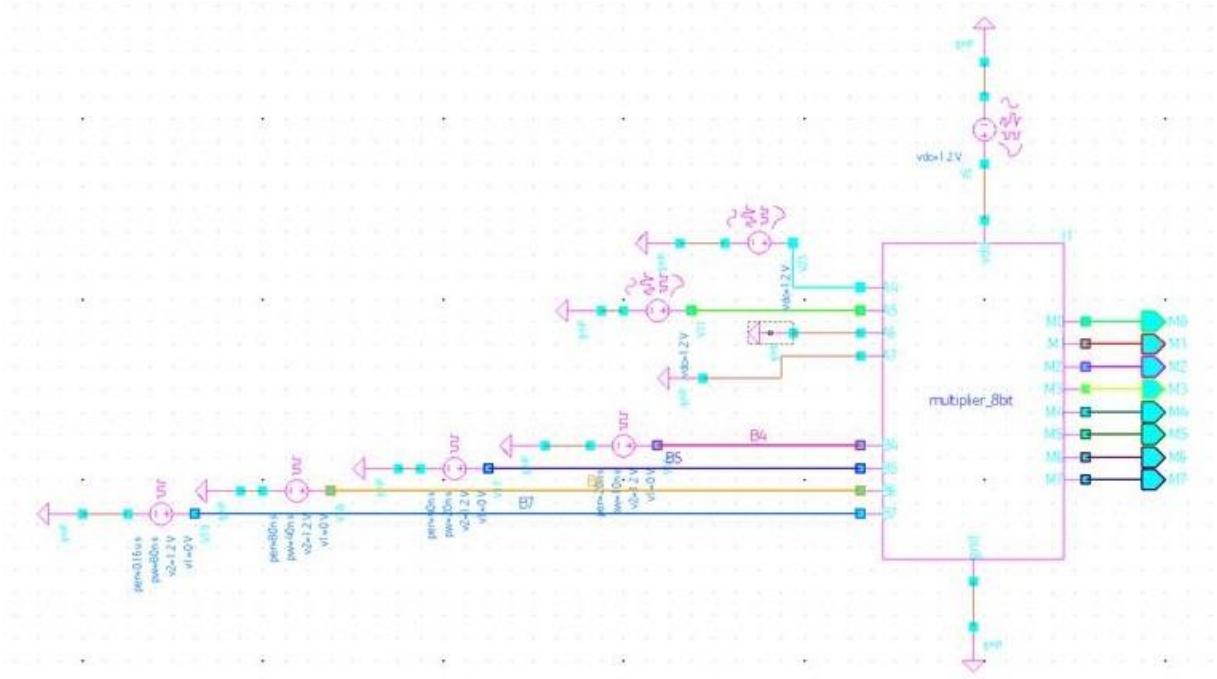


Figure 66.Multiplier Testbench

This figure illustrates the testbench configuration used for the simulation of the multiplier circuit. The input operand buses are driven by voltage sources configured to apply predefined binary values and time-varying signals. One operand is assigned fixed logic levels to represent a constant multiplicand, while the other operand is driven by VPULSE sources to generate varying input values over time. This configuration enables observation of the multiplier response under different input combinations.

The VPULSE sources are configured with appropriate periods and duty cycles to ensure systematic transitions of the input bits, allowing verification of correct multiplication behavior for multiple operand values. The VDD pin is connected to a constant DC voltage source to provide the required supply voltage, and the GND pin is connected to ground to establish a stable reference level during simulation.

The output pins are monitored directly to observe the generated product bits. This testbench setup ensures controlled input application and enables functional verification of the multiplier circuit under dynamic operating conditions.



Figure 67.Multiplier Waveforms

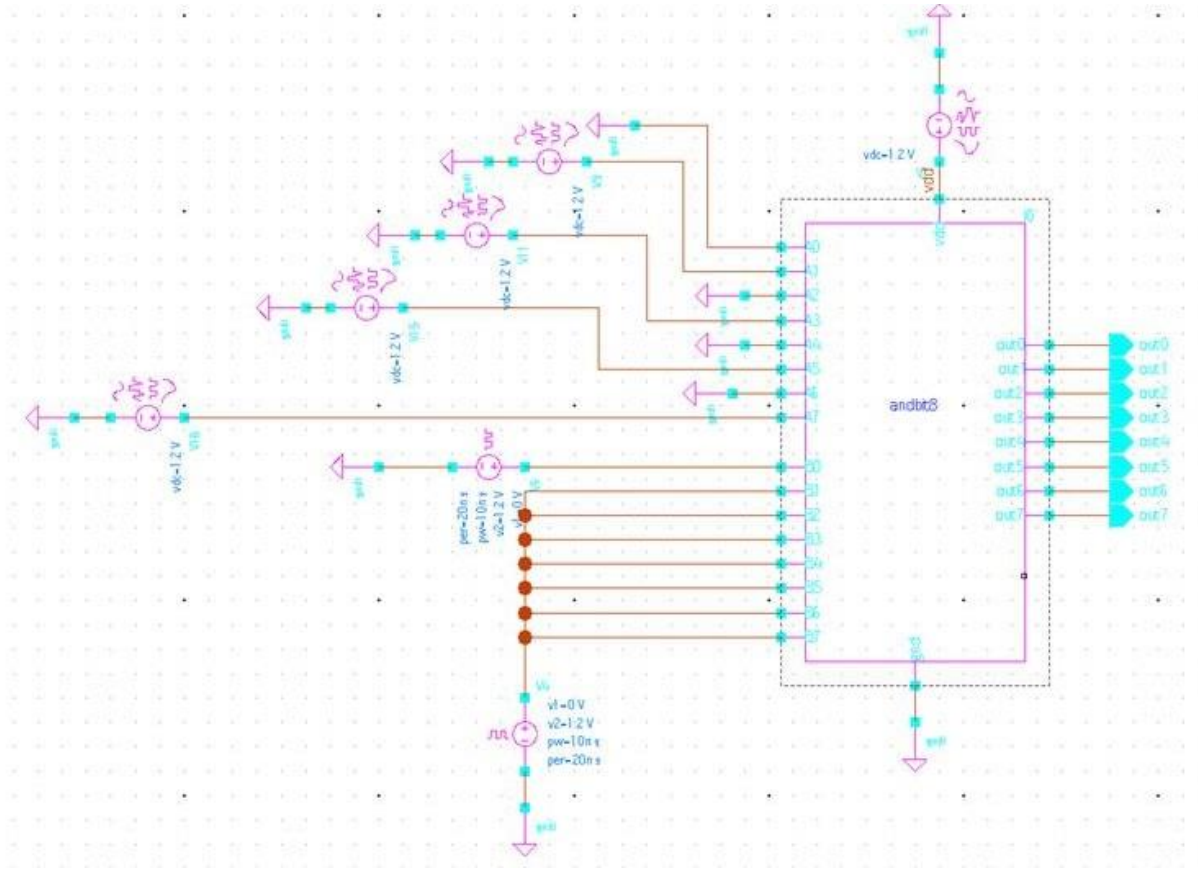
This figure presents the transient simulation waveforms of the multiplier circuit under the applied testbench conditions. One operand is held at predefined logic levels, while the other operand varies over time according to the VPULSE-driven input signals. As the input values change, the output product bits update accordingly, reflecting the expected results of the multiplication operation.

Correct multiplier functionality is verified by comparing the output waveforms with the expected binary multiplication results for the corresponding input combinations. When the input operands remain constant, the output product also remains stable. Conversely, when the varying operand changes its value, the output product transitions consistently to the new correct value. The observed timing and bit-level behavior of the output signals match the theoretical multiplication outcomes, confirming correct operation of the multiplier.

The absence of incorrect transitions or unexpected glitches further indicates stable and reliable multiplier behavior throughout the simulation interval.

5.2. Simulation of Logic Sub-Blocks

5.2.1 AND Gate Simulation


vdc=1.2 V
vdd
andbit8
out0
out1
out2
out3
out4
out5
out6
out7
vdc=1.2 V
vdc=1.2 V
vdc=1.2 V

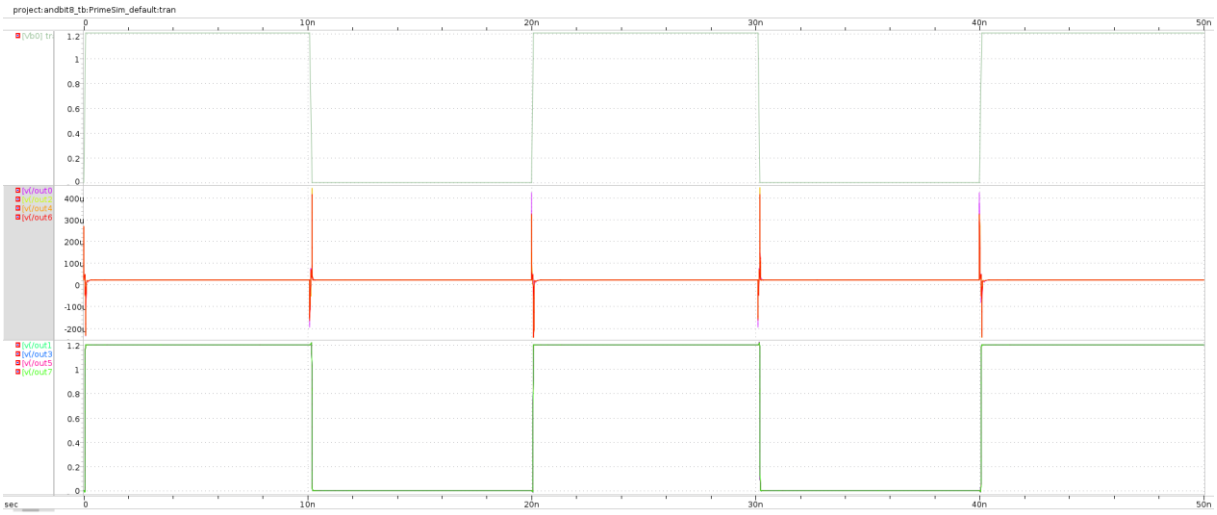


Figure 69.AND Waveforms

This figure shows the transient simulation waveforms of the AND gate for the applied testbench conditions. Between 0 ns and 10 ns, all B input bits are at logic ‘1’, corresponding to B = 11111111, while the A input remains fixed at 10101010.

Under these conditions, the AND operation is evaluated bitwise. Output bits OUT0, OUT2, OUT4, and OUT6 remain at logic ‘0’, as the corresponding A bits are ‘0’. Output bits OUT1, OUT3, OUT5, and OUT7 transition to logic ‘1’, since both A and B bits are at logic ‘1’. As a result, the observed output value is 10101010, which matches the expected bitwise AND result.

The waveform behavior is consistent with the AND gate truth table, confirming correct functional operation.

5.2.2 OR Gate Simulation

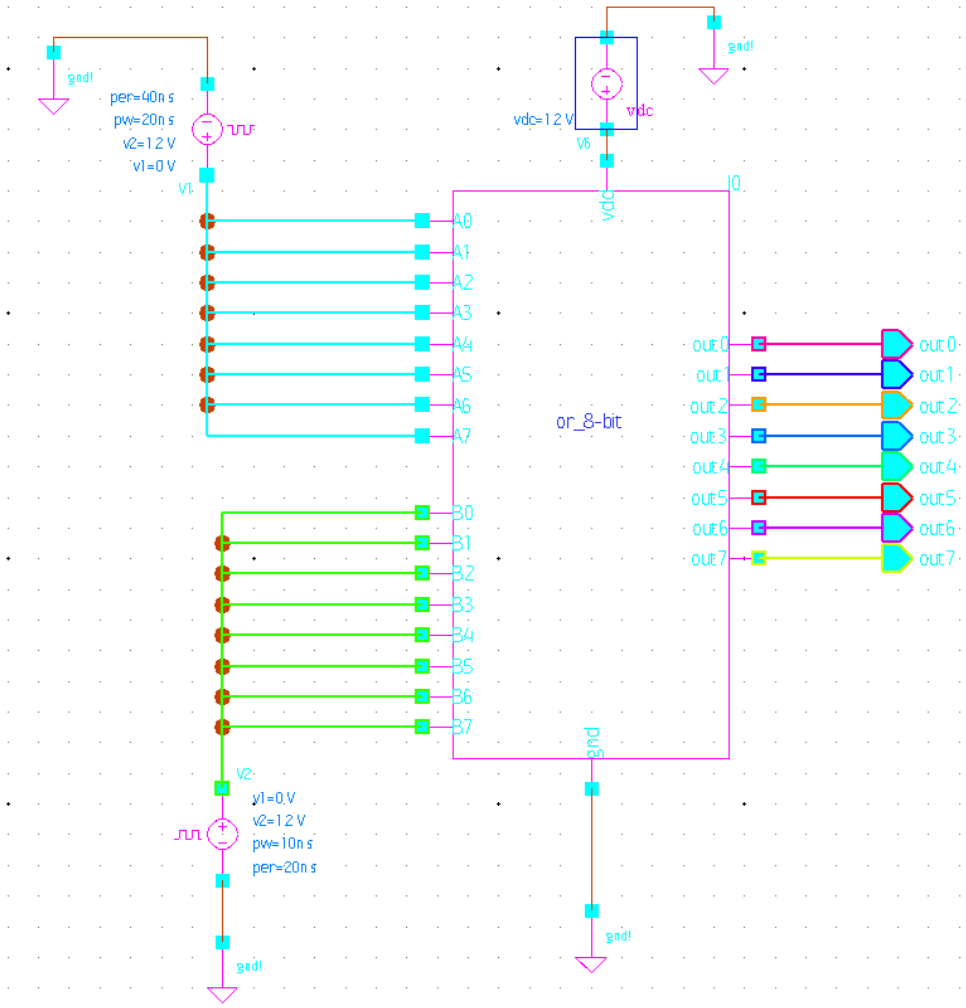


Figure 70.OR Testbench

This figure illustrates the testbench setup used for the simulation of the OR gate. The A input bus is assigned fixed logic values to establish a reference pattern, while the B input bus is driven by identical VPULSE voltage sources to apply time-varying logic levels simultaneously across all bit positions. This configuration ensures controlled input transitions and enables systematic verification of the OR operation. The VDD and GND pins are connected to constant supply and reference voltages to provide stable operating conditions during the simulation.

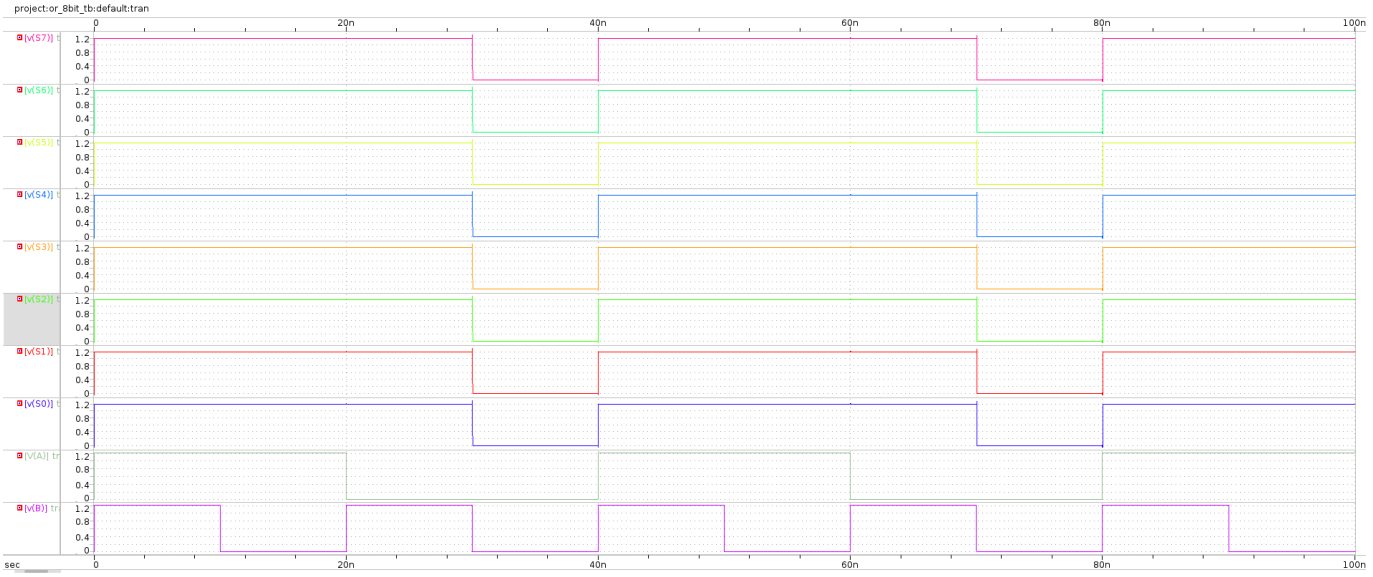


Figure 71.OR Waveforms

This figure presents the transient simulation waveforms of the OR gate under the applied testbench conditions. The output bits transition to logic ‘1’ whenever either A or B input bits are at logic ‘1’, and remain at logic ‘0’ only when both inputs are at logic ‘0’. The observed waveform behavior matches the OR gate truth table, confirming correct logical operation across all bit positions.

5.2.3 XOR Gate Simulation

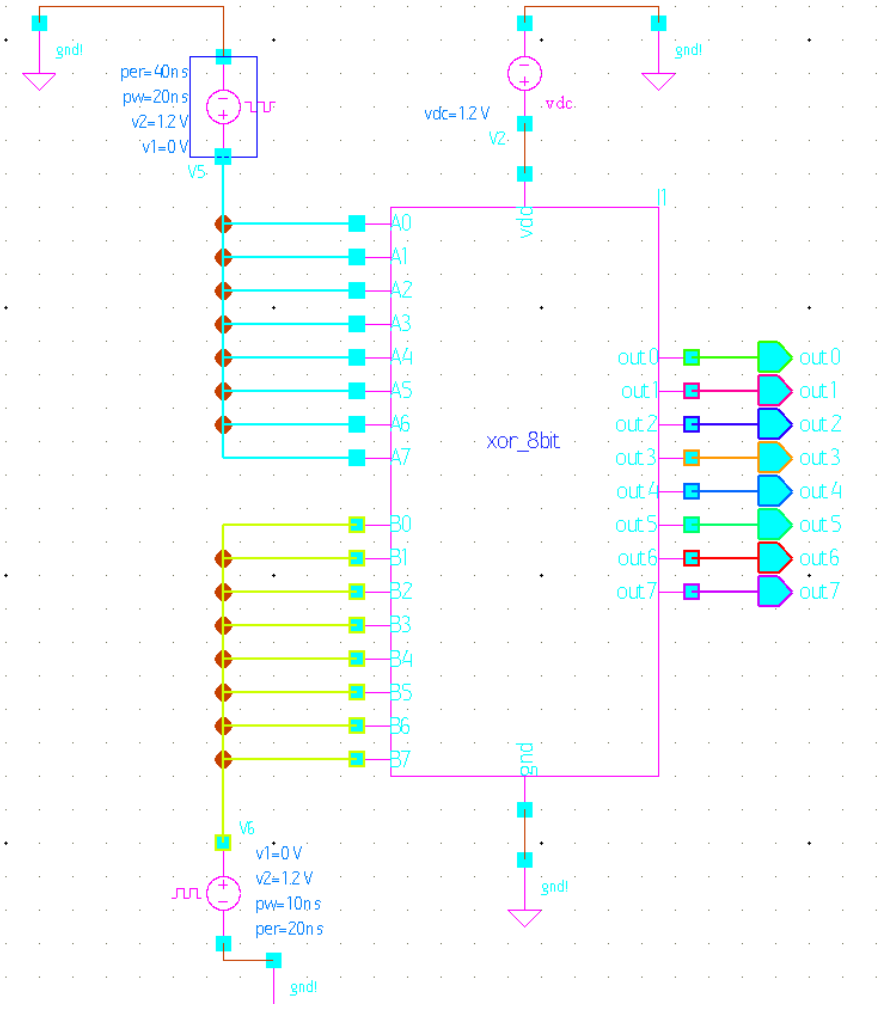


Figure 72.XOR Testbench

This figure shows the testbench configuration used for the simulation of the XOR gate. The A input bus is assigned fixed logic values to create a reference pattern across all bit positions. The B input bus is driven by identical VPULSE voltage sources so that all B bits toggle simultaneously over time. This configuration allows controlled variation of the B inputs while keeping the A inputs constant, enabling clear observation of XOR behavior.

The VDD pin is connected to a constant DC voltage source to provide the required supply voltage, and the GND pin is connected to ground to establish a stable reference level. The output pins are monitored without additional constraints in order to directly observe the result of the XOR operation. This testbench setup ensures that all relevant XOR input conditions are applied systematically and evaluated under dynamic input transitions.

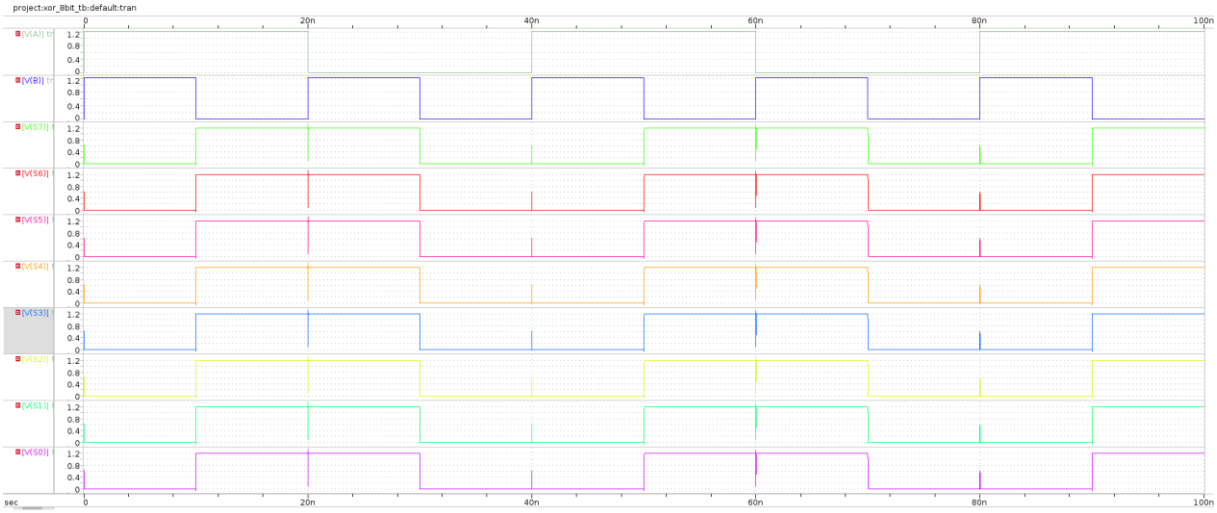


Figure 73.XOR Waveforms

This figure presents the transient simulation waveforms of the XOR gate for the applied testbench conditions. The XOR operation is verified by comparing the input and output signals on a bit-by-bit basis. As observed from the waveforms, the output bit transitions to logic ‘1’ only when the corresponding A and B input bits are different, and remains at logic ‘0’ when the input bits are the same.

During time intervals where A = B, the output bits remain at logic ‘0’, confirming correct suppression of identical input states. Conversely, when A ≠ B, the output bits transition to logic ‘1’, which is consistent with the XOR truth table. The absence of incorrect transitions or unexpected glitches further confirms stable and correct XOR functionality throughout the simulation interval.

5.2.4 NOT (Inverter) Simulation

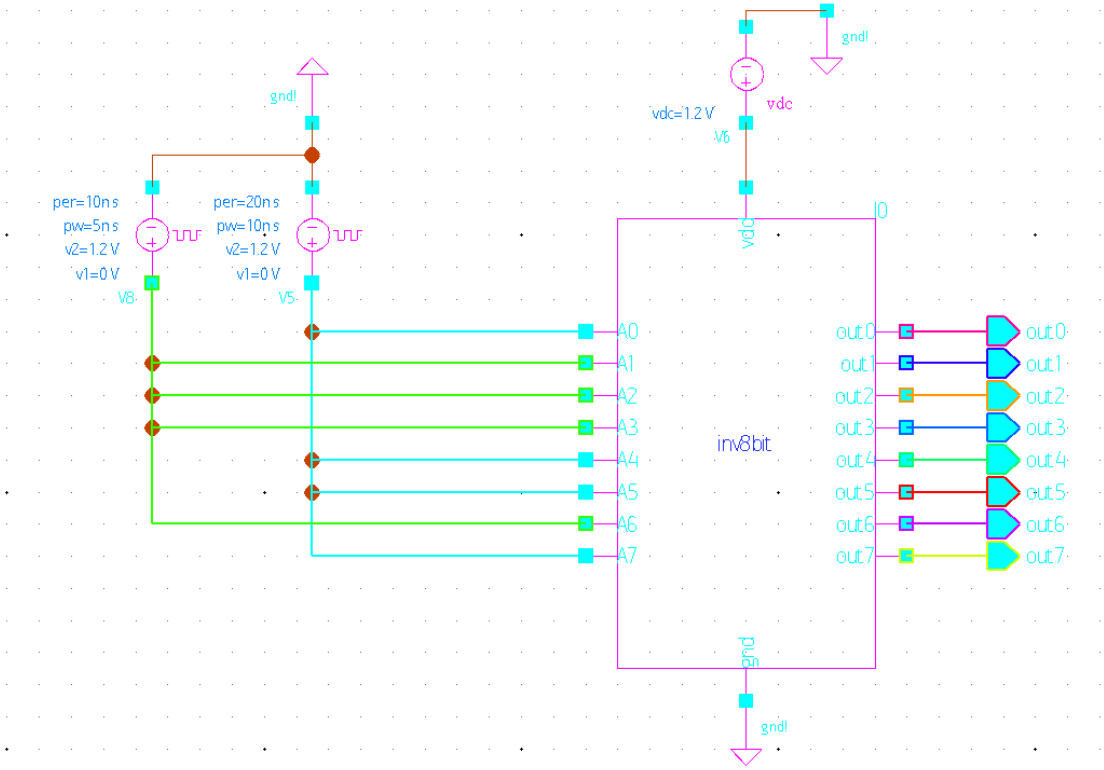


Figure 74.NOT Testbench

This figure illustrates the testbench configuration used for the simulation of the NOT (inverter) circuit. The input pins are driven by VPULSE voltage sources in order to apply time-varying logic levels. The VPULSE source is configured to alternate between logic ‘0’ and logic ‘1’, allowing observation of the inverter response for both input states.

The VDD pin of the inverter is connected to a constant DC voltage source to provide the required supply voltage, while the GND pin is connected to ground to establish a stable reference level. This configuration ensures correct biasing of the inverter during the simulation.

The output pins are left unconstrained and monitored to observe the inverted output behavior. By applying a pulsed input signal, the output transitions are clearly observed as the logical complement of the input signal. This testbench setup enables verification of correct inverter functionality and confirms proper logic inversion under dynamic operating conditions.

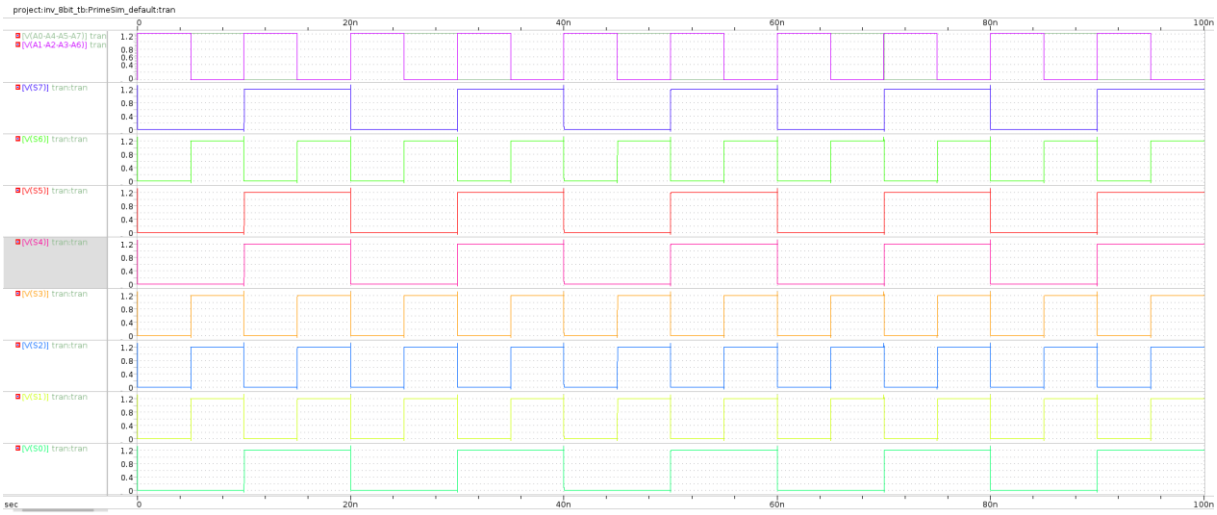


Figure 75.NOT Waveforms

This figure presents the transient simulation waveforms of the NOT (inverter) circuit. The input signal is driven by a VPULSE source and alternates periodically between logic ‘0’ and logic ‘1’. The output waveform is observed to be the logical complement of the input signal throughout the entire simulation interval.

Correct inverter operation is verified by comparing the input and output transitions. Whenever the input signal transitions to logic ‘1’, the output transitions to logic ‘0’, and when the input returns to logic ‘0’, the output transitions to logic ‘1’. The consistent inverse relationship between input and output signals confirms correct logic inversion. Additionally, the absence of unexpected glitches or incorrect transitions indicates stable and reliable inverter behavior under dynamic operating conditions.

5.3. Shifter Unit Simulation

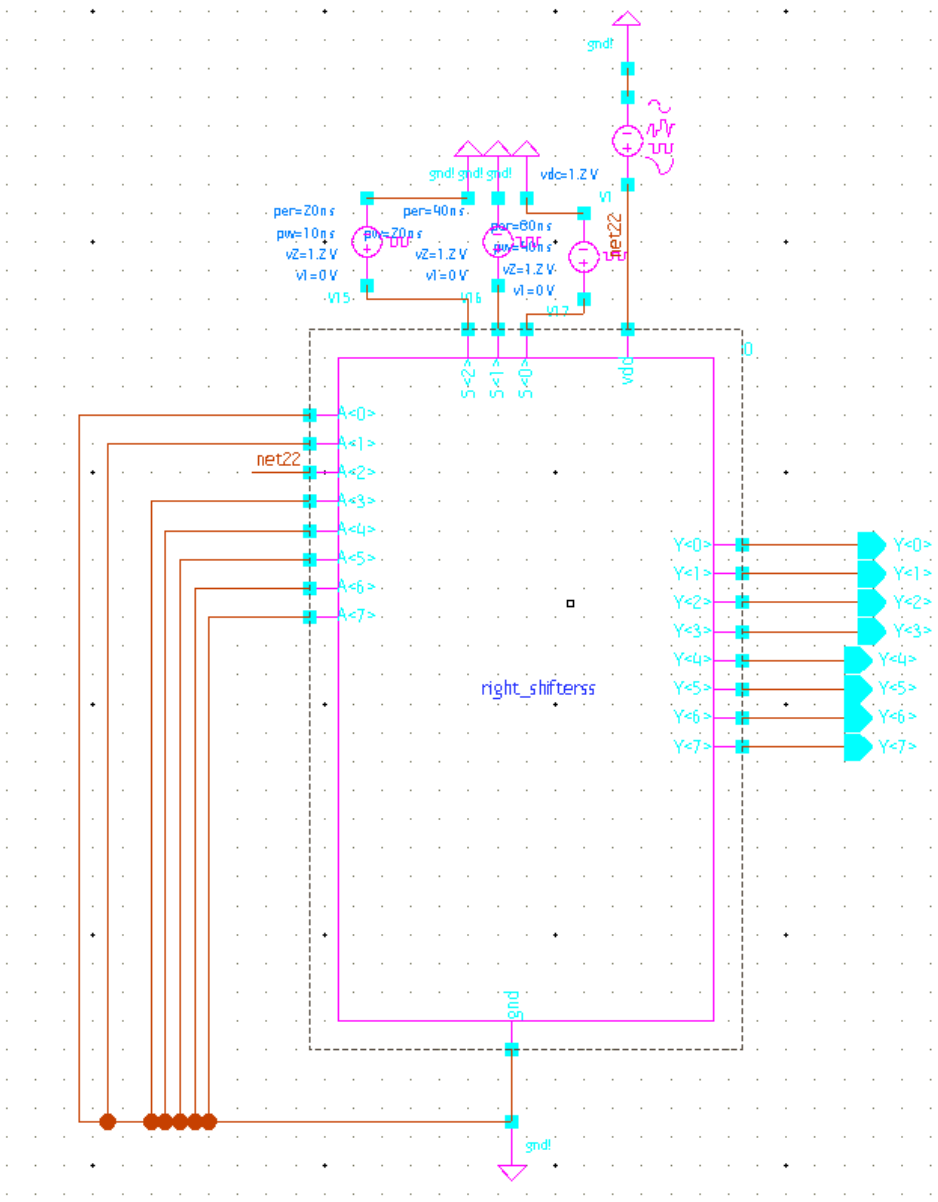


Figure 76.Right Shift Testbench

This figure illustrates the testbench configuration used for the simulation of the right shifter unit. The data input bus (X[7:0]) is driven by fixed logic values in order to clearly observe the bit-shifting behavior at the output. Assigning constant input data allows direct tracking of how each bit position is shifted when different shift control signals are applied.

The shift control inputs (S0, S1, S2) are driven by VPULSE voltage sources with different periods and duty cycles. These control signals are configured to generate various binary combinations over time, enabling verification of multiple right-shift operations within a single simulation run. By changing the control inputs sequentially, different shift amounts are applied to the input data, allowing systematic evaluation of the shifter functionality.

The VDD pin is connected to a constant DC voltage source to provide the required supply voltage, while the GND pin is connected to ground to establish a stable reference level. The output bus (Y[7:0]) is

monitored directly to observe the shifted data bits. This testbench setup ensures controlled input application and enables functional verification of correct right-shift operation under dynamic control conditions.

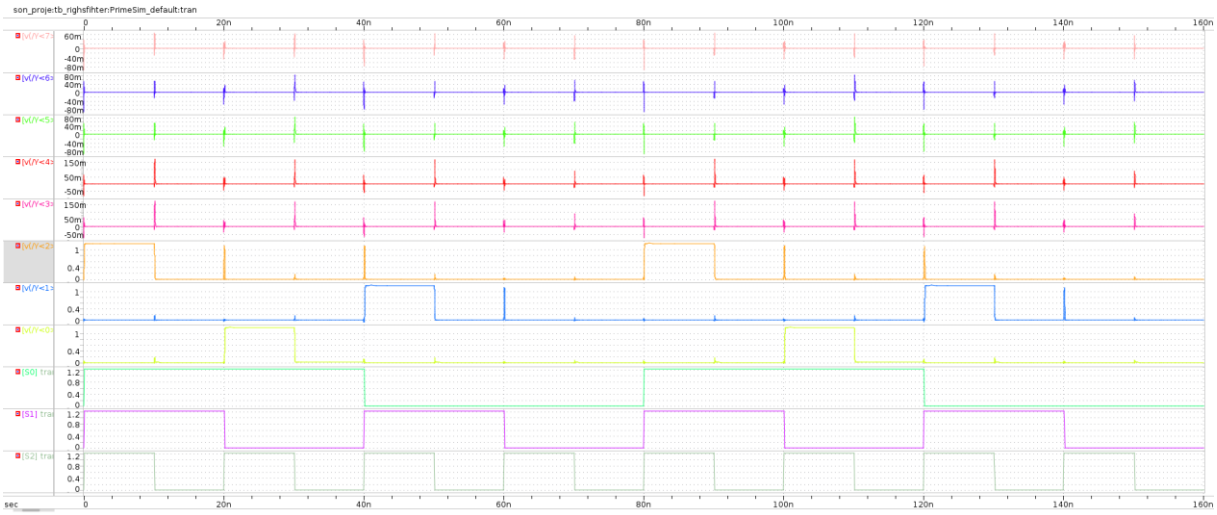


Figure 77.Right Shift Waveforms

This figure presents the simulated output waveforms of the 8-bit right shifter under the applied testbench conditions. The input data bits are driven with defined logic patterns, while the shift control signals are toggled periodically to apply different right-shift amounts during the simulation interval.

As observed in the waveforms, the output bits shift toward the least significant bit (LSB) side in accordance with the selected shift control signals. For each shift operation, the vacated most significant bit (MSB) positions are filled with logic ‘0’, confirming correct logical right-shift behavior. The correspondence between the input data transitions and the resulting shifted outputs demonstrates that the shifter correctly propagates data according to the applied control signals.

The correctness of the right-shift operation is verified by comparing the output bit patterns with the expected shifted versions of the input data for each control state. The stable and consistent transitions observed in the waveforms confirm proper functionality of the right shifter under dynamic operating conditions.

6.CONCLUSION

In this project, a complete 8-bit Arithmetic Logic Unit (ALU) was successfully designed, implemented, and verified using a full VLSI design flow. The ALU supports a comprehensive set of arithmetic, logical, and shift operations, including addition, subtraction, multiplication, AND, OR, XOR, NOT, left shift, and right shift. Operation selection is achieved through a dedicated opcode decoder, enabling correct and deterministic control of the ALU datapath.

The design process began at the RTL level using Verilog HDL, followed by schematic-level implementation and transistor-level realization of each sub-block. All major functional units were individually verified through dedicated testbenches, ensuring correct logical behavior prior to integration. The modular design approach simplified debugging, verification, and physical implementation.

Physical layouts of all major sub-blocks were generated with careful attention to regularity, routing efficiency, and parasitic effects. Post-layout parasitic extraction was performed for critical units, allowing realistic evaluation of timing and signal integrity. The obtained results confirm that the designed circuits maintain correct functionality under layout-aware conditions.

Status flag generation was partially implemented and analyzed. The Carry flag was verified through arithmetic operations, while the Zero flag logic was successfully implemented and observed at the layout level. The Sign flag was derived from the most significant bit of the output, consistent with two’s complement representation. Overflow flag implementation was left for future work.

Overall, the proposed 8-bit ALU design demonstrates the fundamental principles of digital arithmetic, logic design, and VLSI implementation. The project provides a solid foundation for extending the architecture to wider word lengths, enhancing flag support, and integrating the ALU into more complex processor systems.

CODE

In this section, the Verilog HDL codes used for the RTL-level design of the arithmetic and logical units are presented. These codes describe the register-transfer level functionality of the designed modules and are used solely for RTL representation prior to schematic and layout generation.

Arithmetic Unit

```
module full_adder (
    input  wire A,
    input  wire B,
    input  wire Cin,
    output wire SUM,
    output wire Cout
);

    assign SUM  = A ^ B ^ Cin;
    assign Cout = (A & B) | (A & Cin) | (B & Cin);

endmodule

```

Figure 78. Verilog HDL code of the 1-bit full adder

```
module adder8 (
    input  wire [7:0] A,
    input  wire [7:0] B,
    input  wire       Cin,
    output wire [7:0] SUM,
    output wire       Cout
);

    wire [8:0] c;
    assign c[0] = Cin;

    genvar i;
    generate
        for (i = 0; i < 8; i = i + 1) begin : FA_CHAIN
            full_adder fa (
                .A   (A[i]),
                .B   (B[i]),
                .Cin (c[i]),
                .SUM (SUM[i]),
                .Cout(c[i+1])
            );
        end
    endgenerate

    assign Cout = c[8];

endmodule
```

Figure 79. Verilog HDL code of the 8-bit adder

```
1  module subtractor8 (
2      input  wire [7:0] A,
3      input  wire [7:0] B,
4      output wire [7:0] D,
5      output wire       Bout
6  );
7
8      wire [7:0] Bn;      // inverted B
9      wire [7:0] C;       // carry chain
10
11     assign Bn = ~B;
12
13     // Bit 0: Cin = 1 (two's complement)
14     full_adder FA0 (
15         .A   (A[0]),
16         .B   (Bn[0]),
17         .Cin (1'b1),
18         .SUM (D[0]),
19         .Cout(C[0])
20     );
21
22     // Bits 1..7: ripple carry
23     genvar i;
24     generate
25         for (i = 1; i < 8; i = i + 1) begin : FA_CHAIN
26             full_adder FAi (
27                 .A   (A[i]),
28                 .B   (Bn[i]),
29                 .Cin (C[i-1]),
30                 .SUM (D[i]),
31                 .Cout(C[i])
32             );
33         end
34     endgenerate
35
36     // Borrow out = NOT(Carry out)
37     assign Bout = ~C[7];
38
39 endmodule
40
```

Figure 80. Verilog HDL code of the 8-bit subtractor

```v
module multiplier_8bit (
    input  wire [7:0] A,
    input  wire [7:0] B,
    output wire [8:0] M
);

    // Üst nibble'ları kullanıyoruz (4x4 multiplier)
    wire [3:0] a = A[7:4];
    wire [3:0] b = B[7:4];

    // Partial products (nand+inv = AND)
    wire pp00 = a[0] & b[0];
    wire pp01 = a[0] & b[1];
    wire pp02 = a[0] & b[2];
    wire pp03 = a[0] & b[3];

    wire pp10 = a[1] & b[0];
    wire pp11 = a[1] & b[1];
    wire pp12 = a[1] & b[2];
    wire pp13 = a[1] & b[3];

    wire pp20 = a[2] & b[0];
    wire pp21 = a[2] & b[1];
    wire pp22 = a[2] & b[2];
    wire pp23 = a[2] & b[3];

    wire pp30 = a[3] & b[0];
    wire pp31 = a[3] & b[1];
    wire pp32 = a[3] & b[2];
    wire pp33 = a[3] & b[3];

    wire [7:0] P;

    // P[0]
    assign P[0] = pp00;

    // Column 1
    wire c1;
    full_adder FA1 (
        .A(pp01), .B(pp10), .Cin(1'b0),
        .SUM(P[1]), .Cout(c1)
    );

    // Column 2
    wire s2a, c2a, c2;
    full_adder FA2a (
        .A(pp02), .B(pp11), .Cin(pp20),
        .SUM(s2a), .Cout(c2a)
    );
    full_adder FA2b (
        .A(s2a), .B(c1), .Cin(1'b0),
        .SUM(P[2]), .Cout(c2)
    );

    // Column 3
    wire s3a, c3a, s3b, c3b, c3;
    full_adder FA3a (
        .A(pp03), .B(pp12), .Cin(pp21),
        .SUM(s3a), .Cout(c3a)
    );
    full_adder FA3b (
        .A(s3a), .B(pp30), .Cin(c2a),
        .SUM(s3b), .Cout(c3b)
    );
    full_adder FA3c (
        .A(s3b), .B(c2), .Cin(1'b0),
        .SUM(P[3]), .Cout(c3)
    );

    // Column 4
    wire s4a, c4a, s4b, c4b, c4;
    full_adder FA4a (
        .A(pp13), .B(pp22), .Cin(pp31),
        .SUM(s4a), .Cout(c4a)
    );
    full_adder FA4b (
        .A(s4a), .B(c3a), .Cin(c3b),
        .SUM(s4b), .Cout(c4b)
    );
    full_adder FA4c (
        .A(s4b), .B(c3), .Cin(1'b0),
        .SUM(P[4]), .Cout(c4)
    );

    // Column 5
    wire s5a, c5a, s5b, c5b;
    full_adder FA5a (
        .A(pp23), .B(pp32), .Cin(c4a),
        .SUM(s5a), .Cout(c5a)
    );
    full_adder FA5b (
        .A(s5a), .B(c4b), .Cin(c4),
        .SUM(P[5]), .Cout(c5b)
    );

    // Column 6
    wire c6;
    full_adder FA6 (
        .A(pp33), .B(c5a), .Cin(c5b),
        .SUM(P[6]), .Cout(c6)
    );

    // Column 7
    assign P[7] = c6;

    // 9-bit output
    assign M = {1'b0, P};

endmodule
```

Figure 81. Verilog HDL code of the 8-bit multiplier

Logical Unit

```
1  module andbit8 (
2      input  wire [7:0] A,
3      input  wire [7:0] B,
4      output wire [7:0] Out
5  );
6      assign Out = A & B;
7  endmodule
```

Figure 82. Verilog HDL code of the 8-bit AND

```
1  module orbit8 (
2      input  wire [7:0] A,
3      input  wire [7:0] B,
4      output wire [7:0] Out
5  );
6      assign Out = A | B;
7  endmodule
8
```

Figure 83. Verilog HDL code of the 8-bit OR

```
1  module xorbit8 (
2      input  wire [7:0] A,
3      input  wire [7:0] B,
4      output wire [7:0] Out
5  );
6      assign Out = A ^ B;
7  endmodule
8
```

Figure 84. Verilog HDL code of the 8-bit XOR

```
1  module invbit8 (
2      input  wire [7:0] A,
3      output wire [7:0] Out
4  );
5      assign Out = ~A;
6  endmodule
```

Figure 84. Verilog HDL code of the 8-bit NOT(Inverter)

Shifter Unit

```
1   module shifter8 (
2       input  wire [7:0] A,
3       input  wire       D,   // D=0: left shift, D=1: right shift
4       output wire [7:0] Y
5   );
6
7       wire Dn;   // ~D
8       assign Dn = ~D;
9
10      wire [7:0] left_s;
11      wire [7:0] right_s;
12
13      // left shift by 1:  A << 1  (LSB filled with 0)
14      assign left_s  = {A[6:0], 1'b0};
15
16      // right shift by 1: A >> 1  (MSB filled with 0)
17      assign right_s = {1'b0, A[7:1]};
18
19      // Gate-style select (avoid ?: to reduce mux inference)
20      assign Y[0] = (left_s[0]  & Dn) | (right_s[0]  & D);
21      assign Y[1] = (left_s[1]  & Dn) | (right_s[1]  & D);
22      assign Y[2] = (left_s[2]  & Dn) | (right_s[2]  & D);
23      assign Y[3] = (left_s[3]  & Dn) | (right_s[3]  & D);
24      assign Y[4] = (left_s[4]  & Dn) | (right_s[4]  & D);
25      assign Y[5] = (left_s[5]  & Dn) | (right_s[5]  & D);
26      assign Y[6] = (left_s[6]  & Dn) | (right_s[6]  & D);
27      assign Y[7] = (left_s[7]  & Dn) | (right_s[7]  & D);
28
29  endmodule
30
```

Figure 86. Verilog HDL code of the 8-bit shifter unit

EIGHT-BIT ALU

```
module alu (
    input  wire [7:0] a,
    input  wire [7:0] b,
    input  wire [2:0] opcode,
    input  wire       shift_dir,   // 0: left, 1: right
    output reg  [7:0] s,
    output reg        carry,
    output reg        overflow,
    output wire       zero,
    output wire       sign
);

    wire [8:0] add_ext = {1'b0,a} + {1'b0,b};
    wire [8:0] sub_ext = {1'b0,a} + {1'b0,(~b)} + 9'b1;

    wire [7:0] add_res = add_ext[7:0];
    wire [7:0] sub_res = sub_ext[7:0];

    wire add_ovf = (~(a[7]^b[7])) & (add_res[7]^a[7]);
    wire sub_ovf = ( (a[7]^b[7])) & (sub_res[7]^a[7]);

    always @(*) begin
        s        = 8'b0;
        carry    = 1'b0;
        overflow = 1'b0;

        case (opcode)
            3'b000: begin // ADD
                s        = add_res;
                carry    = add_ext[8];
                overflow = add_ovf;
            end
            3'b001: begin // SUB
                s        = sub_res;
                carry    = sub_ext[8];   // borrow flag yerine çoğu tasarımda carry/no-borrow gibi kullanılır
                overflow = sub_ovf;
            end
            3'b010: begin // MUL (truncate)
                s = (a * b);             // otomatik [7:0] kesilir
            end
            3'b011: s = (a & b);         // AND
            3'b100: s = (a | b);         // OR
            3'b101: s = (a ^ b);         // XOR
            3'b110: s = (~a);            // NOT(A)
            3'b111: s = (shift_dir ? (a >> 1) : (a << 1)); // SHIFT(A)
            default: s = 8'b0;
        endcase
    end

    assign zero = (s == 8'b0);
    assign sign = s[7];

endmodule
```

Figure 87. Verilog HDL code of the ALU

Verification Checklist

Table 1. Design Verification Checklist of the Implemented ALU and Sub-Blocks

Block / Flag	**RTL**	**Schematic**	**Testbench**	**Layout**	**Post-Layout**
Adder	✓	✓	✓	✓	✓
Subtractor	✓	✓	✓	✓	✓
Multiplier	✓	✓	✓	✓	✓
AND Gate	✓	✓	✓	✓	✓
OR Gate	✓	✓	✓	✓	✓
XOR Gate	✓	✓	✓	✓	✓
NOT Gate	✓	✓	✓	✓	✓
Shifter	✓	✓	✓	✓	✓
Opcode	-	✓	-	✓	-
Carry Flag	-	-	-	-	-
Zero Flag	-	✓	–	✓	–
Sign Flag	-	-	–	-	–
Overflow Flag	–	–	–	–	–
8-bit ALU	✓	-	-	-	-

The Carry flag is generated and observed during both addition and subtraction operations, as it is directly related to arithmetic carry and borrow behavior. Therefore, the Carry flag is verified through the adder and subtractor blocks.

The Zero flag is derived from the ALU output bus and indicates whether the result is equal to zero. The Zero flag logic is visible in the layout; however, a dedicated testbench and post-layout simulation were not performed for this flag.

The Sign flag is generated by monitoring the most significant bit of the output operand. In this design, the Sign flag is obtained by checking the **A[7]** bit, which represents the sign of the result in two's complement representation.

REFERENCES

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed., Morgan Kaufmann, 2014.

[2] A. S. Sedra and K. C. Smith, *Microelectronic Circuits*, 7th ed., Oxford University Press, 2014.

[3] 101 Computing, "Binary Shifters Using Logic Gates," Available at: https://www.101computing.net/binary-shifters-using-logic-gates/